Reinforcement Learning Algorithms and PySCF

Table of Contents

Asynchronous Methods for Deep Reinforcement Learning **2**

Supplementary Material for "Asynchronous Methods for Deep Reinforcement Learning" **10**

Proximal Policy Optimization Algorithms **19**

Recurrent Neural Networks and Long Short-Term Memory Networks: Tutorial and Survey **30**

Human-level control through deep reinforcement learning **43**

OpenAI Baselines: ACKTR & A2C **55**

RLtools: A Fast, Portable Deep Reinforcement Learning Library for Continuous Control **61**

Prioritized Level Replay **77**

PySCF: The Python-based Simulations of Chemistry Framework **101**

END **124**

Asynchronous Methods for Deep Reinforcement Learning

Volodymyr Mnih[1] VMNIH@GOOGLE.COM
Adrià Puigdomènech Badia[1] ADRIAP@GOOGLE.COM
Mehdi Mirza[1,2] MIRZAMOM@IRO.UMONTREAL.CA
Alex Graves[1] GRAVESA@GOOGLE.COM
Tim Harley[1] THARLEY@GOOGLE.COM
Timothy P. Lillicrap[1] COUNTZERO@GOOGLE.COM
David Silver[1] DAVIDSILVER@GOOGLE.COM
Koray Kavukcuoglu [1] KORAYK@GOOGLE.COM

[1] Google DeepMind
[2] Montreal Institute for Learning Algorithms (MILA), University of Montreal

Abstract

We propose a conceptually simple and lightweight framework for deep reinforcement learning that uses asynchronous gradient descent for optimization of deep neural network controllers. We present asynchronous variants of four standard reinforcement learning algorithms and show that parallel actor-learners have a stabilizing effect on training allowing all four methods to successfully train neural network controllers. The best performing method, an asynchronous variant of actor-critic, surpasses the current state-of-the-art on the Atari domain while training for half the time on a single multi-core CPU instead of a GPU. Furthermore, we show that asynchronous actor-critic succeeds on a wide variety of continuous motor control problems as well as on a new task of navigating random 3D mazes using a visual input.

1. Introduction

Deep neural networks provide rich representations that can enable reinforcement learning (RL) algorithms to perform effectively. However, it was previously thought that the combination of simple online RL algorithms with deep neural networks was fundamentally unstable. Instead, a variety of solutions have been proposed to stabilize the algorithm (Riedmiller, 2005; Mnih et al., 2013; 2015; Van Hasselt et al., 2015; Schulman et al., 2015a). These approaches share a common idea: the sequence of observed data encountered by an online RL agent is non-stationary, and online RL updates are strongly correlated. By storing the agent's data in an experience replay memory, the data can be batched (Riedmiller, 2005; Schulman et al., 2015a) or randomly sampled (Mnih et al., 2013; 2015; Van Hasselt et al., 2015) from different time-steps. Aggregating over memory in this way reduces non-stationarity and decorrelates updates, but at the same time limits the methods to off-policy reinforcement learning algorithms.

Deep RL algorithms based on experience replay have achieved unprecedented success in challenging domains such as Atari 2600. However, experience replay has several drawbacks: it uses more memory and computation per real interaction; and it requires off-policy learning algorithms that can update from data generated by an older policy.

In this paper we provide a very different paradigm for deep reinforcement learning. Instead of experience replay, we asynchronously execute multiple agents in parallel, on multiple instances of the environment. This parallelism also decorrelates the agents' data into a more stationary process, since at any given time-step the parallel agents will be experiencing a variety of different states. This simple idea enables a much larger spectrum of fundamental on-policy RL algorithms, such as Sarsa, n-step methods, and actor-critic methods, as well as off-policy RL algorithms such as Q-learning, to be applied robustly and effectively using deep neural networks.

Our parallel reinforcement learning paradigm also offers practical benefits. Whereas previous approaches to deep reinforcement learning rely heavily on specialized hardware such as GPUs (Mnih et al., 2015; Van Hasselt et al., 2015; Schaul et al., 2015) or massively distributed architectures (Nair et al., 2015), our experiments run on a single machine with a standard multi-core CPU. When applied to a variety of Atari 2600 domains, on many games asynchronous reinforcement learning achieves better results, in far less

Proceedings of the 33^{rd} International Conference on Machine Learning, New York, NY, USA, 2016. JMLR: W&CP volume 48.

time than previous GPU-based algorithms, using far less resource than massively distributed approaches. The best of the proposed methods, asynchronous advantage actor-critic (A3C), also mastered a variety of continuous motor control tasks as well as learned general strategies for exploring 3D mazes purely from visual inputs. We believe that the success of A3C on both 2D and 3D games, discrete and continuous action spaces, as well as its ability to train feedforward and recurrent agents makes it the most general and successful reinforcement learning agent to date.

2. Related Work

The General Reinforcement Learning Architecture (Gorila) of (Nair et al., 2015) performs asynchronous training of reinforcement learning agents in a distributed setting. In Gorila, each process contains an actor that acts in its own copy of the environment, a separate replay memory, and a learner that samples data from the replay memory and computes gradients of the DQN loss (Mnih et al., 2015) with respect to the policy parameters. The gradients are asynchronously sent to a central parameter server which updates a central copy of the model. The updated policy parameters are sent to the actor-learners at fixed intervals. By using 100 separate actor-learner processes and 30 parameter server instances, a total of 130 machines, Gorila was able to significantly outperform DQN over 49 Atari games. On many games Gorila reached the score achieved by DQN over 20 times faster than DQN. We also note that a similar way of parallelizing DQN was proposed by (Chavez et al., 2015).

In earlier work, (Li & Schuurmans, 2011) applied the Map Reduce framework to parallelizing batch reinforcement learning methods with linear function approximation. Parallelism was used to speed up large matrix operations but not to parallelize the collection of experience or stabilize learning. (Grounds & Kudenko, 2008) proposed a parallel version of the Sarsa algorithm that uses multiple separate actor-learners to accelerate training. Each actor-learner learns separately and periodically sends updates to weights that have changed significantly to the other learners using peer-to-peer communication.

(Tsitsiklis, 1994) studied convergence properties of Q-learning in the asynchronous optimization setting. These results show that Q-learning is still guaranteed to converge when some of the information is outdated as long as outdated information is always eventually discarded and several other technical assumptions are satisfied. Even earlier, (Bertsekas, 1982) studied the related problem of distributed dynamic programming.

Another related area of work is in evolutionary methods, which are often straightforward to parallelize by distributing fitness evaluations over multiple machines or threads (Tomassini, 1999). Such parallel evolutionary approaches have recently been applied to some visual reinforcement learning tasks. In one example, (Koutník et al., 2014) evolved convolutional neural network controllers for the TORCS driving simulator by performing fitness evaluations on 8 CPU cores in parallel.

3. Reinforcement Learning Background

We consider the standard reinforcement learning setting where an agent interacts with an environment $\mathcal{E}$ over a number of discrete time steps. At each time step t, the agent receives a state s_t and selects an action a_t from some set of possible actions $\mathcal{A}$ according to its policy π, where π is a mapping from states s_t to actions a_t. In return, the agent receives the next state s_{t+1} and receives a scalar reward r_t. The process continues until the agent reaches a terminal state after which the process restarts. The return $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ is the total accumulated return from time step t with discount factor $\gamma \in (0, 1]$. The goal of the agent is to maximize the expected return from each state s_t.

The action value $Q^\pi(s, a) = \mathbb{E}\left[R_t | s_t = s, a\right]$ is the expected return for selecting action a in state s and following policy π. The optimal value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ gives the maximum action value for state s and action a achievable by any policy. Similarly, the value of state s under policy π is defined as $V^\pi(s) = \mathbb{E}\left[R_t | s_t = s\right]$ and is simply the expected return for following policy π from state s.

In value-based model-free reinforcement learning methods, the action value function is represented using a function approximator, such as a neural network. Let $Q(s, a; \theta)$ be an approximate action-value function with parameters θ. The updates to θ can be derived from a variety of reinforcement learning algorithms. One example of such an algorithm is Q-learning, which aims to directly approximate the optimal action value function: $Q^*(s, a) \approx Q(s, a; \theta)$. In one-step Q-learning, the parameters θ of the action value function $Q(s, a; \theta)$ are learned by iteratively minimizing a sequence of loss functions, where the ith loss function defined as

$$L_i(\theta_i) = \mathbb{E}\left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right)^2$$

where s' is the state encountered after state s.

We refer to the above method as one-step Q-learning because it updates the action value $Q(s, a)$ toward the one-step return $r + \gamma \max_{a'} Q(s', a'; \theta)$. One drawback of using one-step methods is that obtaining a reward r only directly affects the value of the state action pair s, a that led to the reward. The values of other state action pairs are affected only indirectly through the updated value $Q(s, a)$. This can make the learning process slow since many updates are required the propagate a reward to the relevant preceding states and actions.

One way of propagating rewards faster is by using n-step returns (Watkins, 1989; Peng & Williams, 1996). In n-step Q-learning, $Q(s,a)$ is updated toward the n-step return defined as $r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \max_a \gamma^n Q(s_{t+n}, a)$. This results in a single reward r directly affecting the values of n preceding state action pairs. This makes the process of propagating rewards to relevant state-action pairs potentially much more efficient.

In contrast to value-based methods, policy-based model-free methods directly parameterize the policy $\pi(a|s;\theta)$ and update the parameters θ by performing, typically approximate, gradient ascent on $\mathbb{E}[R_t]$. One example of such a method is the REINFORCE family of algorithms due to Williams (1992). Standard REINFORCE updates the policy parameters θ in the direction $\nabla_\theta \log \pi(a_t|s_t;\theta) R_t$, which is an unbiased estimate of $\nabla_\theta \mathbb{E}[R_t]$. It is possible to reduce the variance of this estimate while keeping it unbiased by subtracting a learned function of the state $b_t(s_t)$, known as a baseline (Williams, 1992), from the return. The resulting gradient is $\nabla_\theta \log \pi(a_t|s_t;\theta)\,(R_t - b_t(s_t))$.

A learned estimate of the value function is commonly used as the baseline $b_t(s_t) \approx V^\pi(s_t)$ leading to a much lower variance estimate of the policy gradient. When an approximate value function is used as the baseline, the quantity $R_t - b_t$ used to scale the policy gradient can be seen as an estimate of the *advantage* of action a_t in state s_t, or $A(a_t, s_t) = Q(a_t, s_t) - V(s_t)$, because R_t is an estimate of $Q^\pi(a_t, s_t)$ and b_t is an estimate of $V^\pi(s_t)$. This approach can be viewed as an actor-critic architecture where the policy π is the actor and the baseline b_t is the critic (Sutton & Barto, 1998; Degris et al., 2012).

4. Asynchronous RL Framework

We now present multi-threaded asynchronous variants of one-step Sarsa, one-step Q-learning, n-step Q-learning, and advantage actor-critic. The aim in designing these methods was to find RL algorithms that can train deep neural network policies reliably and without large resource requirements. While the underlying RL methods are quite different, with actor-critic being an on-policy policy search method and Q-learning being an off-policy value-based method, we use two main ideas to make all four algorithms practical given our design goal.

First, we use asynchronous actor-learners, similarly to the Gorila framework (Nair et al., 2015), but instead of using separate machines and a parameter server, we use multiple CPU threads on a single machine. Keeping the learners on a single machine removes the communication costs of sending gradients and parameters and enables us to use Hogwild! (Recht et al., 2011) style updates for training.

Second, we make the observation that multiple actor-learners running in parallel are likely to be exploring different parts of the environment. Moreover, one can explicitly use different exploration policies in each actor-learner to maximize this diversity. By running different exploration policies in different threads, the overall changes being made to the parameters by multiple actor-learners applying online updates in parallel are likely to be less correlated in time than a single agent applying online updates. Hence, we do not use a replay memory and rely on parallel actors employing different exploration policies to perform the stabilizing role undertaken by experience replay in the DQN training algorithm.

Algorithm 1 Asynchronous one-step Q-learning - pseudocode for each actor-learner thread.

// Assume global shared θ, θ^-, and counter $T = 0$.
Initialize thread step counter $t \leftarrow 0$
Initialize target network weights $\theta^- \leftarrow \theta$
Initialize network gradients $d\theta \leftarrow 0$
Get initial state s
repeat
  Take action a with ϵ-greedy policy based on $Q(s, a; \theta)$
  Receive new state s' and reward r
  $y = \begin{cases} r & \text{for terminal } s' \\ r + \gamma \max_{a'} Q(s', a'; \theta^-) & \text{for non-terminal } s' \end{cases}$
  Accumulate gradients wrt θ: $d\theta \leftarrow d\theta + \frac{\partial (y - Q(s,a;\theta))^2}{\partial \theta}$
  $s = s'$
  $T \leftarrow T + 1$ and $t \leftarrow t + 1$
  if $T \mod I_{target} == 0$ **then**
    Update the target network $\theta^- \leftarrow \theta$
  end if
  if $t \mod I_{AsyncUpdate} == 0$ or s is terminal **then**
    Perform asynchronous update of θ using $d\theta$.
    Clear gradients $d\theta \leftarrow 0$.
  end if
until $T > T_{max}$

In addition to stabilizing learning, using multiple parallel actor-learners has multiple practical benefits. First, we obtain a reduction in training time that is roughly linear in the number of parallel actor-learners. Second, since we no longer rely on experience replay for stabilizing learning we are able to use on-policy reinforcement learning methods such as Sarsa and actor-critic to train neural networks in a stable way. We now describe our variants of one-step Q-learning, one-step Sarsa, n-step Q-learning and advantage actor-critic.

Asynchronous one-step Q-learning: Pseudocode for our variant of Q-learning, which we call Asynchronous one-step Q-learning, is shown in Algorithm 1. Each thread interacts with its own copy of the environment and at each step computes a gradient of the Q-learning loss. We use a shared and slowly changing target network in computing the Q-learning loss, as was proposed in the DQN training method. We also accumulate gradients over multiple timesteps before they are applied, which is similar to us-

ing minibatches. This reduces the chances of multiple actor learners overwriting each other's updates. Accumulating updates over several steps also provides some ability to trade off computational efficiency for data efficiency.

Finally, we found that giving each thread a different exploration policy helps improve robustness. Adding diversity to exploration in this manner also generally improves performance through better exploration. While there are many possible ways of making the exploration policies differ we experiment with using ϵ-greedy exploration with ϵ periodically sampled from some distribution by each thread.

Asynchronous one-step Sarsa: The asynchronous one-step Sarsa algorithm is the same as asynchronous one-step Q-learning as given in Algorithm 1 except that it uses a different target value for $Q(s,a)$. The target value used by one-step Sarsa is $r + \gamma Q(s', a'; \theta^-)$ where a' is the action taken in state s' (Rummery & Niranjan, 1994; Sutton & Barto, 1998). We again use a target network and updates accumulated over multiple timesteps to stabilize learning.

Asynchronous n-step Q-learning: Pseudocode for our variant of multi-step Q-learning is shown in Supplementary Algorithm S2. The algorithm is somewhat unusual because it operates in the forward view by explicitly computing n-step returns, as opposed to the more common backward view used by techniques like eligibility traces (Sutton & Barto, 1998). We found that using the forward view is easier when training neural networks with momentum-based methods and backpropagation through time. In order to compute a single update, the algorithm first selects actions using its exploration policy for up to t_{max} steps or until a terminal state is reached. This process results in the agent receiving up to t_{max} rewards from the environment since its last update. The algorithm then computes gradients for n-step Q-learning updates for each of the state-action pairs encountered since the last update. Each n-step update uses the longest possible n-step return resulting in a one-step update for the last state, a two-step update for the second last state, and so on for a total of up to t_{max} updates. The accumulated updates are applied in a single gradient step.

Asynchronous advantage actor-critic: The algorithm, which we call asynchronous advantage actor-critic (A3C), maintains a policy $\pi(a_t|s_t;\theta)$ and an estimate of the value function $V(s_t;\theta_v)$. Like our variant of n-step Q-learning, our variant of actor-critic also operates in the forward view and uses the same mix of n-step returns to update both the policy and the value-function. The policy and the value function are updated after every t_{max} actions or when a terminal state is reached. The update performed by the algorithm can be seen as $\nabla_{\theta'} \log \pi(a_t|s_t;\theta') A(s_t, a_t; \theta, \theta_v)$ where $A(s_t, a_t; \theta, \theta_v)$ is an estimate of the advantage function given by $\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k};\theta_v) - V(s_t;\theta_v)$, where k can vary from state to state and is upper-bounded by t_{max}. The pseudocode for the algorithm is presented in Supplementary Algorithm S3.

As with the value-based methods we rely on parallel actor-learners and accumulated updates for improving training stability. Note that while the parameters θ of the policy and θ_v of the value function are shown as being separate for generality, we always share some of the parameters in practice. We typically use a convolutional neural network that has one softmax output for the policy $\pi(a_t|s_t;\theta)$ and one linear output for the value function $V(s_t;\theta_v)$, with all non-output layers shared.

We also found that adding the entropy of the policy π to the objective function improved exploration by discouraging premature convergence to suboptimal deterministic policies. This technique was originally proposed by (Williams & Peng, 1991), who found that it was particularly helpful on tasks requiring hierarchical behavior. The gradient of the full objective function including the entropy regularization term with respect to the policy parameters takes the form $\nabla_{\theta'} \log \pi(a_t|s_t;\theta')(R_t - V(s_t;\theta_v)) + \beta \nabla_{\theta'} H(\pi(s_t;\theta'))$, where H is the entropy. The hyperparameter β controls the strength of the entropy regularization term.

Optimization: We investigated three different optimization algorithms in our asynchronous framework – SGD with momentum, RMSProp (Tieleman & Hinton, 2012) without shared statistics, and RMSProp with shared statistics. We used the standard non-centered RMSProp update given by

$$g = \alpha g + (1-\alpha)\Delta\theta^2 \text{ and } \theta \leftarrow \theta - \eta \frac{\Delta\theta}{\sqrt{g+\epsilon}}, \qquad (1)$$

where all operations are performed elementwise. A comparison on a subset of Atari 2600 games showed that a variant of RMSProp where statistics g are shared across threads is considerably more robust than the other two methods. Full details of the methods and comparisons are included in Supplementary Section 7.

5. Experiments

We use four different platforms for assessing the properties of the proposed framework. We perform most of our experiments using the Arcade Learning Environment (Bellemare et al., 2012), which provides a simulator for Atari 2600 games. This is one of the most commonly used benchmark environments for RL algorithms. We use the Atari domain to compare against state of the art results (Van Hasselt et al., 2015; Wang et al., 2015; Schaul et al., 2015; Nair et al., 2015; Mnih et al., 2015), as well as to carry out a detailed stability and scalability analysis of the proposed methods. We performed further comparisons using the TORCS 3D car racing simulator (Wymann et al., 2013). We also use

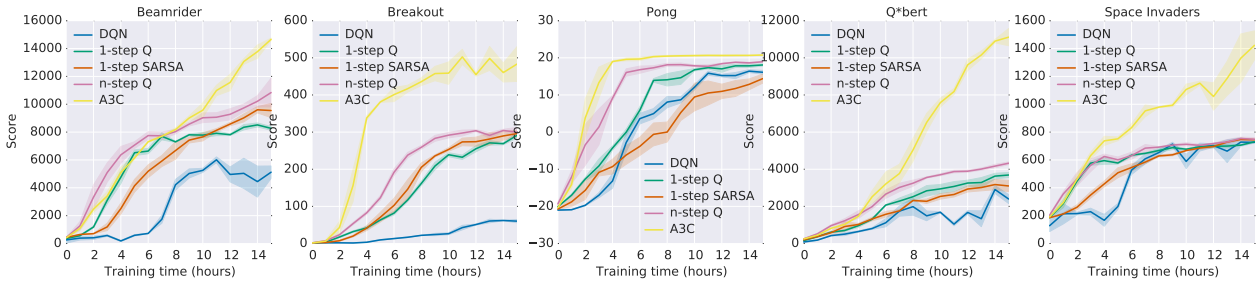


Figure 1. Learning speed comparison for DQN and the new asynchronous algorithms on five Atari 2600 games. DQN was trained on a single Nvidia K40 GPU while the asynchronous methods were trained using 16 CPU cores. The plots are averaged over 5 runs. In the case of DQN the runs were for different seeds with fixed hyperparameters. For asynchronous methods we average over the best 5 models from 50 experiments with learning rates sampled from $LogUniform(10^{-4}, 10^{-2})$ and all other hyperparameters fixed.

two additional domains to evaluate only the A3C algorithm – Mujoco and Labyrinth. MuJoCo (Todorov, 2015) is a physics simulator for evaluating agents on continuous motor control tasks with contact dynamics. Labyrinth is a new 3D environment where the agent must learn to find rewards in randomly generated mazes from a visual input. The precise details of our experimental setup can be found in Supplementary Section 8.

5.1. Atari 2600 Games

We first present results on a subset of Atari 2600 games to demonstrate the training speed of the new methods. Figure 1 compares the learning speed of the DQN algorithm trained on an Nvidia K40 GPU with the asynchronous methods trained using 16 CPU cores on five Atari 2600 games. The results show that all four asynchronous methods we presented can successfully train neural network controllers on the Atari domain. The asynchronous methods tend to learn faster than DQN, with significantly faster learning on some games, while training on only 16 CPU cores. Additionally, the results suggest that n-step methods learn faster than one-step methods on some games. Overall, the policy-based advantage actor-critic method significantly outperforms all three value-based methods.

We then evaluated asynchronous advantage actor-critic on 57 Atari games. In order to compare with the state of the art in Atari game playing, we largely followed the training and evaluation protocol of (Van Hasselt et al., 2015). Specifically, we tuned hyperparameters (learning rate and amount of gradient norm clipping) using a search on six Atari games (Beamrider, Breakout, Pong, Q*bert, Seaquest and Space Invaders) and then fixed all hyperparameters for all 57 games. We trained both a feedforward agent with the same architecture as (Mnih et al., 2015; Nair et al., 2015; Van Hasselt et al., 2015) as well as a recurrent agent with an additional 256 LSTM cells after the final hidden layer. We additionally used the final network weights for evaluation to make the results more comparable to the original results

Method	Training Time	Mean	Median
DQN	8 days on GPU	121.9%	47.5%
Gorila	4 days, 100 machines	215.2%	71.3%
D-DQN	8 days on GPU	332.9%	110.9%
Dueling D-DQN	8 days on GPU	343.8%	117.1%
Prioritized DQN	8 days on GPU	463.6%	127.6%
A3C, FF	1 day on CPU	344.1%	68.2%
A3C, FF	4 days on CPU	496.8%	116.6%
A3C, LSTM	4 days on CPU	623.0%	112.6%

Table 1. Mean and median human-normalized scores on 57 Atari games using the human starts evaluation metric. Supplementary Table S3 shows the raw scores for all games.

from (Bellemare et al., 2012). We trained our agents for four days using 16 CPU cores, while the other agents were trained for 8 to 10 days on Nvidia K40 GPUs. Table 1 shows the average and median human-normalized scores obtained by our agents trained by asynchronous advantage actor-critic (A3C) as well as the current state-of-the art. Supplementary Table S3 shows the scores on all games. A3C significantly improves on state-of-the-art the average score over 57 games in half the training time of the other methods while using only 16 CPU cores and no GPU. Furthermore, after just one day of training, A3C matches the average human normalized score of Dueling Double DQN and almost reaches the median human normalized score of Gorila. We note that many of the improvements that are presented in Double DQN (Van Hasselt et al., 2015) and Dueling Double DQN (Wang et al., 2015) can be incorporated to 1-step Q and n-step Q methods presented in this work with similar potential improvements.

5.2. TORCS Car Racing Simulator

We also compared the four asynchronous methods on the TORCS 3D car racing game (Wymann et al., 2013). TORCS not only has more realistic graphics than Atari 2600 games, but also requires the agent to learn the dynamics of the car it is controlling. At each step, an agent received only a visual input in the form of an RGB image

of the current frame as well as a reward proportional to the agent's velocity along the center of the track at the agent's current position. We used the same neural network architecture as the one used in the Atari experiments specified in Supplementary Section 8. We performed experiments using four different settings – the agent controlling a slow car with and without opponent bots, and the agent controlling a fast car with and without opponent bots. Full results can be found in Supplementary Figure S6. A3C was the best performing agent, reaching between roughly 75% and 90% of the score obtained by a human tester on all four game configurations in about 12 hours of training. A video showing the learned driving behavior of the A3C agent can be found at https://youtu.be/0xo1Ldx3L5Q.

5.3. Continuous Action Control Using the MuJoCo Physics Simulator

We also examined a set of tasks where the action space is continuous. In particular, we looked at a set of rigid body physics domains with contact dynamics where the tasks include many examples of manipulation and locomotion. These tasks were simulated using the Mujoco physics engine. We evaluated only the asynchronous advantage actor-critic algorithm since, unlike the value-based methods, it is easily extended to continuous actions. In all problems, using either the physical state or pixels as input, Asynchronous Advantage-Critic found good solutions in less than 24 hours of training and typically in under a few hours. Some successful policies learned by our agent can be seen in the following video https://youtu.be/Ajjc08-iPx8. Further details about this experiment can be found in Supplementary Section 9.

5.4. Labyrinth

We performed an additional set of experiments with A3C on a new 3D environment called Labyrinth. The specific task we considered involved the agent learning to find rewards in randomly generated mazes. At the beginning of each episode the agent was placed in a new randomly generated maze consisting of rooms and corridors. Each maze contained two types of objects that the agent was rewarded for finding – apples and portals. Picking up an apple led to a reward of 1. Entering a portal led to a reward of 10 after which the agent was respawned in a new random location in the maze and all previously collected apples were regenerated. An episode terminated after 60 seconds after which a new episode would begin. The aim of the agent is to collect as many points as possible in the time limit and the optimal strategy involves first finding the portal and then repeatedly going back to it after each respawn. This task is much more challenging than the TORCS driving domain because the agent is faced with a new maze in each episode and must learn a general strategy for exploring random mazes.

	Number of threads				
Method	1	2	4	8	16
1-step Q	1.0	**3.0**	**6.3**	**13.3**	**24.1**
1-step SARSA	1.0	**2.8**	**5.9**	**13.1**	**22.1**
n-step Q	1.0	**2.7**	**5.9**	**10.7**	**17.2**
A3C	1.0	2.1	3.7	6.9	12.5

Table 2. The average training speedup for each method and number of threads averaged over seven Atari games. To compute the training speed-up on a single game we measured the time to required reach a fixed reference score using each method and number of threads. The speedup from using n threads on a game was defined as the time required to reach a fixed reference score using one thread divided the time required to reach the reference score using n threads. The table shows the speedups averaged over seven Atari games (Beamrider, Breakout, Enduro, Pong, Q*bert, Seaquest, and Space Invaders).

We trained an A3C LSTM agent on this task using only 84×84 RGB images as input. The final average score of around 50 indicates that the agent learned a reasonable strategy for exploring random 3D maxes using only a visual input. A video showing one of the agents exploring previously unseen mazes is included at https://youtu.be/nMR5mjCFZCw.

5.5. Scalability and Data Efficiency

We analyzed the effectiveness of our proposed framework by looking at how the training time and data efficiency changes with the number of parallel actor-learners. When using multiple workers in parallel and updating a shared model, one would expect that in an ideal case, for a given task and algorithm, the number of training steps to achieve a certain score would remain the same with varying numbers of workers. Therefore, the advantage would be solely due to the ability of the system to consume more data in the same amount of wall clock time and possibly improved exploration. Table 2 shows the training speed-up achieved by using increasing numbers of parallel actor-learners averaged over seven Atari games. These results show that all four methods achieve substantial speedups from using multiple worker threads, with 16 threads leading to at least an order of magnitude speedup. This confirms that our proposed framework scales well with the number of parallel workers, making efficient use of resources.

Somewhat surprisingly, asynchronous one-step Q-learning and Sarsa algorithms exhibit superlinear speedups that cannot be explained by purely computational gains. We observe that one-step methods (one-step Q and one-step Sarsa) often require less data to achieve a particular score when using more parallel actor-learners. We believe this is due to positive effect of multiple threads to reduce the bias in one-step methods. These effects are shown more clearly in Figure 3, which shows plots of the average score against the total number of training frames for different

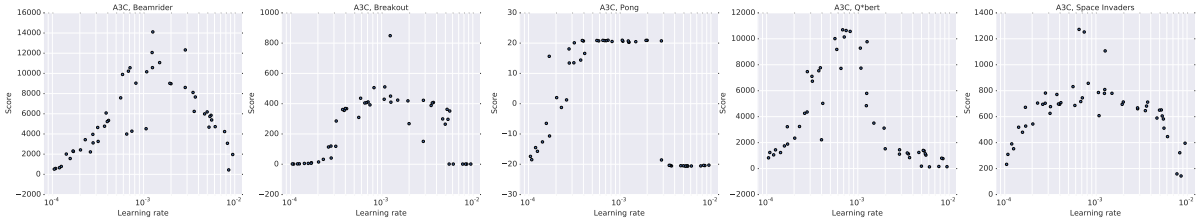


Figure 2. Scatter plots of scores obtained by asynchronous advantage actor-critic on five games (Beamrider, Breakout, Pong, Q*bert, Space Invaders) for 50 different learning rates and random initializations. On each game, there is a wide range of learning rates for which all random initializations acheive good scores. This shows that A3C is quite robust to learning rates and initial random weights.

numbers of actor-learners and training methods on five Atari games, and Figure 4, which shows plots of the average score against wall-clock time.

5.6. Robustness and Stability

Finally, we analyzed the stability and robustness of the four proposed asynchronous algorithms. For each of the four algorithms we trained models on five games (Breakout, Beamrider, Pong, Q*bert, Space Invaders) using 50 different learning rates and random initializations. Figure 2 shows scatter plots of the resulting scores for A3C, while Supplementary Figure S11 shows plots for the other three methods. There is usually a range of learning rates for each method and game combination that leads to good scores, indicating that all methods are quite robust to the choice of learning rate and random initialization. The fact that there are virtually no points with scores of 0 in regions with good learning rates indicates that the methods are stable and do not collapse or diverge once they are learning.

6. Conclusions and Discussion

We have presented asynchronous versions of four standard reinforcement learning algorithms and showed that they are able to train neural network controllers on a variety of domains in a stable manner. Our results show that in our proposed framework stable training of neural networks through reinforcement learning is possible with both value-based and policy-based methods, off-policy as well as on-policy methods, and in discrete as well as continuous domains. When trained on the Atari domain using 16 CPU cores, the proposed asynchronous algorithms train faster than DQN trained on an Nvidia K40 GPU, with A3C surpassing the current state-of-the-art in half the training time.

One of our main findings is that using parallel actor-learners to update a shared model had a stabilizing effect on the learning process of the three value-based methods we considered. While this shows that stable online Q-learning is possible without experience replay, which was used for this purpose in DQN, it does not mean that experience replay is not useful. Incorporating experience replay into the asynchronous reinforcement learning framework could substantially improve the data efficiency of these methods by reusing old data. This could in turn lead to much faster training times in domains like TORCS where interacting with the environment is more expensive than updating the model for the architecture we used.

Combining other existing reinforcement learning methods or recent advances in deep reinforcement learning with our asynchronous framework presents many possibilities for immediate improvements to the methods we presented. While our n-step methods operate in the *forward view* (Sutton & Barto, 1998) by using corrected n-step returns directly as targets, it has been more common to use the *backward view* to implicitly combine different returns through eligibility traces (Watkins, 1989; Sutton & Barto, 1998; Peng & Williams, 1996). The asynchronous advantage actor-critic method could be potentially improved by using other ways of estimating the advantage function, such as generalized advantage estimation of (Schulman et al., 2015b). All of the value-based methods we investigated could benefit from different ways of reducing overestimation bias of Q-values (Van Hasselt et al., 2015; Bellemare et al., 2016). Yet another, more speculative, direction is to try and combine the recent work on true online temporal difference methods (van Seijen et al., 2015) with non-linear function approximation.

In addition to these algorithmic improvements, a number of complementary improvements to the neural network architecture are possible. The dueling architecture of (Wang et al., 2015) has been shown to produce more accurate estimates of Q-values by including separate streams for the state value and advantage in the network. The spatial softmax proposed by (Levine et al., 2015) could improve both value-based and policy-based methods by making it easier for the network to represent feature coordinates.

ACKNOWLEDGMENTS

We thank Thomas Degris, Remi Munos, Marc Lanctot, Sasha Vezhnevets and Joseph Modayil for many helpful discussions, suggestions and comments on the paper. We also thank the DeepMind evaluation team for setting up the environments used to evaluate the agents in the paper.

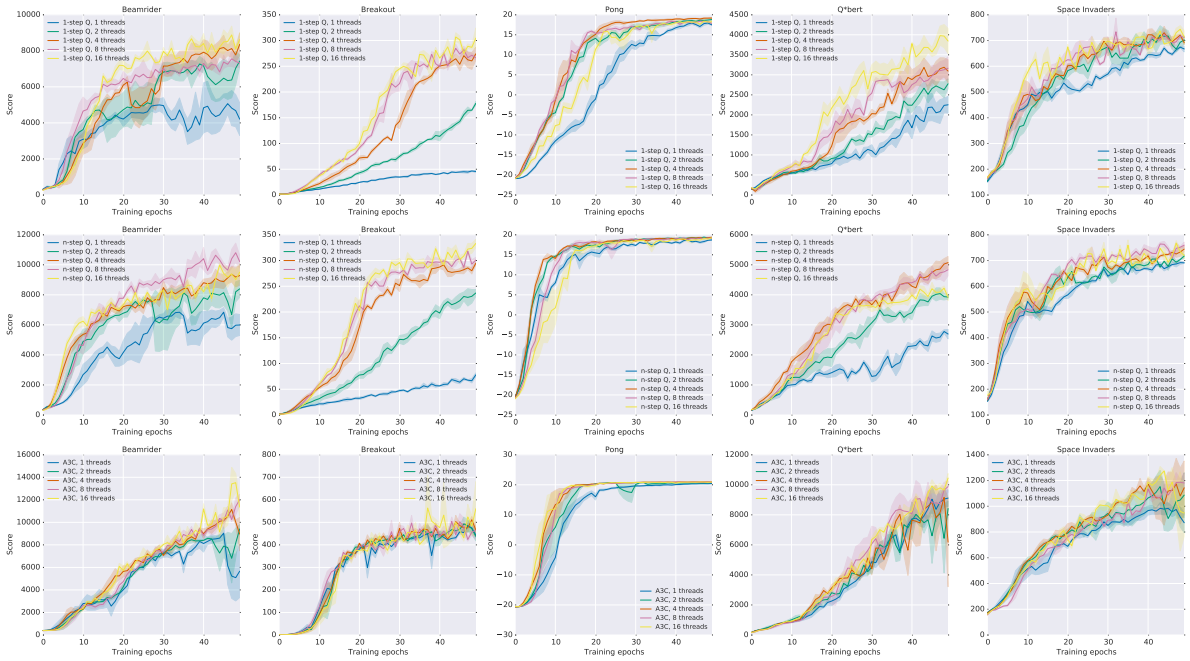


Figure 3. Data efficiency comparison of different numbers of actor-learners for three asynchronous methods on five Atari games. The x-axis shows the total number of training epochs where an epoch corresponds to four million frames (across all threads). The y-axis shows the average score. Each curve shows the average over the three best learning rates. Single step methods show increased data efficiency from more parallel workers. Results for Sarsa are shown in Supplementary Figure S9.

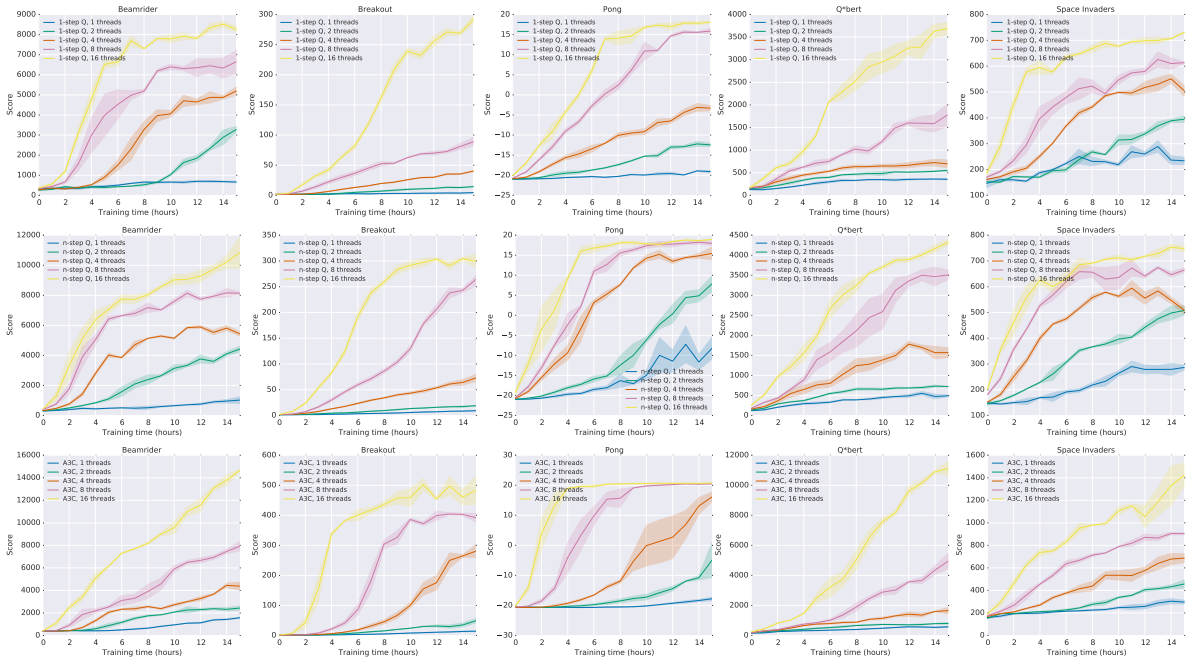


Figure 4. Training speed comparison of different numbers of actor-learners on five Atari games. The x-axis shows training time in hours while the y-axis shows the average score. Each curve shows the average over the three best learning rates. All asynchronous methods show significant speedups from using greater numbers of parallel actor-learners. Results for Sarsa are shown in Supplementary Figure S10.

Supplementary Material for "Asynchronous Methods for Deep Reinforcement Learning"

June 17, 2016

7. Optimization Details

We investigated two different optimization algorithms with our asynchronous framework – stochastic gradient descent and RMSProp. Our implementations of these algorithms do not use any locking in order to maximize throughput when using a large number of threads.

Momentum SGD: The implementation of SGD in an asynchronous setting is relatively straightforward and well studied (Recht et al., 2011). Let θ be the parameter vector that is shared across all threads and let $\Delta\theta_i$ be the accumulated gradients of the loss with respect to parameters θ computed by thread number i. Each thread i independently applies the standard momentum SGD update $m_i = \alpha m_i + (1-\alpha)\Delta\theta_i$ followed by $\theta \leftarrow \theta - \eta m_i$ with learning rate η, momentum α and without any locks. Note that in this setting, each thread maintains its own separate gradient and momentum vector.

RMSProp: While RMSProp (Tieleman & Hinton, 2012) has been widely used in the deep learning literature, it has not been extensively studied in the asynchronous optimization setting. The standard non-centered RMSProp update is given by

$$g = \alpha g + (1-\alpha)\Delta\theta^2 \tag{S2}$$

$$\theta \leftarrow \theta - \eta \frac{\Delta\theta}{\sqrt{g+\epsilon}}, \tag{S3}$$

where all operations are performed elementwise. In order to apply RMSProp in the asynchronous optimization setting one must decide whether the moving average of elementwise squared gradients g is shared or per-thread. We experimented with two versions of the algorithm. In one version, which we refer to as RMSProp, each thread maintains its own g shown in Equation S2. In the other version, which we call Shared RMSProp, the vector g is shared among threads and is updated asynchronously and without locking. Sharing statistics among threads also reduces memory requirements by using one fewer copy of the parameter vector per thread.

We compared these three asynchronous optimization algorithms in terms of their sensitivity to different learning rates and random network initializations. Figure S5 shows a comparison of the methods for two different reinforcement learning methods (Async n-step Q and Async Advantage Actor-Critic) on four different games (Breakout, Beamrider, Seaquest and Space Invaders). Each curve shows the scores for 50 experiments that correspond to 50 different random learning rates and initializations. The x-axis shows the rank of the model after sorting in descending order by final average score and the y-axis shows the final average score achieved by the corresponding model. In this representation, the algorithm that performs better would achieve higher maximum rewards on the y-axis and the algorithm that is most robust would have its slope closest to horizontal, thus maximizing the area under the curve. RMSProp with shared statistics tends to be more robust than RMSProp with per-thread statistics, which is in turn more robust than Momentum SGD.

8. Experimental Setup

The experiments performed on a subset of Atari games (Figures 1, 3, 4 and Table 2) as well as the TORCS experiments (Figure S6) used the following setup. Each experiment used 16 actor-learner threads running on a single machine and no GPUs. All methods performed updates after every 5 actions ($t_{max} = 5$ and $I_{Update} = 5$) and shared RMSProp was used for optimization. The three asynchronous value-based methods used a shared target network that was updated every 40000 frames. The Atari experiments used the same input preprocessing as (Mnih et al., 2015) and an action repeat of 4. The agents used the network architecture from (Mnih et al., 2013). The network used a convolutional layer with 16 filters of size 8×8 with stride 4, followed by a convolutional layer with with 32 filters of size 4×4 with stride 2, followed by a fully connected layer with 256 hidden units. All three hidden layers were followed by a rectifier nonlinearity. The value-based methods had a single linear output unit for each action representing the action-value. The model used by actor-critic agents had two set of outputs – a softmax output with one entry per action representing the probability of selecting the action, and a single linear output representing the value function. All experiments used a discount of $\gamma = 0.99$ and an RMSProp decay factor of $\alpha = 0.99$.

The value based methods sampled the exploration rate ϵ from a distribution taking three values $\epsilon_1, \epsilon_2, \epsilon_3$ with probabilities $0.4, 0.3, 0.3$. The values of $\epsilon_1, \epsilon_2, \epsilon_3$ were annealed from 1 to $0.1, 0.01, 0.5$ respectively over the first four million frames. Advantage actor-critic used entropy regularization with a weight $\beta = 0.01$ for all Atari and TORCS experiments. We performed a set of 50 experiments for five Atari games and every TORCS level, each using a different random initialization and initial learning rate. The initial learning rate was sampled from a $LogUniform(10^{-4}, 10^{-2})$ distribution and annealed to 0 over the course of training. Note that in comparisons to prior work (Tables 1 and S3) we followed standard evaluation protocol and used fixed hyperparameters.

9. Continuous Action Control Using the MuJoCo Physics Simulator

To apply the asynchronous advantage actor-critic algorithm to the Mujoco tasks the necessary setup is nearly identical to that used in the discrete action domains, so here we enumerate only the differences required for the continuous action domains. The essential elements for many of the tasks (i.e. the physics models and task objectives) are near identical to the tasks examined in (Lillicrap et al., 2015). However, the rewards and thus performance are not comparable for most of the tasks due to changes made by the developers of Mujoco which altered the contact model.

For all the domains we attempted to learn the task using the physical state as input. The physical state consisted of the joint positions and velocities as well as the target position if the task required a target. In addition, for three of the tasks (pendulum, pointmass2D, and gripper) we also examined training directly from RGB pixel inputs. In the low dimensional physical state case, the inputs are mapped to a hidden state using one hidden layer with 200 ReLU units. In the cases where we used pixels, the input was passed through two layers of spatial convolutions without any non-linearity or pooling. In either case, the output of the encoder layers were fed to a single layer of 128 LSTM cells. The most important difference in the architecture is in the the output layer of the policy network. Unlike the discrete action domain where the action output is a Softmax, here the two outputs of the policy network are two real number vectors which we treat as the mean vector μ and scalar variance σ^2 of a multidimensional normal distribution with a spherical covariance. To act, the input is passed through the model to the output layer where we sample from the normal distribution determined by μ and σ^2. In practice, μ is modeled by a linear layer and σ^2 by a SoftPlus operation, $\log(1 + \exp(x))$, as the activation computed as a function of the output of a linear layer. In our experiments with continuous control problems the networks for policy network and value network do not share any parameters, though this detail is unlikely to be crucial. Finally, since the episodes were typically at most several hundred time steps long, we did not use any bootstrapping in the policy or value function updates and batched each episode into a single update.

As in the discrete action case, we included an entropy cost which encouraged exploration. In the continuous

case the we used a cost on the differential entropy of the normal distribution defined by the output of the actor network, $-\frac{1}{2}(\log(2\pi\sigma^2)+1)$, we used a constant multiplier of 10^{-4} for this cost across all of the tasks examined. The asynchronous advantage actor-critic algorithm finds solutions for all the domains. Figure S8 shows learning curves against wall-clock time, and demonstrates that most of the domains from states can be solved within a few hours. All of the experiments, including those done from pixel based observations, were run on CPU. Even in the case of solving the domains directly from pixel inputs we found that it was possible to reliably discover solutions within 24 hours. Figure S7 shows scatter plots of the top scores against the sampled learning rates. In most of the domains there is large range of learning rates that consistently achieve good performance on the task.

Algorithm S2 Asynchronous n-step Q-learning - pseudocode for each actor-learner thread.

// Assume global shared parameter vector θ.
// Assume global shared target parameter vector θ^-.
// Assume global shared counter $T = 0$.
Initialize thread step counter $t \leftarrow 1$
Initialize target network parameters $\theta^- \leftarrow \theta$
Initialize thread-specific parameters $\theta' = \theta$
Initialize network gradients $d\theta \leftarrow 0$
repeat
  Clear gradients $d\theta \leftarrow 0$
  Synchronize thread-specific parameters $\theta' = \theta$
  $t_{start} = t$
  Get state s_t
  repeat
    Take action a_t according to the ϵ-greedy policy based on $Q(s_t, a; \theta')$
    Receive reward r_t and new state s_{t+1}
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  until terminal s_t **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ \max_a Q(s_t, a; \theta^-) & \text{for non-terminal } s_t \end{cases}$
  for $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \frac{\partial \left(R - Q(s_i, a_i; \theta')\right)^2}{\partial \theta'}$
  end for
  Perform asynchronous update of θ using $d\theta$.
  if $T \mod I_{target} == 0$ **then**
    $\theta^- \leftarrow \theta$
  end if
until $T > T_{max}$

Algorithm S3 Asynchronous advantage actor-critic - pseudocode for each actor-learner thread.

// Assume global shared parameter vectors θ and θ_v and global shared counter $T = 0$
// Assume thread-specific parameter vectors θ' and θ'_v
Initialize thread step counter $t \leftarrow 1$
repeat
  Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
  Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$
  $t_{start} = t$
  Get state s_t
  repeat
    Perform a_t according to policy $\pi(a_t|s_t;\theta')$
    Receive reward r_t and new state s_{t+1}
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  until terminal s_t **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t \text{// Bootstrap from last state} \end{cases}$
  for $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i;\theta')(R - V(s_i;\theta'_v))$
    Accumulate gradients wrt θ'_v: $d\theta_v \leftarrow d\theta_v + \partial \left(R - V(s_i;\theta'_v)\right)^2 / \partial \theta'_v$
  end for
  Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.
until $T > T_{max}$

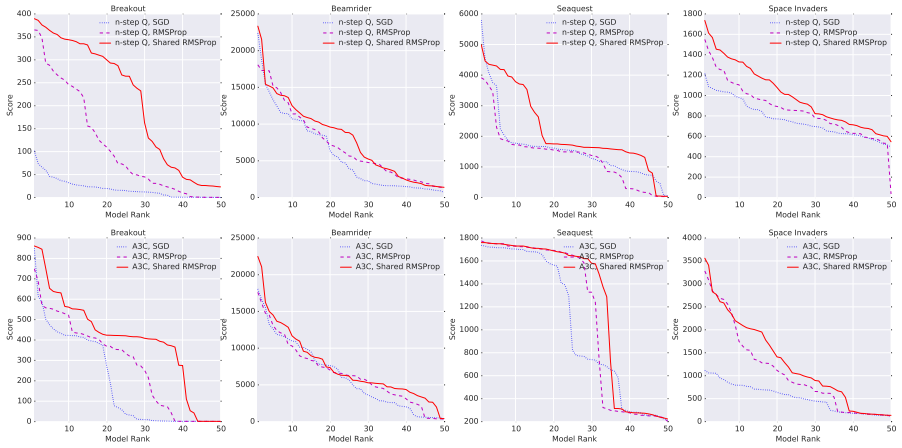


Figure S5. Comparison of three different optimization methods (Momentum SGD, RMSProp, Shared RMSProp) tested using two different algorithms (Async n-step Q and Async Advantage Actor-Critic) on four different Atari games (Breakout, Beamrider, Seaquest and Space Invaders). Each curve shows the final scores for 50 experiments sorted in descending order that covers a search over 50 random initializations and learning rates. The top row shows results using Async n-step Q algorithm and bottom row shows results with Async Advantage Actor-Critic. Each individual graph shows results for one of the four games and three different optimization methods. Shared RMSProp tends to be more robust to different learning rates and random initializations than Momentum SGD and RMSProp without sharing.

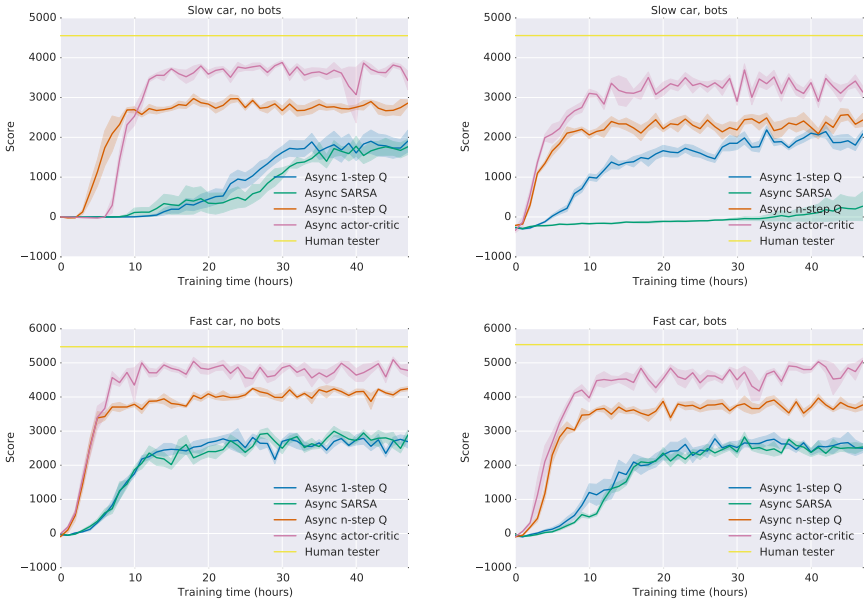


Figure S6. Comparison of algorithms on the TORCS car racing simulator. Four different configurations of car speed and opponent presence or absence are shown. In each plot, all four algorithms (one-step Q, one-step Sarsa, n-step Q and Advantage Actor-Critic) are compared on score vs training time in wall clock hours. Multi-step algorithms achieve better policies much faster than one-step algorithms on all four levels. The curves show averages over the 5 best runs from 50 experiments with learning rates sampled from $LogUniform(10^{-4}, 10^{-2})$ and all other hyperparameters fixed.

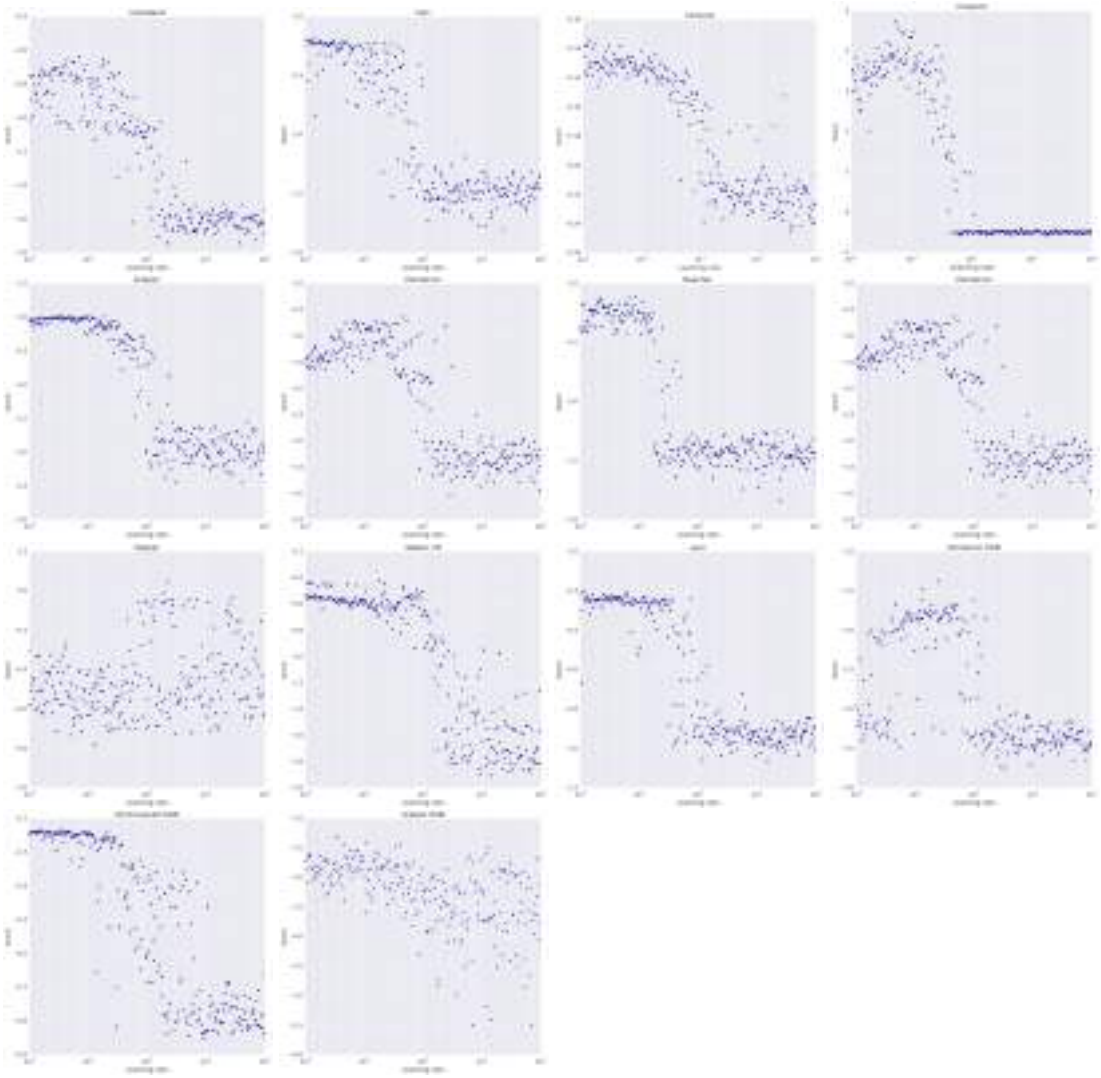


Figure S7. Performance for the Mujoco continuous action domains. Scatter plot of the best score obtained against learning rates sampled from $LogUniform(10^{-5}, 10^{-1})$. For nearly all of the tasks there is a wide range of learning rates that lead to good performance on the task.

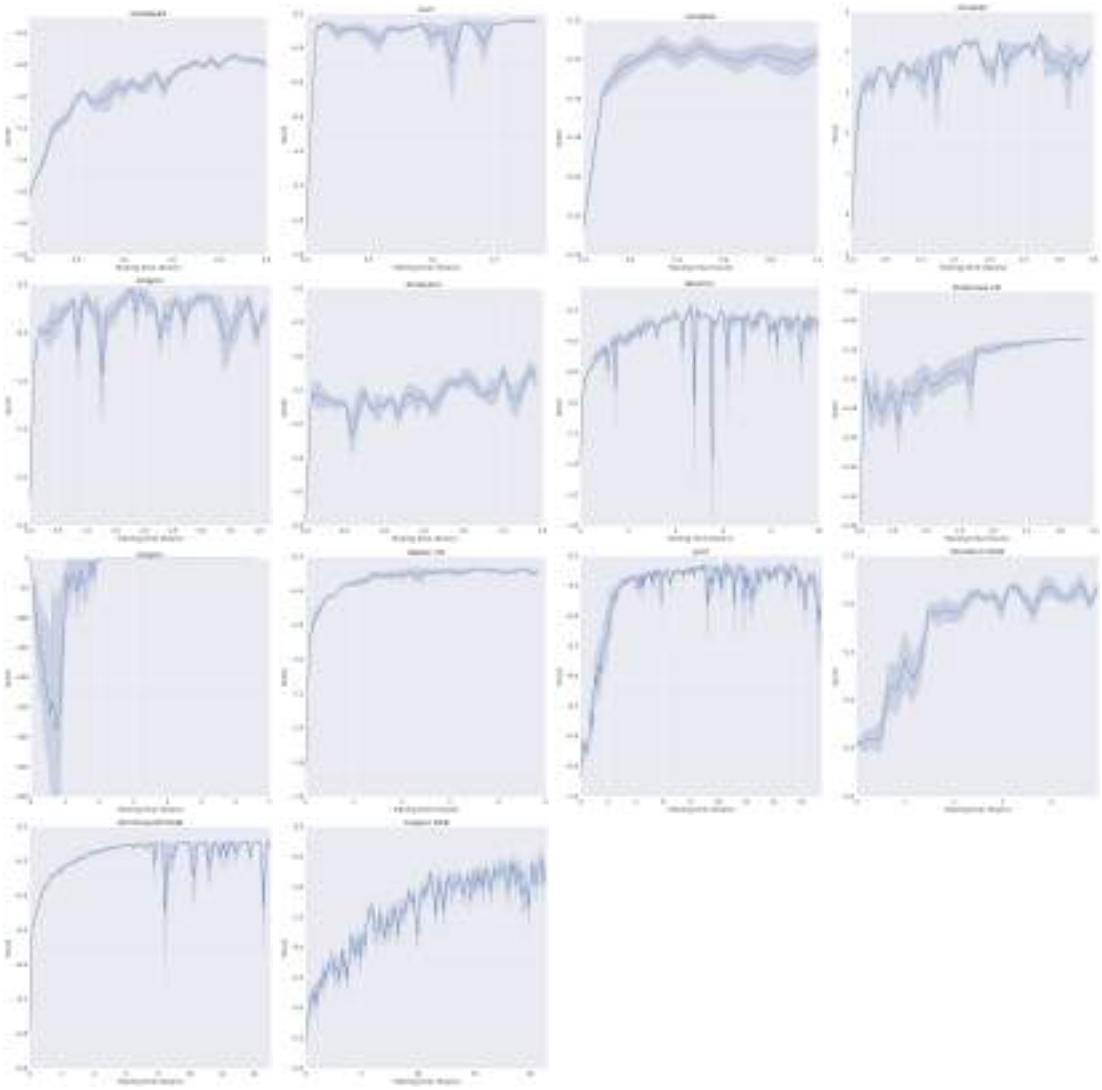


Figure S8. Score per episode vs wall-clock time plots for the Mujoco domains. Each plot shows error bars for the top 5 experiments.

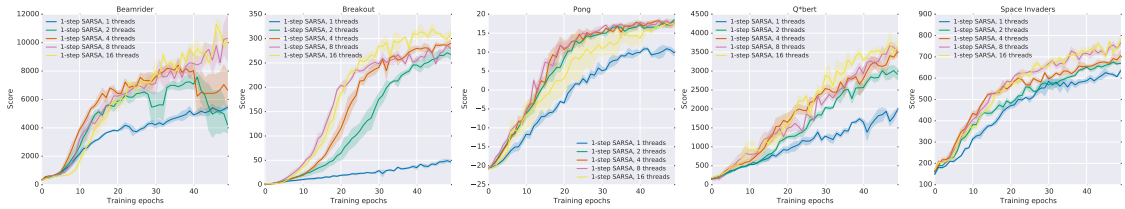


Figure S9. Data efficiency comparison of different numbers of actor-learners one-step Sarsa on five Atari games. The x-axis shows the total number of training epochs where an epoch corresponds to four million frames (across all threads). The y-axis shows the average score. Each curve shows the average of the three best performing agents from a search over 50 random learning rates. Sarsa shows increased data efficiency with increased numbers of parallel workers.

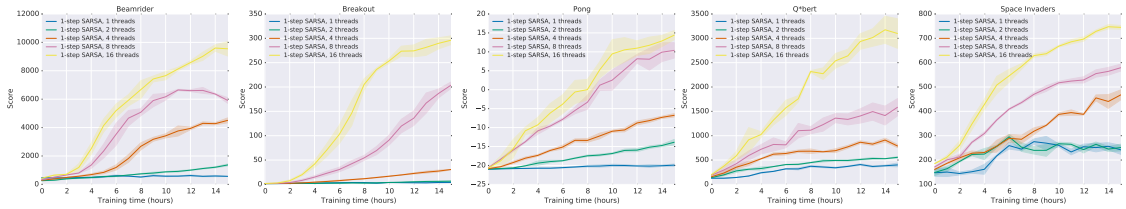


Figure S10. Training speed comparison of different numbers of actor-learners for all one-step Sarsa on five Atari games. The x-axis shows training time in hours while the y-axis shows the average score. Each curve shows the average of the three best performing agents from a search over 50 random learning rates. Sarsa shows significant speedups from using greater numbers of parallel actor-learners.

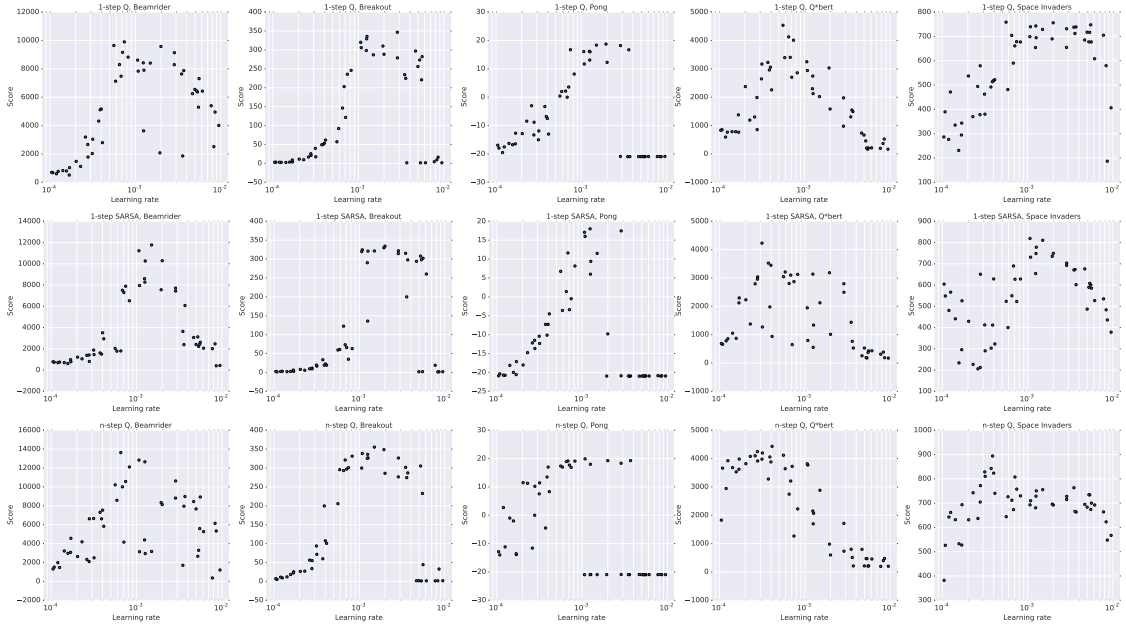


Figure S11. Scatter plots of scores obtained by one-step Q, one-step Sarsa, and n-step Q on five games (Beamrider, Breakout, Pong, Q*bert, Space Invaders) for 50 different learning rates and random initializations. All algorithms exhibit some level of robustness to the choice of learning rate.

Game	DQN	Gorila	Double	Dueling	Prioritized	A3C FF, 1 day	A3C FF	A3C LSTM
Alien	570.2	813.5	1033.4	**1486.5**	900.5	182.1	518.4	945.3
Amidar	133.4	189.2	169.1	172.7	218.4	**283.9**	263.9	173.0
Assault	3332.3	1195.8	6060.8	3994.8	7748.5	3746.1	5474.9	**14497.9**
Asterix	124.5	3324.7	16837.0	15840.0	**31907.5**	6723.0	22140.5	17244.5
Asteroids	697.1	933.6	1193.2	2035.4	1654.0	3009.4	4474.5	**5093.1**
Atlantis	76108.0	629166.5	319688.0	445360.0	593642.0	772392.0	**911091.0**	875822.0
Bank Heist	176.3	399.4	886.0	**1129.3**	816.8	946.0	970.1	932.8
Battle Zone	17560.0	19938.0	24740.0	**31320.0**	29100.0	11340.0	12950.0	20760.0
Beam Rider	8672.4	3822.1	17417.2	14591.3	**26172.7**	13235.9	22707.9	24622.2
Berzerk			1011.1	910.6	1165.6	**1433.4**	817.9	862.2
Bowling	41.2	54.0	**69.6**	65.7	65.8	36.2	35.1	41.8
Boxing	25.8	74.2	73.5	**77.3**	68.6	33.7	59.8	37.3
Breakout	303.9	313.0	368.9	411.6	371.6	551.6	681.9	**766.8**
Centipede	3773.1	**6296.9**	3853.5	4881.0	3421.9	3306.5	3755.8	1997.0
Chopper Comman	3046.0	3191.8	3495.0	3784.0	6604.0	4669.0	7021.0	**10150.0**
Crazy Climber	50992.0	65451.0	113782.0	124566.0	131086.0	101624.0	112646.0	**138518.0**
Defender			27510.0	33996.0	21093.5	36242.5	56533.0	**233021.5**
Demon Attack	12835.2	14880.1	69803.4	56322.8	73185.8	84997.5	113308.4	**115201.9**
Double Dunk	-21.6	-11.3	-0.3	-0.8	**2.7**	0.1	-0.1	0.1
Enduro	475.6	71.0	1216.6	**2077.4**	1884.4	-82.2	-82.5	-82.5
Fishing Derby	-2.3	4.6	3.2	-4.1	9.2	13.6	18.8	**22.6**
Freeway	25.8	10.2	**28.8**	0.2	27.9	0.1	0.1	0.1
Frostbite	157.4	426.6	1448.1	2332.4	**2930.2**	180.1	190.5	197.6
Gopher	2731.8	4373.0	15253.0	20051.4	**57783.8**	8442.8	10022.8	17106.8
Gravitar	216.5	**538.4**	200.5	297.0	218.0	269.5	303.5	320.0
H.E.R.O.	12952.5	8963.4	14892.5	15207.9	20506.4	28765.8	**32464.1**	28889.5
Ice Hockey	-3.8	-1.7	-2.5	-1.3	**-1.0**	-4.7	-2.8	-1.7
James Bond	348.5	444.0	573.0	835.5	**3511.5**	351.5	541.0	613.0
Kangaroo	2696.0	1431.0	**11204.0**	10334.0	10241.0	106.0	94.0	125.0
Krull	3864.0	6363.1	6796.1	8051.6	7406.5	**8066.6**	5560.0	5911.4
Kung-Fu Master	11875.0	20620.0	30207.0	24288.0	31244.0	3046.0	28819.0	**40835.0**
Montezuma's Revenge	50.0	**84.0**	42.0	22.0	13.0	53.0	67.0	41.0
Ms. Pacman	763.5	1263.0	1241.3	**2250.6**	1824.6	594.4	653.7	850.7
Name This Game	5439.9	9238.5	8960.3	11185.1	11836.1	5614.0	10476.1	**12093.7**
Phoenix			12366.5	20410.5	27430.1	28181.8	52894.1	**74786.7**
Pit Fall			-186.7	-46.9	**-14.8**	-123.0	-78.5	-135.7
Pong	16.2	16.7	**19.1**	18.8	18.9	11.4	5.6	10.7
Private Eye	298.2	**2598.6**	-575.5	292.6	179.0	194.4	206.9	421.1
Q*Bert	4589.8	7089.8	11020.8	14175.8	11277.0	13752.3	15148.8	**21307.5**
River Raid	4065.3	5310.3	10838.4	16569.4	**18184.4**	10001.2	12201.8	6591.9
Road Runner	9264.0	43079.8	43156.0	58549.0	56990.0	31769.0	34216.0	**73949.0**
Robotank	58.5	61.8	59.1	**62.0**	55.4	2.3	32.8	2.6
Seaquest	2793.9	10145.9	14498.0	37361.6	**39096.7**	2300.2	2355.4	1326.1
Skiing			-11490.4	-11928.0	**-10852.8**	-13700.0	-10911.1	-14863.8
Solaris			810.0	1768.4	**2238.2**	1884.8	1956.0	1936.4
Space Invaders	1449.7	1183.3	2628.7	5993.1	9063.0	2214.7	15730.5	**23846.0**
Star Gunner	34081.0	14919.2	58365.0	90804.0	51959.0	64393.0	138218.0	**164766.0**
Surround			1.9	**4.0**	-0.9	-9.6	-9.7	-8.3
Tennis	-2.3	-0.7	-7.8	**4.4**	-2.0	-10.2	-6.3	-6.4
Time Pilot	5640.0	8267.8	6608.0	6601.0	7448.0	5825.0	12679.0	**27202.0**
Tutankham	32.4	118.5	92.2	48.0	33.6	26.1	**156.3**	144.2
Up and Down	3311.3	8747.7	19086.9	24759.2	29443.7	54525.4	74705.7	**105728.7**
Venture	54.0	**523.4**	21.0	200.0	244.0	19.0	23.0	25.0
Video Pinball	20228.1	112093.4	367823.7	110976.2	374886.9	185852.6	331628.1	**470310.5**
Wizard of Wor	246.0	10431.0	6201.0	7054.0	7451.0	5278.0	17244.0	**18082.0**
Yars Revenge			6270.6	**25976.5**	5965.1	7270.8	7157.5	5615.5
Zaxxon	831.0	6159.4	8593.0	10164.0	9501.0	2659.0	**24622.0**	23519.0

Table S3. Raw scores for the human start condition (30 minutes emulator time). DQN scores taken from (Nair et al., 2015). Double DQN scores taken from (Van Hasselt et al., 2015), Dueling scores from (Wang et al., 2015) and Prioritized scores taken from (Schaul et al., 2015)

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains the data efficiency and reliable performance of TRPO, while using only first-order optimization. We propose a novel objective with clipped probability ratios, which forms a pessimistic estimate (i.e., lower bound) of the performance of the policy. To optimize policies, we alternate between sampling data from the policy and performing several epochs of optimization on the sampled data.

Our experiments compare the performance of various different versions of the surrogate objective, and find that the version with the clipped probability ratios performs best. We also compare PPO to several previous algorithms from the literature. On continuous control tasks, it performs better than the algorithms we compare against. On Atari, it performs significantly better (in terms of sample complexity) than A2C and similarly to ACER though it is much simpler.

[1] While DQN works well on game environments like the Arcade Learning Environment [Bel+15] with discrete action spaces, it has not been demonstrated to perform well on continuous control benchmarks such as those in OpenAI Gym [Bro+16] and described by Duan et al. [Dua+16].

2 Background: Policy Optimization

2.1 Policy Gradient Methods

Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. The most commonly used gradient estimator has the form

$$\hat{g} = \hat{\mathbb{E}}_t\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t\Big] \tag{1}$$

where π_θ is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t. Here, the expectation $\hat{\mathbb{E}}_t[\ldots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization. Implementations that use automatic differentiation software work by constructing an objective function whose gradient is the policy gradient estimator; the estimator $\hat{g}$ is obtained by differentiating the objective

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t\Big[\log \pi_\theta(a_t \mid s_t)\hat{A}_t\Big]. \tag{2}$$

While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates (see Section 6.1; results are not shown but were similar or worse than the "no clipping or penalty" setting).

2.2 Trust Region Methods

In TRPO [Sch+15b], an objective function (the "surrogate" objective) is maximized subject to a constraint on the size of the policy update. Specifically,

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] \tag{3}$$

$$\text{subject to} \quad \hat{\mathbb{E}}_t[\mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]] \le \delta. \tag{4}$$

Here, θ_{old} is the vector of policy parameters before the update. This problem can efficiently be approximately solved using the conjugate gradient algorithm, after making a linear approximation to the objective and a quadratic approximation to the constraint.

The theory justifying TRPO actually suggests using a penalty instead of a constraint, i.e., solving the unconstrained optimization problem

$$\underset{\theta}{\text{maximize}}\, \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t - \beta\, \mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]\right] \tag{5}$$

for some coefficient β. This follows from the fact that a certain surrogate objective (which computes the max KL over states instead of the mean) forms a lower bound (i.e., a pessimistic bound) on the performance of the policy π. TRPO uses a hard constraint rather than a penalty because it is hard to choose a single value of β that performs well across different problems—or even within a single problem, where the the characteristics change over the course of learning. Hence, to achieve our goal of a first-order algorithm that emulates the monotonic improvement of TRPO, experiments show that it is not sufficient to simply choose a fixed penalty coefficient β and optimize the penalized objective Equation (5) with SGD; additional modifications are required.

3 Clipped Surrogate Objective

Let $r_t(\theta)$ denote the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$, so $r(\theta_{\text{old}}) = 1$. TRPO maximizes a "surrogate" objective

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] = \hat{\mathbb{E}}_t\Big[r_t(\theta)\hat{A}_t\Big]. \tag{6}$$

The superscript CPI refers to conservative policy iteration [KL02], where this objective was proposed. Without a constraint, maximization of L^{CPI} would lead to an excessively large policy update; hence, we now consider how to modify the objective, to penalize changes to the policy that move $r_t(\theta)$ away from 1.

The main objective we propose is the following:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\Big[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\Big] \tag{7}$$

where epsilon is a hyperparameter, say, $\epsilon = 0.2$. The motivation for this objective is as follows. The first term inside the min is L^{CPI}. The second term, $\text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t$, modifies the surrogate objective by clipping the probability ratio, which removes the incentive for moving r_t outside of the interval $[1-\epsilon, 1+\epsilon]$. Finally, we take the minimum of the clipped and unclipped objective, so the final objective is a lower bound (i.e., a pessimistic bound) on the unclipped objective. With this scheme, we only ignore the change in probability ratio when it would make the objective improve, and we include it when it makes the objective worse. Note that $L^{CLIP}(\theta) = L^{CPI}(\theta)$ to first order around θ_{old} (i.e., where $r = 1$), however, they become different as θ moves away from θ_{old}. Figure 1 plots a single term (i.e., a single t) in L^{CLIP}; note that the probability ratio r is clipped at $1-\epsilon$ or $1+\epsilon$ depending on whether the advantage is positive or negative.

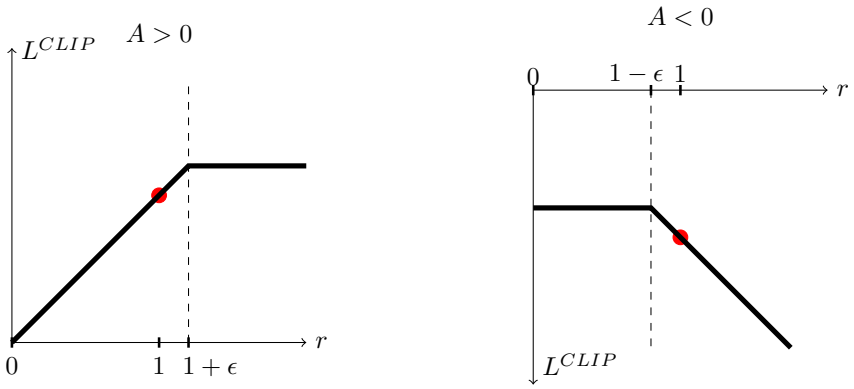


Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r, for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

Figure 2 provides another source of intuition about the surrogate objective L^{CLIP}. It shows how several objectives vary as we interpolate along the policy update direction, obtained by proximal policy optimization (the algorithm we will introduce shortly) on a continuous control problem. We can see that L^{CLIP} is a lower bound on L^{CPI}, with a penalty for having too large of a policy update.

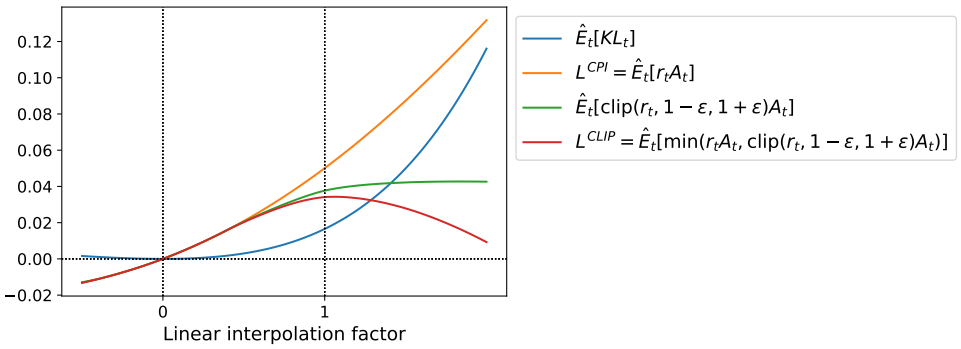


Figure 2: Surrogate objectives, as we interpolate between the initial policy parameter θ_{old}, and the updated policy parameter, which we compute after one iteration of PPO. The updated policy has a KL divergence of about 0.02 from the initial policy, and this is the point at which L^{CLIP} is maximal. This plot corresponds to the first policy update on the Hopper-v1 problem, using hyperparameters provided in Section 6.1.

4 Adaptive KL Penalty Coefficient

Another approach, which can be used as an alternative to the clipped surrogate objective, or in addition to it, is to use a penalty on KL divergence, and to adapt the penalty coefficient so that we achieve some target value of the KL divergence d_{targ} each policy update. In our experiments, we found that the KL penalty performed worse than the clipped surrogate objective, however, we've included it here because it's an important baseline.

In the simplest instantiation of this algorithm, we perform the following steps in each policy update:

- Using several epochs of minibatch SGD, optimize the KL-penalized objective

$$L^{KLPEN}(\theta) = \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t - \beta\,\text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]\right] \tag{8}$$

- Compute $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]]$
  - If $d < d_{\text{targ}}/1.5$, $\beta \leftarrow \beta/2$
  - If $d > d_{\text{targ}} \times 1.5$, $\beta \leftarrow \beta \times 2$

The updated β is used for the next policy update. With this scheme, we occasionally see policy updates where the KL divergence is significantly different from d_{targ}, however, these are rare, and β quickly adjusts. The parameters 1.5 and 2 above are chosen heuristically, but the algorithm is not very sensitive to them. The initial value of β is a another hyperparameter but is not important in practice because the algorithm quickly adjusts it.

5 Algorithm

The surrogate losses from the previous sections can be computed and differentiated with a minor change to a typical policy gradient implementation. For implementations that use automatic differentation, one simply constructs the loss L^{CLIP} or L^{KLPEN} instead of L^{PG}, and one performs multiple steps of stochastic gradient ascent on this objective.

Most techniques for computing variance-reduced advantage-function estimators make use a learned state-value function $V(s)$; for example, generalized advantage estimation [Sch+15a], or the

finite-horizon estimators in [Mni+16]. If using a neural network architecture that shares parameters between the policy and value function, we must use a loss function that combines the policy surrogate and a value function error term. This objective can further be augmented by adding an entropy bonus to ensure sufficient exploration, as suggested in past work [Wil92; Mni+16]. Combining these terms, we obtain the following objective, which is (approximately) maximized each iteration:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\big[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\big], \tag{9}$$

where c_1, c_2 are coefficients, and S denotes an entropy bonus, and L_t^{VF} is a squared-error loss $(V_\theta(s_t) - V_t^{\text{targ}})^2$.

One style of policy gradient implementation, popularized in [Mni+16] and well-suited for use with recurrent neural networks, runs the policy for T timesteps (where T is much less than the episode length), and uses the collected samples for an update. This style requires an advantage estimator that does not look beyond timestep T. The estimator used by [Mni+16] is

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T) \tag{10}$$

where t specifies the time index in $[0, T]$, within a given length-T trajectory segment. Generalizing this choice, we can use a truncated version of generalized advantage estimation, which reduces to Equation (10) when $\lambda = 1$:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1}, \tag{11}$$

$$\text{where} \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{12}$$

A proximal policy optimization (PPO) algorithm that uses fixed-length trajectory segments is shown below. Each iteration, each of N (parallel) actors collect T timesteps of data. Then we construct the surrogate loss on these NT timesteps of data, and optimize it with minibatch SGD (or usually for better performance, Adam [KB14]), for K epochs.

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

6 Experiments

6.1 Comparison of Surrogate Objectives

First, we compare several different surrogate objectives under different hyperparameters. Here, we compare the surrogate objective L^{CLIP} to several natural variations and ablated versions.

No clipping or penalty:	$L_t(\theta) = r_t(\theta)\hat{A}_t$
Clipping:	$L_t(\theta) = \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta)), 1-\epsilon, 1+\epsilon)\hat{A}_t$
KL penalty (fixed or adaptive)	$L_t(\theta) = r_t(\theta)\hat{A}_t - \beta\,\text{KL}[\pi_{\theta_{\text{old}}}, \pi_\theta]$

For the KL penalty, one can either use a fixed penalty coefficient β or an adaptive coefficient as described in Section 4 using target KL value d_{targ}. Note that we also tried clipping in log space, but found the performance to be no better.

Because we are searching over hyperparameters for each algorithm variant, we chose a computationally cheap benchmark to test the algorithms on. Namely, we used 7 simulated robotics tasks[2] implemented in OpenAI Gym [Bro+16], which use the MuJoCo [TET12] physics engine. We do one million timesteps of training on each one. Besides the hyperparameters used for clipping (ϵ) and the KL penalty (β, d_{targ}), which we search over, the other hyperparameters are provided in in Table 3.

To represent the policy, we used a fully-connected MLP with two hidden layers of 64 units, and tanh nonlinearities, outputting the mean of a Gaussian distribution, with variable standard deviations, following [Sch+15b; Dua+16]. We don't share parameters between the policy and value function (so coefficient c_1 is irrelevant), and we don't use an entropy bonus.

Each algorithm was run on all 7 environments, with 3 random seeds on each. We scored each run of the algorithm by computing the average total reward of the last 100 episodes. We shifted and scaled the scores for each environment so that the random policy gave a score of 0 and the best result was set to 1, and averaged over 21 runs to produce a single scalar for each algorithm setting.

The results are shown in Table 1. Note that the score is negative for the setting without clipping or penalties, because for one environment (half cheetah) it leads to a very negative score, which is worse than the initial random policy.

algorithm	avg. normalized score
No clipping or penalty	-0.39
Clipping, $\epsilon = 0.1$	0.76
Clipping, $\epsilon = 0.2$	**0.82**
Clipping, $\epsilon = 0.3$	0.70
Adaptive KL $d_{\text{targ}} = 0.003$	0.68
Adaptive KL $d_{\text{targ}} = 0.01$	0.74
Adaptive KL $d_{\text{targ}} = 0.03$	0.71
Fixed KL, $\beta = 0.3$	0.62
Fixed KL, $\beta = 1.$	0.71
Fixed KL, $\beta = 3.$	0.72
Fixed KL, $\beta = 10.$	0.69

Table 1: Results from continuous control benchmark. Average normalized scores (over 21 runs of the algorithm, on 7 environments) for each algorithm / hyperparameter setting . β was initialized at 1.

6.2 Comparison to Other Algorithms in the Continuous Domain

Next, we compare PPO (with the "clipped" surrogate objective from Section 3) to several other methods from the literature, which are considered to be effective for continuous problems. We compared against tuned implementations of the following algorithms: trust region policy optimization [Sch+15b], cross-entropy method (CEM) [SL06], vanilla policy gradient with adaptive stepsize[3],

[2] HalfCheetah, Hopper, InvertedDoublePendulum, InvertedPendulum, Reacher, Swimmer, and Walker2d, all "-v1"

[3] After each batch of data, the Adam stepsize is adjusted based on the KL divergence of the original and updated policy, using a rule similar to the one shown in Section 4. An implementation is available at https://github.com/berkeleydeeprlcourse/homework/tree/master/hw4.

A2C [Mni+16], A2C with trust region [Wan+16]. A2C stands for advantage actor critic, and is a synchronous version of A3C, which we found to have the same or better performance than the asynchronous version. For PPO, we used the hyperparameters from the previous section, with $\epsilon = 0.2$. We see that PPO outperforms the previous methods on almost all the continuous control environments.

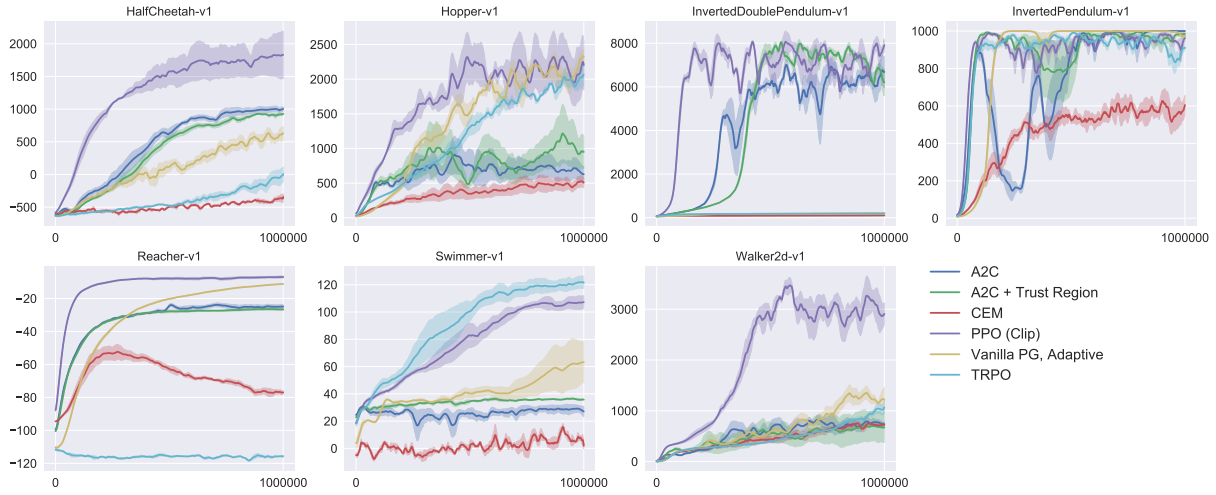


Figure 3: Comparison of several algorithms on several MuJoCo environments, training for one million timesteps.

6.3 Showcase in the Continuous Domain: Humanoid Running and Steering

To showcase the performance of PPO on high-dimensional continuous control problems, we train on a set of problems involving a 3D humanoid, where the robot must run, steer, and get up off the ground, possibly while being pelted by cubes. The three tasks we test on are (1) RoboschoolHumanoid: forward locomotion only, (2) RoboschoolHumanoidFlagrun: position of target is randomly varied every 200 timesteps or whenever the goal is reached, (3) RoboschoolHumanoidFlagrunHarder, where the robot is pelted by cubes and needs to get up off the ground. See Figure 5 for still frames of a learned policy, and Figure 4 for learning curves on the three tasks. Hyperparameters are provided in Table 4. In concurrent work, Heess et al. [Hee+17] used the adaptive KL variant of PPO (Section 4) to learn locomotion policies for 3D robots.

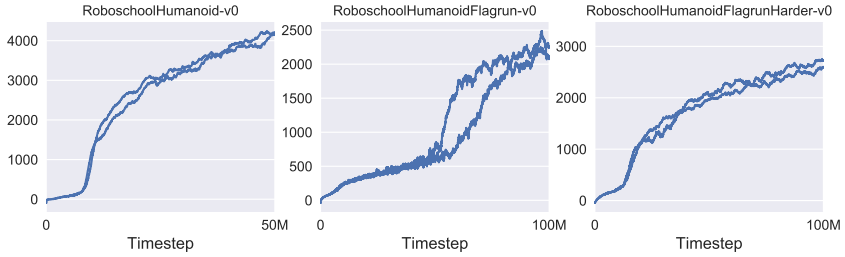


Figure 4: Learning curves from PPO on 3D humanoid control tasks, using Roboschool.

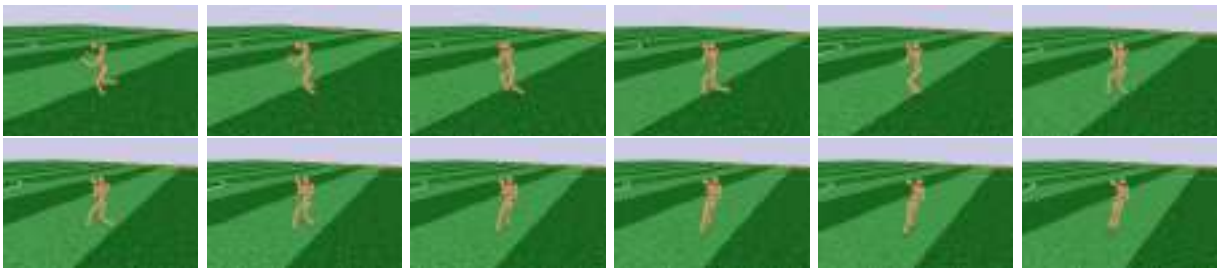


Figure 5: Still frames of the policy learned from RoboschoolHumanoidFlagrun. In the first six frames, the robot runs towards a target. Then the position is randomly changed, and the robot turns and runs toward the new target.

6.4 Comparison to Other Algorithms on the Atari Domain

We also ran PPO on the Arcade Learning Environment [Bel+15] benchmark and compared against well-tuned implementations of A2C [Mni+16] and ACER [Wan+16]. For all three algorithms, we used the same policy network architechture as used in [Mni+16]. The hyperparameters for PPO are provided in Table 5. For the other two algorithms, we used hyperparameters that were tuned to maximize performance on this benchmark.

A table of results and learning curves for all 49 games is provided in Appendix B. We consider the following two scoring metrics: (1) *average reward per episode over entire training period* (which favors fast learning), and (2) *average reward per episode over last 100 episodes of training* (which favors final performance). Table 2 shows the number of games "won" by each algorithm, where we compute the victor by averaging the scoring metric across three trials.

	A2C	ACER	PPO	Tie
(1) avg. episode reward over all of training	1	18	**30**	0
(2) avg. episode reward over last 100 episodes	1	**28**	19	1

Table 2: Number of games "won" by each algorithm, where the scoring metric is averaged across three trials.

7 Conclusion

We have introduced proximal policy optimization, a family of policy optimization methods that use multiple epochs of stochastic gradient ascent to perform each policy update. These methods have the stability and reliability of trust-region methods but are much simpler to implement, requiring only few lines of code change to a vanilla policy gradient implementation, applicable in more general settings (for example, when using a joint architecture for the policy and value function), and have better overall performance.

8 Acknowledgements

Thanks to Rocky Duan, Peter Chen, and others at OpenAI for insightful comments.

A Hyperparameters

Hyperparameter	Value
Horizon (T)	2048
Adam stepsize	3×10^{-4}
Num. epochs	10
Minibatch size	64
Discount (γ)	0.99
GAE parameter (λ)	0.95

Table 3: PPO hyperparameters used for the Mujoco 1 million timestep benchmark.

Hyperparameter	Value
Horizon (T)	512
Adam stepsize	$*$
Num. epochs	15
Minibatch size	4096
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	32 (locomotion), 128 (flagrun)
Log stdev. of action distribution	LinearAnneal$(-0.7, -1.6)$

Table 4: PPO hyperparameters used for the Roboschool experiments. Adam stepsize was adjusted based on the target value of the KL divergence.

Hyperparameter	Value
Horizon (T)	128
Adam stepsize	$2.5 \times 10^{-4} \times \alpha$
Num. epochs	3
Minibatch size	32×8
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	8
Clipping parameter ϵ	$0.1 \times \alpha$
VF coeff. c_1 (9)	1
Entropy coeff. c_2 (9)	0.01

Table 5: PPO hyperparameters used in Atari experiments. α is linearly annealed from 1 to 0 over the course of learning.

B Performance on More Atari Games

Here we include a comparison of PPO against A2C on a larger collection of 49 Atari games. Figure 6 shows the learning curves of each of three random seeds, while Table 6 shows the mean performance.

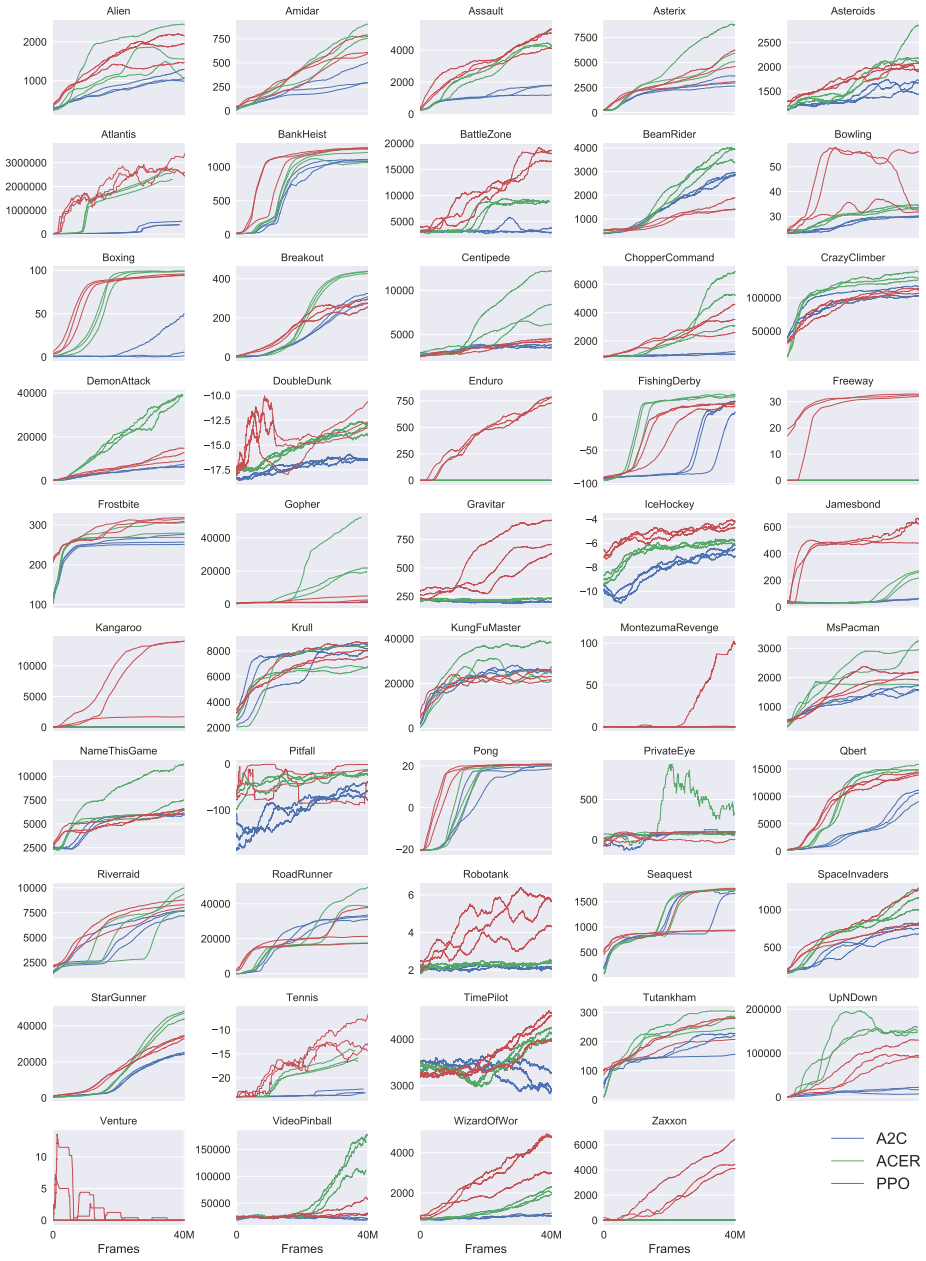


Figure 6: Comparison of PPO and A2C on all 49 ATARI games included in OpenAI Gym at the time of publication.

	A2C	ACER	PPO
Alien	1141.7	1655.4	**1850.3**
Amidar	380.8	**827.6**	674.6
Assault	1562.9	4653.8	**4971.9**
Asterix	3176.3	**6801.2**	4532.5
Asteroids	1653.3	**2389.3**	2097.5
Atlantis	729265.3	1841376.0	**2311815.0**
BankHeist	1095.3	1177.5	**1280.6**
BattleZone	3080.0	8983.3	**17366.7**
BeamRider	3031.7	**3863.3**	1590.0
Bowling	30.1	33.3	**40.1**
Boxing	17.7	**98.9**	94.6
Breakout	303.0	**456.4**	274.8
Centipede	3496.5	**8904.8**	4386.4
ChopperCommand	1171.7	**5287.7**	3516.3
CrazyClimber	107770.0	**132461.0**	110202.0
DemonAttack	6639.1	**38808.3**	11378.4
DoubleDunk	-16.2	**-13.2**	-14.9
Enduro	0.0	0.0	**758.3**
FishingDerby	20.6	**34.7**	17.8
Freeway	0.0	0.0	**32.5**
Frostbite	261.8	285.6	**314.2**
Gopher	1500.9	**37802.3**	2932.9
Gravitar	194.0	225.3	**737.2**
IceHockey	-6.4	-5.9	**-4.2**
Jamesbond	52.3	261.8	**560.7**
Kangaroo	45.3	50.0	**9928.7**
Krull	**8367.4**	7268.4	7942.3
KungFuMaster	24900.3	**27599.3**	23310.3
MontezumaRevenge	0.0	0.3	**42.0**
MsPacman	1626.9	**2718.5**	2096.5
NameThisGame	5961.2	**8488.0**	6254.9
Pitfall	-55.0	**-16.9**	-32.9
Pong	19.7	**20.7**	20.7
PrivateEye	91.3	**182.0**	69.5
Qbert	10065.7	**15316.6**	14293.3
Riverraid	7653.5	**9125.1**	8393.6
RoadRunner	32810.0	**35466.0**	25076.0
Robotank	2.2	2.5	**5.5**
Seaquest	1714.3	**1739.5**	1204.5
SpaceInvaders	744.5	**1213.9**	942.5
StarGunner	26204.0	**49817.7**	32689.0
Tennis	-22.2	-17.6	**-14.8**
TimePilot	2898.0	4175.7	**4342.0**
Tutankham	206.8	**280.8**	254.4
UpNDown	17369.8	**145051.4**	95445.0
Venture	0.0	0.0	0.0
VideoPinball	19735.9	**156225.6**	37389.0
WizardOfWor	859.0	2308.3	**4185.3**
Zaxxon	16.3	29.0	**5008.7**

Table 6: Mean final scores (last 100 episodes) of PPO and A2C on Atari games after 40M game frames (10M timesteps).

To appear as a part of an upcoming textbook on deep learning.

Recurrent Neural Networks and Long Short-Term Memory Networks: Tutorial and Survey

Benyamin Ghojogh BGHOJOGH@UWATERLOO.CA
Ali Ghodsi ALI.GHODSI@UWATERLOO.CA
Department of Statistics and Actuarial Science & David R. Cheriton School of Computer Science,
Data Analytics Laboratory, University of Waterloo, Waterloo, ON, Canada

YouTube channel of Benyamin Ghojogh: https://www.youtube.com/@bghojogh
YouTube channel of Ali Ghodsi: https://www.youtube.com/@DataScienceCoursesUW

Abstract

This is a tutorial paper on Recurrent Neural Network (RNN), Long Short-Term Memory Network (LSTM), and their variants. We start with a dynamical system and backpropagation through time for RNN. Then, we discuss the problems of gradient vanishing and explosion in long-term dependencies. We explain close-to-identity weight matrix, long delays, leaky units, and echo state networks for solving this problem. Then, we introduce LSTM gates and cells, history and variants of LSTM, and Gated Recurrent Units (GRU). Finally, we introduce bidirectional RNN, bidirectional LSTM, and the Embeddings from Language Model (ELMo) network, for processing a sequence in both directions.

1. Introduction

Before the era of transformers in deep learning, regular neural networks could not process sequences, such as sentences (sequence of words) or speech (sequence of phonemes), properly without any recurrence. Recurrent Neural Network (RNN), proposed in (Rumelhart et al., 1986), is a dynamical system which considers recurrence. In recurrence, the output of a model is fed as input to the model again in the next time step. One of the main training algorithms for RNN is Backpropagation Through Time (BPTT), developed by several works (Robinson & Fallside, 1987; Werbos, 1988; Williams, 1989; Williams & Zipser, 1995; Mozer, 1995), which is similar to the backpropagation algorithm (Rumelhart et al., 1986) but has also chain rule through time. There were some problems with gradient vanishing or explosion for long-term dependencies in RNN (Bengio et al., 1993; 1994). Several solutions were proposed for this issue, some of which are close-to-identity weight matrix (Mikolov et al., 2015), long delays (Lin et al., 1995), leaky units (Jaeger et al., 2007; Sutskever & Hinton, 2010), and echo state networks (Jaeger & Haas, 2004; Jaeger, 2007).

Sequence modeling requires both short-term and long-term dependencies. For example, consider the sentence "The police is chasing the thief". In this sentence, the words "police" and "thief" are related to each other with short-term dependency because they are close to one another in the sequence of words. Another example is the sentence "I was born in France. My father was working there for many years during my childhood. My family had a great time there while my father was making money in his business there. That is why I know how to speak French". In this second example, the words "France" and "French" are related to each other with long-term dependency because they are far away from one another in the sequence of words. That inspired researchers to propose the Long Short-Term Memory (LSTM) network to handle both short-term and long-tern dependencies (Hochreiter & Schmidhuber, 1995; 1997). Later, Grated Recurrent Unit (GRU) was proposed (Cho et al., 2014) which simplified LSTM to reduce its unnecessary complexity.

RNN and LSTM networks are causal models which condition every sequence element on the *previous* elements in the sequence. Later researches showed that processing the sequence in both directions can perform better for the sequences which can be processed offline; e.g., if the chunks of sequence can be saved and processed and the sequence elements should not be processed as a stream (Graves & Schmidhuber, 2005a;b). Therefore, bidirectional RNN (Schuster & Paliwal, 1997; Baldi et al., 1999) and bidirectional LSTM (Graves & Schmidhuber, 2005a;b)

were proposed to process sequences in both directions. The Embeddings from Language Model (ELMo) network (Peters et al., 2018) is a language model which makes use of the bidirectional LSTM.

This is a tutorial on RNN, LSTM, and their variants. There exist some other tutorials and surveys about this topic, some of which are (Jaeger, 2002; Jozefowicz et al., 2015; Lipton et al., 2015; Schmidhuber, 2015; Greff et al., 2016; Salehinejad et al., 2017; Staudemeyer & Morris, 2019; Yu et al., 2019; Smagulova & James, 2019). A very good survey on the variants of LSTM is (Greff et al., 2016).

2. Recurrent Neural Network

2.1. Dynamical System

A dynamical system is recursive and its classical form is as follows:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}), \tag{1}$$

where t denotes the time step, $\boldsymbol{h}_t$ is the state at time t, and $f_\theta(.)$ is a function fixed between the states of all time steps. Figure 1-a shows such a system. Dynamical systems are widely used in chaos theory (Broer & Takens, 2011).

We can have a dynamical system with external input signal where $\boldsymbol{x}_t$ denotes the input signal at time t. This system is modeled as:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t). \tag{2}$$

This system is depicted in Fig. 1-b.

2.2. Parameter Sharing

The state $\boldsymbol{h}_t$ can be considered as a summary of the past sequence of inputs and states. If a different function f_θ is defined for each possible sequence length, the model will not have generalization. If the same parameters are used for any sequence length, the model will have generalization properties. Therefore, the parameters are shared for all lengths and between all states. Such a dynamical system with parameter sharing can be implemented as a neural network with weights. Such a network is called a Recurrent Neural Network (RNN), which was proposed in (Rumelhart et al., 1986).

RNN is illustrated in Fig. 1-c, where the same weight matrices are used for all time slots. RNN gets a sequence as input and outputs a sequence as a decision for a task such as regression or classification. Suppose the input, output, and state at time slot t are denoted by $\boldsymbol{x}_t \in \mathbb{R}^d$, $\boldsymbol{y}_t \in \mathbb{R}^q$, and $\boldsymbol{h}_t \in \mathbb{R}^p$, respectively. Let $\boldsymbol{W} \in \mathbb{R}^{p \times p}$ be the weight matrix between states, $\boldsymbol{U} \in \mathbb{R}^{p \times d}$ be the weight matrix between the inputs and the states, and $\boldsymbol{V} \in \mathbb{R}^{q \times p}$ denote the weight matrix between the states and outputs. The bias weights for the state and the output are denoted by $\boldsymbol{b}_i \in \mathbb{R}^p$ and $\boldsymbol{b}_y \in \mathbb{R}^q$, respectively. As shown in Fig. 1-c, we have:

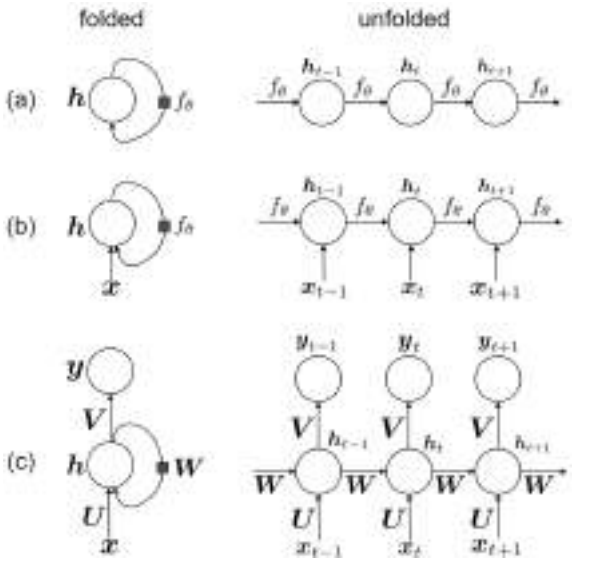


Figure 1. The folded and unfolded structures of (a) a dynamic system without input, (b) a dynamical system with input, and (c) an RNN. Every square on an edge means connection from one time slot before.

$$\mathbb{R}^p \ni \boldsymbol{i}_t = \boldsymbol{W}\boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i, \tag{3}$$

$$[-1,1]^p \ni \boldsymbol{h}_t = \tanh(\boldsymbol{i}_t) = \tanh(\boldsymbol{W}\boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i), \tag{4}$$

$$\mathbb{R}^q \ni \boldsymbol{y}_t = \boldsymbol{V}\boldsymbol{h}_t + \boldsymbol{b}_y, \tag{5}$$

where $\tanh(.) \in (-1, 1)$ denotes the hyperbolic tangent function, which is used as an element-wise activation function for the states.

If there is an activation function, such as softmax, at the output layer, we denote the output of activation function by:

$$\mathbb{R}^q \ni \widehat{\boldsymbol{y}}_t = \text{softmax}(\boldsymbol{y}_t) = \frac{\exp(y_{t,1})}{\sum_{j=1}^{q} \exp(y_{t,j})}, \tag{6}$$

where $y_{t,j}$ denotes the j-th component of $\boldsymbol{y}_t$.

2.3. Backpropagation Through Time (BPTT)

One of the methods for training RNN is Backpropagation Through Time (BPTT), which is very similar to the backpropagation algorithm (Rumelhart et al., 1986) because it is based on gradient descent and chain rule (Ghojogh et al., 2021), but it has also chain rule through time. BPTT was developed by several works (Robinson & Fallside, 1987; Werbos, 1988; Williams, 1989; Williams & Zipser, 1995; Mozer, 1995). This algorithm is very solid in theory; however, it does not show the best performance in practice.

In BPTT, the loss is considered as a summation of loss functions at the previous time steps until now. As it is im-

practical to consider all time steps from the start of training (especially after a long time of training), we only consider the T previous time steps. In other words, we assume that RNN has T-order Markov property (Ghojogh et al., 2019b). Therefore, the loss function is:

$$\mathbb{R} \ni \mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t, \tag{7}$$

where $\mathcal{L}_1$ is the loss function at the current time slot and $\mathcal{L}_t$ denotes the loss function at the previous $(t-1)$ time slot.

This loss functions needs to be optimized using gradient descent and chain rule. Therefore, we calculate its gradient with respect to the parameters of RNN. These parameters are $\boldsymbol{y}_t$, $\boldsymbol{h}_t$, $\boldsymbol{V}$, $\boldsymbol{W}$, $\boldsymbol{U}$, $\boldsymbol{b}_i$, and $\boldsymbol{b}_y$, based on Eqs. (3), (4), and (5) and Fig. 1-c.

2.3.1. Gradient With Respect to the Output

If there is no activation function at the last layer, the gradient of the loss function of RNN with respect to the output at time t is:

$$\mathbb{R}^q \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \overset{(a)}{=} \frac{\partial \mathcal{L}}{\partial \mathcal{L}_t} \times \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t} \overset{(7)}{=} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t}, \tag{8}$$

where (a) is because of the chain rule.

The gradient of the loss function at time t with respect to the output at time t, i.e., $\partial \mathcal{L}_t / \partial \boldsymbol{y}_t$, is calculated based on the formula of the loss function. The loss function can be any loss function for classification, regression, or other tasks.

If there is an activation function at the last layer (see Eq. (6)), the gradient is:

$$\mathbb{R}^q \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \overset{(a)}{=} \frac{\partial \mathcal{L}}{\partial \mathcal{L}_t} \times \frac{\partial \mathcal{L}_t}{\partial \widehat{\boldsymbol{y}}_t} \times \frac{\partial \widehat{\boldsymbol{y}}_t}{\partial \boldsymbol{y}_t} \overset{(7)}{=} \frac{\partial \mathcal{L}_t}{\partial \widehat{\boldsymbol{y}}_t} \times \frac{\partial \widehat{\boldsymbol{y}}_t}{\partial \boldsymbol{y}_t}, \tag{9}$$

where (a) is because of the chain rule. The derivative $\partial \widehat{\boldsymbol{y}}_t / \partial \boldsymbol{y}_t$ is calculated based on the formula of the activation function. The other derivative, $\partial \mathcal{L}_t / \partial \widehat{\boldsymbol{y}}_t$, is calculated based on the formula of the loss as a function of the output of the activation function.

2.3.2. Gradient With Respect to the State

The gradient of the loss function of RNN with respect to the state at time t is:

$$\begin{aligned}\mathbb{R}^p \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} &\overset{(a)}{=} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \frac{\partial \boldsymbol{y}_t}{\partial \boldsymbol{h}_t}\Big) + \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_{t+1}} \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}\Big) \\ &\overset{(5)}{=} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \boldsymbol{V}\Big) + \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_{t+1}} \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}\Big),\end{aligned} \tag{10}$$

where (a) is because changing $\boldsymbol{h}_t$ affects both $\boldsymbol{y}_t$ and $\boldsymbol{h}_{t+1}$. We denote $\boldsymbol{\delta}_t := \partial \mathcal{L} / \partial \boldsymbol{h}_t$ so Eq. (10) becomes:

$$\boldsymbol{\delta}_t = \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \boldsymbol{V}\Big) + \Big(\boldsymbol{\delta}_{t+1} \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}\Big). \tag{11}$$

According to Eqs. (3) and (4), we have:

$$\boldsymbol{i}_{t+1} = \boldsymbol{W}\boldsymbol{h}_t + \boldsymbol{U}\boldsymbol{x}_{t+1} + \boldsymbol{b}_i, \tag{12}$$

$$\boldsymbol{h}_{t+1} = \tanh(\boldsymbol{i}_{t+1}) = \tanh(\boldsymbol{W}\boldsymbol{h}_t + \boldsymbol{U}\boldsymbol{x}_{t+1} + \boldsymbol{b}_i). \tag{13}$$

Therefore:

$$\begin{aligned}\mathbb{R}^{p \times p} \ni \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t} &\overset{(a)}{=} \big(\frac{\partial \boldsymbol{i}_{t+1}}{\partial \boldsymbol{h}_t}\big)^\top \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{i}_{t+1}} \\ &\overset{(b)}{=} \boldsymbol{W}(1 - \boldsymbol{h}_{t+1}^\top \boldsymbol{h}_{t+1})\boldsymbol{I}_{p \times p},\end{aligned} \tag{14}$$

where $\boldsymbol{I}_{p \times p}$ is the identity matrix of size $(p \times p)$, (a) is because of the chain rule, and (b) is because $\mathbb{R}^{p \times p} \ni \partial \boldsymbol{i}_{t+1} / \partial \boldsymbol{h}_t = \boldsymbol{W}^\top$ for Eq. (12), and we have:

$$\mathbb{R}^{p \times p} \ni \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{i}_{t+1}} = (1 - \boldsymbol{h}_{t+1}^\top \boldsymbol{h}_{t+1})\boldsymbol{I}_{p \times p}, \tag{15}$$

based on Eq. (13) and the formula for derivative of the hyperbolic tangent function, noticing that the state is multidimensional and not a scalar.

For the time slot $t = T$, the derivative $\partial \mathcal{L} / \partial \boldsymbol{h}_T$ is much simpler:

$$\mathbb{R}^p \ni \boldsymbol{\delta}_T = \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_T} \overset{(a)}{=} \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_T} \times \frac{\partial \boldsymbol{y}_T}{\partial \boldsymbol{h}_T} \overset{(5)}{=} \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_T} \overset{(8)}{=} \frac{\partial \mathcal{L}_T}{\partial \boldsymbol{y}_T}, \tag{16}$$

where (a) is because of the chain rule and the derivative $\partial \mathcal{L}_T / \partial \boldsymbol{y}_T$ is computed based on the formula of loss function.

2.3.3. Gradient With Respect to $\boldsymbol{V}$

The gradient of the loss function of RNN with respect to the weight matrix $\boldsymbol{V}$ is:

$$\mathbb{R}^{q \times p} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{V}} \overset{(a)}{=} \sum_{t=1}^{T} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \frac{\partial \boldsymbol{y}_t}{\partial \boldsymbol{V}}\Big) \overset{(b)}{=} \sum_{t=1}^{T} \Big(\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t} \times \boldsymbol{h}_t^\top\Big), \tag{17}$$

where (a) is because $\boldsymbol{V}$ exists in all time slots and changing $\boldsymbol{V}$ affects the loss $\mathcal{L}$ in all time slots. The equation (b) is because of Eqs. (8) and (5). The derivative $\partial \mathcal{L}_t / \partial \boldsymbol{y}_t \in \mathbb{R}^q$ is calculated based on the formula of the loss function.

2.3.4. Gradient With Respect to $\boldsymbol{W}$

The gradient of the loss function of RNN with respect to the weight matrix $\boldsymbol{W}$ is:

$$\mathbb{R}^{p \times p} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}} \overset{(a)}{=} \sum_{t=1}^{T} \textbf{vec}_{p \times p}^{-1}\Big(\big(\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}}\big)^\top \times \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}\Big), \tag{18}$$

where $\textbf{vec}_{p \times p}^{-1}(.)$ de-vectorizes the vector of length p^2 to a matrix of size $(p \times p)$ and (a) is because $\boldsymbol{W}$ exists in all time slots and changing $\boldsymbol{W}$ affects the loss $\mathcal{L}$ in all time

slots. The derivative $\partial\mathcal{L}/\partial\boldsymbol{h}_t \in \mathbb{R}^p$ in Eq. (18) was computed in Section 2.3.2. The derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{W}$ in Eq. (18) is:

$$\mathbb{R}^{p\times p^2} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}} = \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} \times \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{W}},$$

because of the chain rule. The first term is:

$$\mathbb{R}^{p\times p} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} = (1 - \boldsymbol{h}_t^\top \boldsymbol{h}_t)\boldsymbol{I}_{p\times p}, \tag{19}$$

according to Eq. (15). Based on the Magnus-Neudecker convention (Ghojogh et al., 2021), the second term is calculated as:

$$\mathbb{R}^{p\times p^2} \ni \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{W}} = \boldsymbol{h}_{t-1}^\top \otimes \boldsymbol{I}_{p\times p},$$

where $\otimes$ denotes the Kronecker product.

2.3.5. Gradient With Respect to $\boldsymbol{U}$

The gradient of the loss function of RNN with respect to the weight matrix $\boldsymbol{U}$ is:

$$\mathbb{R}^{p\times d} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{U}} \overset{(a)}{=} \sum_{t=1}^{T} \textbf{vec}^{-1}_{p\times d}\Big(\big(\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{U}}\big)^\top \times \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}\Big), \tag{20}$$

where (a) is because $\boldsymbol{U}$ exists in all time slots and changing $\boldsymbol{U}$ affects the loss $\mathcal{L}$ in all time slots. The derivative $\partial\mathcal{L}/\partial\boldsymbol{h}_t \in \mathbb{R}^p$ in Eq. (18) was computed in Section 2.3.2. The derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{U}$ in Eq. (18) is:

$$\mathbb{R}^{p\times (pd)} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{U}} = \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} \times \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{U}},$$

because of the chain rule. The first term is already calculated in Eq. (19). Based on the Magnus-Neudecker convention (Ghojogh et al., 2021), the second term is calculated as:

$$\mathbb{R}^{p\times (pd)} \ni \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{U}} = \boldsymbol{x}_t^\top \otimes \boldsymbol{I}_{p\times p}.$$

2.3.6. Gradient With Respect to $\boldsymbol{b}_i$

The gradient of the loss function of RNN with respect to the bias $\boldsymbol{b}_i$ is:

$$\mathbb{R}^{p} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_i} \overset{(a)}{=} \sum_{t=1}^{T} \Big(\big(\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{b}_i}\big)^\top \times \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}\Big), \tag{21}$$

where (a) is because $\boldsymbol{b}_i$ exists in all time slots and changing $\boldsymbol{b}$ affects the loss $\mathcal{L}$ in all time slots. The derivative $\partial\mathcal{L}/\partial\boldsymbol{h}_t$ was already calculated in Section 2.3.2. The derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{b}_i$ is calculated as:

$$\mathbb{R}^{p\times p} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{b}_i} \overset{(a)}{=} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} \times \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{b}_i} \overset{(3)}{=} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t},$$

where (a) is because of the chain rule and the derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{i}_t$ was already calculated in Eq. (19).

2.3.7. Gradient With Respect to $\boldsymbol{b}_y$

The gradient of the loss function of RNN with respect to the bias $\boldsymbol{b}_y$ is:

$$\mathbb{R}^{q} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_y} \overset{(a)}{=} \sum_{t=1}^{T} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \frac{\partial \boldsymbol{y}_t}{\partial \boldsymbol{b}_y}\Big) \overset{(b)}{=} \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t}, \tag{22}$$

where (a) is because $\boldsymbol{b}_y$ exists in all time slots and changing $\boldsymbol{b}_y$ affects the loss $\mathcal{L}$ in all time slots. The equation (b) is because of Eqs. (8) and (5). The derivative $\partial\mathcal{L}_t/\partial\boldsymbol{y}_t \in \mathbb{R}^q$ is calculated based on the formula of the loss function.

2.3.8. Updates by Gradient Descent

BPPT updates the parameters of RNN by gradient descent (Ghojogh et al., 2021) using the calculated gradients:

$$\begin{aligned}
&\boldsymbol{h}_t := \boldsymbol{h}_t - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}, \quad \forall t \in \{1, \dots, T\},\\
&\boldsymbol{V} := \boldsymbol{V} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{V}},\\
&\boldsymbol{W} := \boldsymbol{W} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}},\\
&\boldsymbol{U} := \boldsymbol{U} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{U}},\\
&\boldsymbol{b}_i := \boldsymbol{b}_i - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_i},\\
&\boldsymbol{b}_y := \boldsymbol{b}_y - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_y},
\end{aligned}$$

where $\eta > 0$ is the learning rate and the gradients are calculated by Eqs. (10), (17), (18), (20), (21), and (22).

3. Gradient Vanishing or Explosion in Long-term Dependencies

In recurrent neural networks, so as in deep neural networks, the final output is the composition of a large number of non-linear transformations. This results in the problem of either vanishing or exploding gradients in recurrent neural networks, especially for capturing long-term dependencies in sequence processing (Bengio et al., 1993; 1994). This problem is explained in the following. Recall Eq. (2) for a dynamical system:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t).$$

By induction, the hidden state at time t, i.e., $\boldsymbol{h}_t$, can be written as the previous T time steps. If the subscript t denotes the previous t time steps, we have by induction (Hochreiter, 1998; Hochreiter et al., 2001):

$$\boldsymbol{h}_1 = f_\theta\Big(f_\theta\big(\dots f_\theta(\boldsymbol{h}_T, \boldsymbol{x}_{T+1})\dots, \boldsymbol{x}_2\big), \boldsymbol{x}_1\Big).$$

Then, by the chain rule in derivatives, the derivative loss at time T, i.e., $\mathcal{L}_T$, is:

$$\frac{\partial \mathcal{L}_T}{\partial \theta} = \sum_{t \leq T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_t} \frac{\partial \boldsymbol{h}_t}{\partial \theta} \overset{(a)}{=} \sum_{t \leq T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_T} \frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_t} \frac{\partial \boldsymbol{h}_t}{\partial \theta} \overset{(2)}{=} \sum_{t \leq T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_T} \frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_t} \frac{\partial f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)}{\partial \theta}, \tag{23}$$

where (a) is because of the chain rule. In this expression, there is the derivative of $\boldsymbol{h}_T$ with respect to $\boldsymbol{h}_t$ which itself can be calculated by the chain rule:

$$\frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_t} = \frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_{T-1}} \times \frac{\partial \boldsymbol{h}_{T-1}}{\partial \boldsymbol{h}_{T-2}} \times \cdots \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}. \tag{24}$$

for capturing long-term dependencies in the sequence, T should be large. This means that in Eq. (24), and hence in Eq. (23), the number of multiplicand terms becomes huge. On the one hand, if each derivative is slightly smaller than one, the entire derivative in the chain rule becomes very small for multiplication of many terms smaller than one. This problem is referred to as gradient vanishing. On the other hand, if every derivative is slightly larger than one, the entire derivative in the chain rule explodes, resulting in the problem of exploding gradients. Note that gradient vanishing is more common than gradient explosion in recurrent networks.

There exist various attempts for resolving the problem of gradient vanishing or explosion (Hochreiter, 1998; Bengio et al., 2013). In the following, some of these attempts are introduced.

3.1. Close-to-identity Weight Matrix

As Eq. (4) shows, the state is multiplied by a weight matrix $\boldsymbol{W}$ at every time step and if there is long-term dependency, many of these $\boldsymbol{W}$ matrices are multiplied. Suppose the eigenvalue decomposition (Ghojogh et al., 2019a) of the matrix $\boldsymbol{W}$ is $\boldsymbol{W} = \boldsymbol{A}\boldsymbol{\Lambda}\boldsymbol{A}^\top$ where $\boldsymbol{A} \in \mathbb{R}^{p \times p}$ and $\boldsymbol{\Lambda} := \textbf{diag}([\lambda_1, \dots, \lambda_p]^\top)$ contain the eigenvectors and eigenvalues of $\boldsymbol{W}$, respectively. Eq. (4) is restated as:

$$\boldsymbol{h}_t = \tanh(\boldsymbol{A}\boldsymbol{\Lambda}\boldsymbol{A}^\top \boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i). \tag{25}$$

If a change ε in some element of the state $\boldsymbol{h}_{t-1}$ is aligned with an eigenvector of the weight matrix $\boldsymbol{W}$, then the effect of this change in $\boldsymbol{h}_t$ will be $(\lambda^t\, \varepsilon)$ after t time steps, according to Eq. (25). Two cases may happen:

- If the largest eigenvalue is less than one, i.e., $\lambda < 1$, then the change $(\lambda^t\, \varepsilon)$ is contrastive because $\lambda^t \ll 1$ for long-term dependencies. In this case, gradient vanishing occurs in long-term dependency and the network forgets very long time ago.
- If the largest eigenvalue is less than one, i.e., $\lambda > 1$, then the change $(\lambda^t\, \varepsilon)$ is diverging because $\lambda^t \gg 1$ for long-term dependencies. In this case, the gradient network forgets very long time ago. In this case, gradient explosion occurs in long-term dependency and remembering very long time ago dominates the short-term memories.

As remembering short-term memories is usually more important than remembering very past time in different tasks, it is recommended to use the weight matrix $\boldsymbol{W}$ whose largest eigenvalue is less than one; this makes the RNN have the Markovian property because it forgets very past after some point. However, if the largest eigenvalue of $\boldsymbol{W}$ is much less than one, i.e., $\lambda \ll 1$, gradient vanishing happens very sooner than expected. Therefore, it is recommended to use the weight matrix $\boldsymbol{W}$ whose largest eigenvalue is *slightly* less than one, i.e., $\lambda \lesssim 1$. This makes the RNN slightly contrastive. One way to have the weight matrix $\boldsymbol{W}$ whose largest eigenvalue is slightly less than one is to make this matrix close to the identity matrix (Mikolov et al., 2015).

There exist some other ways to determine the weight matrix $\boldsymbol{W}$. For example, the wight matrix can be set to be an orthogonal matrix (Arjovsky et al., 2016). Another approach is to copy the previous state exactly to the current state. In this approach, the Eq. (4) is modified to (Hu et al., 2018):

$$\begin{aligned} \boldsymbol{h}_t &= \tanh(\boldsymbol{W}\boldsymbol{h}_{t-1} + \boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i) \\ &= \tanh\big((\boldsymbol{W} + \boldsymbol{I})\boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i\big), \end{aligned} \tag{26}$$

where $\boldsymbol{I}$ is the identity matrix. This prevents gradient vanishing because it brings a copy of the previous step to the current state. This can also be interpreted as strengthening the diagonal of the weight matrix $\boldsymbol{W}$; hence, increasing the largest eigenvalue of $\boldsymbol{W}$ for preventing gradient vanishing.

3.2. Long Delays

As Eq. (4) and Fig. 1-c show, in the regular RNN, every state $\boldsymbol{h}_t$ is fed by its previous state $\boldsymbol{h}_{t-1}$ through the weight matrix $\boldsymbol{W}$. As discussed in Section 3.1, in the regular RNN, the effect of the change ε in a state results in $(\lambda^t\, \varepsilon)$ after t time steps, where λ is the largest eigenvalue of $\boldsymbol{W}$.

As shown in Fig. 1-c, the regular RNN has one-step connections or delays between the states. It is possible to have longer delays between the states in addition to the one-step delays (Lin et al., 1995). In other words, it is possible to have higher levels of Markov property in the network. Let $\boldsymbol{W}_k$ denote the weight matrix for k-step delays between the states. Then, Eq. (4) can be modified to:

$$\boldsymbol{h}_t = \tanh\Big(\sum_k \boldsymbol{W}_k \boldsymbol{h}_{t-k} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i\Big), \tag{27}$$

where the summation is over the k values for the existing

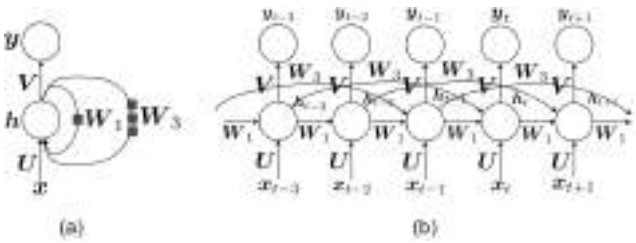


Figure 2. The (a) folded and (b) unfolded structures of an RNN with long delays. Every square on an edge means connection from one time slot before.

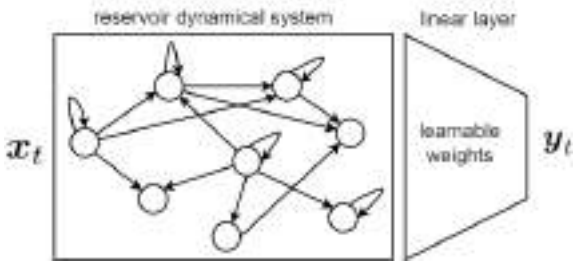


Figure 3. The echo state network.

delays in the RNN structure. An example for an RNN network with one-step and three-step delays is:

$$\boldsymbol{h}_t = \tanh\Big(\boldsymbol{W}_1\boldsymbol{h}_{t-1} + \boldsymbol{W}_3\boldsymbol{h}_{t-3} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i\Big),$$

which is illustrated in Fig. 2.

Having long delays in RNN is one of the attempts for preventing gradient vanishing (Lin et al., 1995). This is justified because every state is having impact not only from the previous state but also from the more previous states. Therefore, in backpropagation through time, there is some skip in gradient flow from a state to more previous states without the need to go through the middle states in the chain rule.

3.3. Leaky Units

Another way to resolve the problem of gradient vanishing is leaky units (Jaeger et al., 2007; Sutskever & Hinton, 2010). Let $h_{t,j}$ denote the j-th element of the state $\boldsymbol{h}_t \in [-1, 1]^p$. In leaky units, Eq. (4) is modified to the following element-wise equation:

$$\begin{aligned} h_{t,j} = &(1 - \frac{1}{\tau_j})\, h_{t-1,j} \\ &+ \frac{1}{\tau_j}\tanh(\boldsymbol{W}_{j:}\boldsymbol{h}_{t-1} + \boldsymbol{U}_{j:}\boldsymbol{x}_t + b_{i,j}), \end{aligned} \tag{28}$$

where $1 \leq \tau_j < \infty$ and $\boldsymbol{W}_{j:}$ is the j-th row of $\boldsymbol{W}$ and $\boldsymbol{U}_{j:}$ is the j-th row of $\boldsymbol{U}$ and $b_{i,j}$ is the j-th element of $\boldsymbol{b}_i$. When $\tau_i = 1$, then Eq. (28) becomes:

$$h_{t,j} = \frac{1}{\tau_j}\tanh(\boldsymbol{W}_{j:}\boldsymbol{h}_{t-1} + \boldsymbol{U}_{j:}\boldsymbol{x}_t + b_{i,j}),$$

which gives back Eq. (4) in the regular RNN. However, when $\tau_i \gg 1$, then Eq. (28) becomes:

$$h_{t,j} = h_{t-1,j},$$

which means that the previous state is copied to the current state. The larger the τ_i, the easier the gradient propagates for $h_{t,i}$. Therefore, by tuning τ_i, it is possible to control how much of the past should be directly copied and how much should be passed through the weight matrix. This can control the amount of gradient vanishing. Note that leaky units use different τ_i's because there may be a need to keep some of the directions of states (with $\tau_i = 1$) or forget some of the directions (with $\tau_i \gg 1$). In other words, it decides about the p directions of states separately.

3.4. Echo State Networks

One of the approaches to handle the problem of gradient vanishing in RNN is to use echo state networks (Jaeger & Haas, 2004; Jaeger, 2007). These networks consider the recurrent neural network as a black box having hidden units with nonlinear activation functions and connections between them. This black box of recurrent connections is called the *reservoir dynamical system* which models the internal structure of a computer or brain. The connections in the reservoir system are usually sparse and the weights of these connections are considered to be fixed. The output of the reservoir system is connected to an additional linear output layer whose weights are learnable. The echo state network minimizes the mean squared error in the output layer; hence, it performs linear regression in the last layer (Jaeger & Haas, 2004). This network is shown in Fig. 3. Because of not learning the recurrent weights in the reservoir system and sufficing to learn the weights of the output layer, the echo state network does not face the gradient vanishing problem. A tutorial on this topic is (Jaeger, 2002).

3.5. Other methods

There exist some other methods for not having gradient vanishing in recurrent networks. For example, hierarchical RNN (Hihi & Bengio, 1995) and deep RNN (Graves, 2013) have been proposed which stack several recurrent networks. Another related category of networks is the time-delay neural networks (Lang et al., 1990; Peddinti et al., 2015) which are used for shift-invariant sequence processing, especially used in speech recognition (Sugiyama et al., 1991). In these networks, every neuron in a layer receives a contextual window of the neurons in the previous layer as well as their delayed outputs in several time slots ago. Moreover, these networks apply backpropagation on several copies of the network shifted across the sequence

(time) so that the network becomes time-invariant.

4. Long Short-Term Memory Network

As the examples in Section 1 showed, we need short-term relations in some cases and long-term relations in some other cases. RNN learns the sequence based on one or several previous states, depending on its structure and the level of its Markov property (see Fig. 2). Therefore, we need to decide on the structure of RNN to be able to handle short-term or long-term dependencies in the sequence. Instead of manual design of the RNN structure or deciding manually when to clear the state, we can let the neural network learn by itself when to clear the state based on its input sequence. Long Short-Term Memory (LSTM), initially proposed in (Hochreiter & Schmidhuber, 1995; 1997), is able to do this; it learns from its input sequence when to use short-term dependency (i.e., when to clear the state) and when to use the long-term memory (i.e., when not to clear the state).

4.1. LSTM Gates and Cells

LSTM consists of several cells, each of which corresponds to a time slot (see Fig. 4). Every LSTM cell contains several gates for learning different aspects of the input time series (see Fig. 5). These gates are introduced in the following.

4.1.1. The Input Gate

One of the gates in the LSTM cell is the *input gate*, first proposed in (Hochreiter & Schmidhuber, 1995; 1997). This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{i}_t \in [0,1]^p$:

$$\boldsymbol{i}_t = \text{sig}\big(\boldsymbol{W}_i\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_i\, \boldsymbol{x}_t + (\boldsymbol{p}_i \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_i\big), \tag{29}$$

where $\boldsymbol{W}_i \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_i \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_i \in \mathbb{R}^p$ are the learnable weights for the input gate, $\odot$ denotes the Hadamard (element-wise) product, $\boldsymbol{c}_{t-1} \in \mathbb{R}^p$ is the final memory of the last time slot (which will be explained in Section 4.1.5), and $\boldsymbol{p}_i \in \mathbb{R}^p$ is the learnable *peephole weight* (Gers & Schmidhuber, 2000) letting a possible leak of information from the previous final memory. The function $\text{sig}(.) \in (0,1)$ is the sigmoid function which is applied element-wise:

$$\text{sig}(x) = \frac{1}{1+\exp(-x)}. \tag{30}$$

As Eq. (29) demonstrates, the input gate considers the effect of the input and the previous hidden state. It may also use a leak of information from the previous memory through the peephole. This gate carries the importance of the information of the input at the current time slot. The input gate is depicted in Fig. 5.

4.1.2. The Forget Gate

Another gate in the LSTM cell is the *forget gate*, first proposed in (Gers et al., 2000). This gate also takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{f}_t \in [0,1]^p$:

$$\boldsymbol{f}_t = \text{sig}\big(\boldsymbol{W}_f\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_f\, \boldsymbol{x}_t + (\boldsymbol{p}_f \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_f\big), \tag{31}$$

where $\boldsymbol{W}_f \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_f \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_f \in \mathbb{R}^p$ are the learnable weights for the forget gate, and $\boldsymbol{p}_f \in \mathbb{R}^p$ is the learnable *peephole weight* (Gers & Schmidhuber, 2000) letting a possible leak of information from the previous final memory.

As Eq. (31) shows, the forget gate considers the effect of the input and the previous hidden state, and perhaps a leak of information from the previous memory. This gate controls the amount of forgetting the previous information with respect to the new-coming information. the forget gate is illustrated in Fig. 5.

4.1.3. The Output Gate

The next gate in the LSTM cell is the *output gate* first proposed in (Hochreiter & Schmidhuber, 1995; 1997). This gate also takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{o}_t \in [0,1]^p$:

$$\boldsymbol{o}_t = \text{sig}\big(\boldsymbol{W}_o\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_o\, \boldsymbol{x}_t + (\boldsymbol{p}_o \odot \boldsymbol{c}_t) + \boldsymbol{b}_o\big), \tag{32}$$

where $\boldsymbol{W}_o \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_o \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_o \in \mathbb{R}^p$ are the learnable weights for the output gate, and $\boldsymbol{p}_o \in \mathbb{R}^p$ is the learnable *peephole weight* (Gers & Schmidhuber, 2000) letting a possible leak of information from the current final memory.

As shown in Eq. (32), the output gate considers the effect of the input and the previous hidden state, and a possible information leak from the current memory. The putput gate is shown in Fig. 5.

4.1.4. The New Memory Cell (Block Input)

The LSTM cell includes a gate named the *new memory cell*. This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\widetilde{\boldsymbol{c}}_t \in [-1,1]^p$. This gate considers the effect of the input and the previous hidden state to represent the new information of current input. It is formulated as:

$$\widetilde{\boldsymbol{c}}_t = \tanh(\boldsymbol{W}_c\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_c\, \boldsymbol{x}_t + \boldsymbol{b}_c), \tag{33}$$

where $\boldsymbol{W}_c \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_c \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_c$ are the learnable weights for the new memory cell. The new memory cell is also referred to as the *block input* in the literature (Greff et al., 2016). The signal $\widetilde{\boldsymbol{c}}_t$ is sometimes denoted by

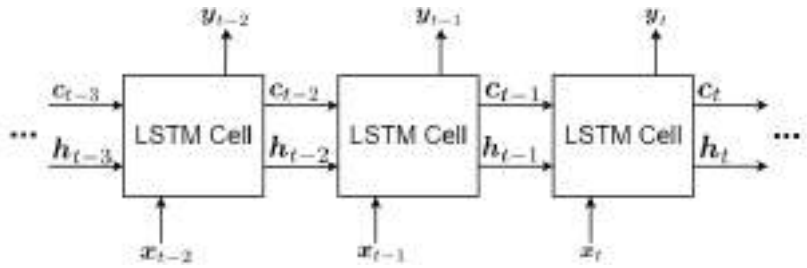


Figure 4. A sequence of LSTM cells processing the input sequence.

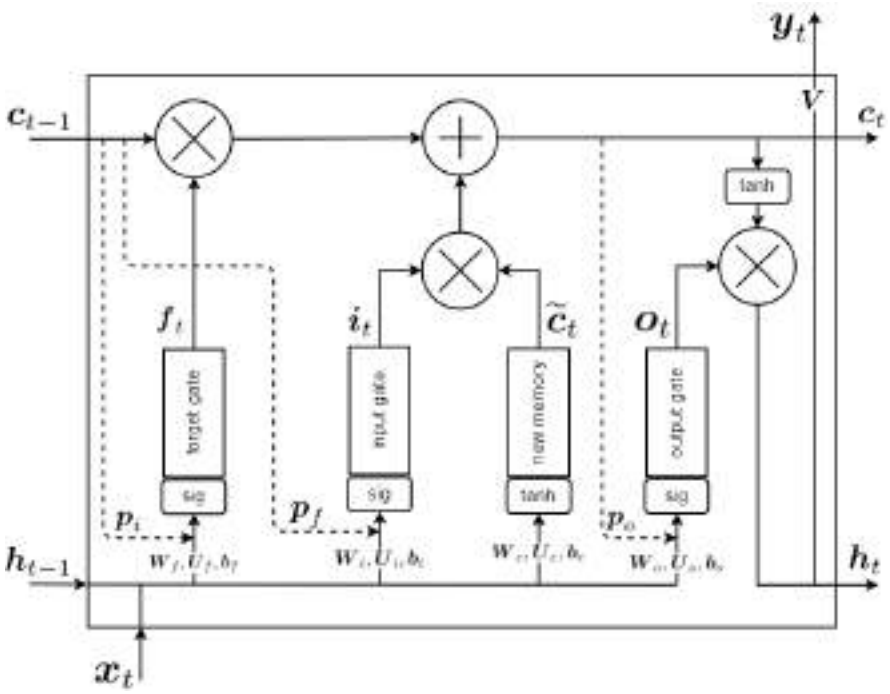


Figure 5. The conveyor belt in the LSTM cell.

$\boldsymbol{z}_t$ in the literature. The new memory cell is illustrated in Fig. 5.

4.1.5. The Final Memory Calculation

After computation of the outputs of the input gate $\boldsymbol{i}_t$, the forget gate $\boldsymbol{f}_t$, and the new memory cell $\widetilde{\boldsymbol{c}}_t$, we calculate the *final memory* $\boldsymbol{c}_t \in \mathbb{R}^p$:

$$\boldsymbol{c}_t = (\boldsymbol{f}_t \odot \boldsymbol{c}_{t-1}) + (\boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t), \tag{34}$$

where $\boldsymbol{c}_{t-1} \in \mathbb{R}^p$ is the final memory of the previous time slot.

As Eq. (34) demonstrates, the final memory considers the effect of the forget gate, the previous memory, the input, and the new memory. In the first term, i.e., $\boldsymbol{f}_t \odot \boldsymbol{c}_{t-1}$, the forget gate $\boldsymbol{f}_t \in [0,1]^p$ controls how much of the previous memory $\boldsymbol{c}_{t-1}$ should be forgotten. The closer the $\boldsymbol{f}_t$ is to zero, the more the network forgets the previous memory $\boldsymbol{c}_{t-1}$. In the second term, i.e., $\boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t$, the input gate $\boldsymbol{i}_t \in [0,1]^p$ and the new memory cell $\widetilde{\boldsymbol{c}}_t \in [-1,1]^p$ both control how much of the new input information should be used. The closer the input gate $\boldsymbol{i}_t$ is to one and the closer the new memory cell $\widetilde{\boldsymbol{c}}_t$ is to ± 1, the more the input information is used.

In other words, the first and second terms in Eq. (34) determine the trade-off of usage of old versus new information in the sequence. The weights of these gates are trained in a way that they pass or block the input/previous information based on the input sequence and the time step in the sequence. The final memory calculation is depicted in Fig. 5.

4.1.6. The Hidden State (Block Output)

After computation of the output of the output gate $\boldsymbol{o}_t$ and the final memory $\boldsymbol{c}_t$, we calculate the *hidden state* $\boldsymbol{h}_t \in$

$[-1, 1]^p$:

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t). \tag{35}$$

This hidden state is also considered as the *block output* of the LSTM cell, depicted in Fig. 5.

4.1.7. The Output

The output $\boldsymbol{y}_t \in \mathbb{R}^q$ of the LSTM cell is as follows:

$$\boldsymbol{y}_t = \boldsymbol{V}\boldsymbol{h}_t + \boldsymbol{b}_y, \tag{36}$$

where $\boldsymbol{V} \in \mathbb{R}^{q \times p}$ and the bias $\boldsymbol{b}_y \in \mathbb{R}^q$ are the learnable weights for the output. It is possible to use an activation function, such as Eq. (6), after this output signal. Figure 5 shows the output signal in the LSTM cell.

Note that in the literature, the output is sometimes considered to be equal to the hidden state, i.e., $\boldsymbol{y}_t = \boldsymbol{h}_t$, by setting $\boldsymbol{V} = \boldsymbol{I}$ (the identity matrix) and $\boldsymbol{b}_y = \boldsymbol{0}$ (the zero vector).

4.2. History and Variants of LSTM

LSTM has gone through various developments and improvements gradually (Greff et al., 2016). Some of the variants of LSTM do not have the peepholes. In this case, the Eqs. (29), (31), and (32) are simplified to:

$$\boldsymbol{i}_t = \text{sig}\big(\boldsymbol{W}_i\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_i\,\boldsymbol{x}_t + \boldsymbol{b}_i\big), \tag{37}$$

$$\boldsymbol{f}_t = \text{sig}\big(\boldsymbol{W}_f\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_f\,\boldsymbol{x}_t + \boldsymbol{b}_f\big), \tag{38}$$

$$\boldsymbol{o}_t = \text{sig}\big(\boldsymbol{W}_o\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_o\,\boldsymbol{x}_t + \boldsymbol{b}_o\big), \tag{39}$$

respectively. In the following, we review a history of variants of the LSTM networks.

4.2.1. Original LSTM

LSTM was originally proposed by Hochreiter and Schmidhuber in years 1995 to 1997 (Hochreiter & Schmidhuber, 1995; 1997). We call it the *original LSTM* (Hochreiter & Schmidhuber, 1997). The original LSTM had only the input and output gates, introduced in Sections 4.1.1 and 4.1.3, and it did not have a forget gate. It also did not contain the peepholes; therefore, its gates were Eqs. (37) and (39). The original LSTM trained the network using BPTT (introduced in Section 2.3) and a mixture of real-time recurrent learning (Robinson & Fallside, 1987; Williams, 1989).

4.2.2. Vanilla LSTM

Later, Gers *et. al.* (Gers et al., 2000; Gers & Schmidhuber, 2000) applied some changes to the original LSTM. The forget gate, introduced in Section 4.1.2, was proposed for the first time in (Gers et al., 2000) to let the network forget its previous states either completely or partially. The peephole connections, introduced in Sections 4.1.1, 4.1.2, and 4.1.3, were first proposed in (Gers & Schmidhuber, 2000). The peepholes let a possible leak of information from the previous or current final memory. This lets the memory control the gates.

These two papers (Gers et al., 2000; Gers & Schmidhuber, 2000) also incorporated the *full gate recurrence*, in which all gates receive additional recurrent inputs from all gates at the previous time step. In full gate recurrence, the Eqs. (29), (31), and (32) become:

$$\begin{aligned}\boldsymbol{i}_t = \text{sig}\big(&\boldsymbol{W}_i\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_i\,\boldsymbol{x}_t + (\boldsymbol{p}_i \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_i \\ &+ \boldsymbol{R}_{ii}\,\boldsymbol{i}_{t-1} + \boldsymbol{R}_{if}\,\boldsymbol{f}_{t-1} + \boldsymbol{R}_{io}\,\boldsymbol{o}_{t-1}\big).\end{aligned} \tag{40}$$

$$\begin{aligned}\boldsymbol{f}_t = \text{sig}\big(&\boldsymbol{W}_f\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_f\,\boldsymbol{x}_t + (\boldsymbol{p}_f \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_f \\ &+ \boldsymbol{R}_{fi}\,\boldsymbol{i}_{t-1} + \boldsymbol{R}_{ff}\,\boldsymbol{f}_{t-1} + \boldsymbol{R}_{fo}\,\boldsymbol{o}_{t-1}\big).\end{aligned} \tag{41}$$

$$\begin{aligned}\boldsymbol{o}_t = \text{sig}\big(&\boldsymbol{W}_o\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_o\,\boldsymbol{x}_t + (\boldsymbol{p}_o \odot \boldsymbol{c}_t) + \boldsymbol{b}_o \\ &+ \boldsymbol{R}_{oi}\,\boldsymbol{i}_{t-1} + \boldsymbol{R}_{of}\,\boldsymbol{f}_{t-1} + \boldsymbol{R}_{oo}\,\boldsymbol{o}_{t-1}\big),\end{aligned} \tag{42}$$

where $\boldsymbol{R}_{ii}, \boldsymbol{R}_{if}, \boldsymbol{R}_{io}, \boldsymbol{R}_{fi}, \boldsymbol{R}_{ff}, \boldsymbol{R}_{fo}, \boldsymbol{R}_{oi}, \boldsymbol{R}_{of}, \boldsymbol{R}_{oo} \in \mathbb{R}^{p \times p}$ are the learnable recurrent weights. Note that the full gate recurrence often disappeared in later papers on LSTM.

Later, Graves and Schmidhuber adapted the original LSTM and proposed the *vanilla LSTM* in 2005 (Graves & Schmidhuber, 2005a), which is one of the most common LSTMs in the literature. The vanilla LSTM incorporated the structures of the original LSTM (Hochreiter & Schmidhuber, 1997) and the papers (Gers et al., 2000; Gers & Schmidhuber, 2000). The full BPTT, introduced in Section 2.3, was used for LSTM in the vanilla LSTM (Graves & Schmidhuber, 2005a).

4.2.3. Other LSTM Variants

There are other variants of LSTM (Greff et al., 2016; Jozefowicz et al., 2015); although, the most common used LSTM is the vanilla LSTM (Graves & Schmidhuber, 2005a). BPTT was used for LSTM training in (Graves & Schmidhuber, 2005a); however, Kalman filtering was used for its training (Gers et al., 2002) before that. Another training method for LSTM was evolutionary learning (Schmidhuber et al., 2007). Context-sensitive evolutionary learning was also used for LSTM training (Bayer et al., 2009).

Later works tried to improve the performance of LSTM. For example, Sak *et. al.* added a linear layer which projects the output of the LSTM to smaller number of parameters before the recurrent connections (Sak et al., 2014). Doetsch *et. al.* converted the slope of activation functions of the LSTM gates to learnable parameters for performance improvement (Doetsch et al., 2014). *Dynamic cortex memory* was another LSTM variant which added connections between the gates within every LSTM cell (Otte et al., 2014). Finally in 2014, one the biggest improvements of LSTM was proposed, which was named the Gated Recurrent Units (GRU) (Cho et al., 2014). The philosophy of GRU was to simplify the LSTM cell because we may no need to have a very complicated cell to learn the sequence information. In

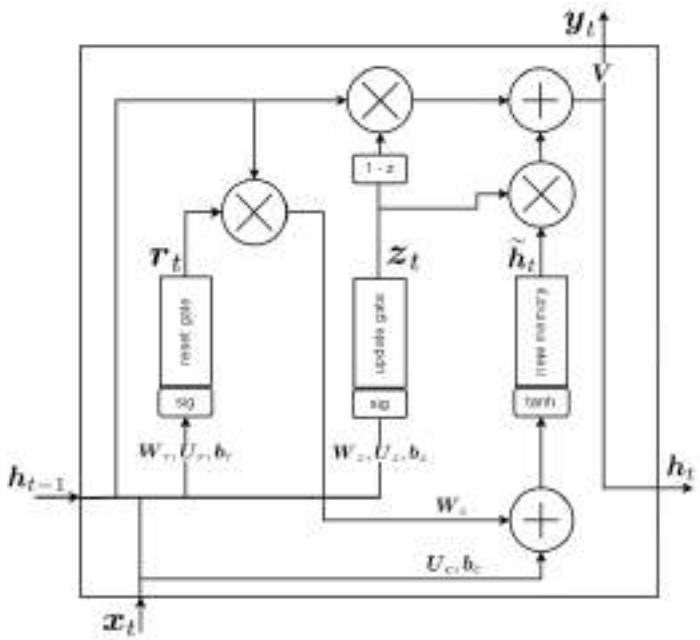


Figure 6. The conveyor belt in the GRU cell (the fully gated unit).

other words, GRU raised the question of whether we need to be that flexible like LSTM to learn the sequence. GRU is less flexible than LSTM but it is good enough for sequence learning.

GRU redesigned the LSTM cell by introducing reset gate, update gate, and new memory cell; therefore, the number of gates were reduced from four to three. It was empirically shown in (Chung et al., 2014) that the performance of LSTM improves by using GRU cells. Later in 2017, the GRU was further simplified by merging the reset and update gates into a forget gate (Heck & Salem, 2017). Nowadays, GRU is the most commonly used LSTM structure. Section 4.3 introduces the details of the GRU cell.

4.3. Gated Recurrent Units (GRU)

GRU was first proposed in (Cho et al., 2014). As was mentioned in Section 4.2.3, GRU simplified the LSTM cell in order to make the flexibility of LSTM.

4.3.1. Fully Gated Unit

The main GRU was the *fully gated unit* (Cho et al., 2014), whose gates are introduced in the following. Its gates are illustrated in Fig. 6.

– The Reset Gate: One of the gates in the GRU cell is the *reset gate*. This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1, 1]^p$, and outputs the signal $\boldsymbol{r}_t \in [0, 1]^p$:

$$\boldsymbol{r}_t = \text{sig}(\boldsymbol{W}_r\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_r\, \boldsymbol{x}_t + \boldsymbol{b}_r), \tag{43}$$

where $\boldsymbol{W}_r \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_r \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_r \in \mathbb{R}^p$ are the learnable weights for the reset gate. The reset gate considers the effect of the input and the previous hidden state, and it controls the amount of forgetting/resetting the previous information with respect to the new-coming information. Comparing Eqs. (38) and (43) shows that the reset gate in the GRU cell is similar to the forget gate in the LSTM cell. The reset gate is depicted in Fig. 6.

– The Update Gate: Another gate in the GRU cell is the *update gate*. This gate also takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1, 1]^p$, and outputs the signal $\boldsymbol{z}_t \in [0, 1]^p$:

$$\boldsymbol{z}_t = \text{sig}(\boldsymbol{W}_z\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_z\, \boldsymbol{x}_t + \boldsymbol{b}_z), \tag{44}$$

where $\boldsymbol{W}_z \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_z \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_z \in \mathbb{R}^p$ are the learnable weights for the update gate. The update gate considers the effect of the input and the previous hidden state, and it controls the amount of using the new input data for updating the cell by the coming information of sequence. Comparing Eqs. (37) and (44) shows that the update gate in the GRU cell is similar to the input gate in the LSTM cell. Figure 6 shows the update gate in the GRU cell.

– The New Memory Cell: The GRU cell includes a gate named the *new memory cell*. This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1, 1]^p$, and outputs the signal

$\widetilde{\boldsymbol{h}}_t \in [-1,1]^p$:

$$\widetilde{\boldsymbol{h}}_t = \tanh\Big(\boldsymbol{W}_c\,\big(\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}\big)\Big) + \boldsymbol{U}_c\,\boldsymbol{x}_t + \boldsymbol{b}_c\Big), \quad (45)$$

where $\boldsymbol{W}_c \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_c \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_c$ are the learnable weights for the new memory cell. This gate considers the effect of the input and the previous hidden state to represent the new information of current input. Comparing Eqs. (33) and (45) shows that the new memory cell in the GRU cell is similar to the new memory cell in the LSTM cell. Note that, in the LSTM cell, the hidden state (see Eq. (35)) and the new memory cell (see Eq. (33)) were different; however, the hidden state of the GRU cell (see Eq. (45)) replaces the new memory signal in the LSTM cell. The new memory cell is shown in Fig. 6.

– The Final Memory (Hidden State): After computation of the outputs of the update gate $\boldsymbol{z}_t$ and the new memory cell $\widetilde{\boldsymbol{h}}_t$, we calculate the *final memory* or the *hidden state* $\boldsymbol{h}_t \in \mathbb{R}^p$:

$$\boldsymbol{h}_t = \big((\boldsymbol{1} - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1}\big) + (\boldsymbol{z}_t \odot \widetilde{\boldsymbol{h}}_t), \quad (46)$$

where $\boldsymbol{h}_{t-1} \in \mathbb{R}^p$ is the hidden state of the previous time slot. The final memory block is illustrated in Fig. 6.

As Eq. (46) demonstrates, the final memory considers the effect of the update gate, the previous memory, and the new memory. In the first term, i.e., $(\boldsymbol{1} - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1}$, the update gate $\boldsymbol{z}_t \in [0,1]^p$ controls how much of the previous state $\boldsymbol{h}_{t-1}$ should be used based on the input data. The closer the $\boldsymbol{z}_t$ is to one (resp. zero), the more the network forgets (resp. considers) the previous state $\boldsymbol{h}_{t-1}$. In the second term, i.e., $\boldsymbol{z}_t \odot \widetilde{\boldsymbol{h}}_t$, the update gate $\boldsymbol{z}_t \in [0,1]^p$ and the new memory cell $\widetilde{\boldsymbol{h}}_t \in [-1,1]^p$ both control how much of the new input information should be used. In other words, it controls how much the information should be updated by the new information. The closer the update gate $\boldsymbol{z}_t$ is to one and the closer the new memory cell $\widetilde{\boldsymbol{h}}_t$ is to ± 1, the more the input information is used.

Overall, the first and second terms in Eq. (46) determine the trade-off of usage of old versus new information in the sequence. The weights of these gates are trained in a way that they pass or block the input/previous information based on the input sequence and the time step in the sequence. Comparing Eqs. (34) and (46) shows that the final memory in the GRU cell is in the form of the final memory in the LSTM cell; however, they have somewhat different functionality.

4.3.2. Minimal Gated Unit

Minimal gated unit (Heck & Salem, 2017) is another variant of GRU which has simplified the gate by merging the reset and update gates into a forget gate. This merging is possible because the forget gate can control both the previous and new information of the sequence.

– The Forget Gate: The *forget gate* takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{r}_t \in [0,1]^p$:

$$\boldsymbol{f}_t = \text{sig}(\boldsymbol{W}_f\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_f\,\boldsymbol{x}_t + \boldsymbol{b}_f), \quad (47)$$

where $\boldsymbol{W}_f \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_f \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_f \in \mathbb{R}^p$ are the learnable weights for the forget gate. The forget gate considers the effect of the input and the previous hidden state, and it controls the amount of forgetting the previous information with respect to the new-coming information. Therefore, it controls both forgetting or using the previous memory and using the new coming information.

– The New Memory Cell and the Final Memory: Because the forget gate replaces the reset and the update gate in the minimal gate unit, Eqs. (45) and (46) are changed to:

$$\widetilde{\boldsymbol{h}}_t = \tanh\Big(\boldsymbol{W}_c\,\big(\boldsymbol{f}_t \odot \boldsymbol{h}_{t-1}\big)\Big) + \boldsymbol{U}_c\,\boldsymbol{x}_t + \boldsymbol{b}_c\Big), \quad (48)$$

$$\boldsymbol{h}_t = \big((\boldsymbol{1} - \boldsymbol{f}_t) \odot \boldsymbol{h}_{t-1}\big) + (\boldsymbol{f}_t \odot \widetilde{\boldsymbol{h}}_t), \quad (49)$$

respectively, to be the new memory cell and the final memory in the minimal gate unit.

5. Bidirectional RNN and LSTM

5.1. Justification of Bidirectional Processing

A bidirectional RNN or LSTM network processes the sequence in both directions; left to right and right to left. In the first glance, online causal tasks such as reading a text or listening to a speech do not have access to the future. Therefore, bidirectional networks seem to violate causality in them. However, in many of these tasks, it is possible to wait for the completion of a part of the sequence such as a sentence and then decide about it. For example, it is normal to wait for the completion of sentence in speech recognition and then recognize it (Graves & Schmidhuber, 2005a;b). In text processing, the text is usually available except in a streaming text. Even in streaming text, it is possible to wait for a sentence to complete. Therefore, it makes sense to use bidirectional networks for processing sequences because, sometimes, the important related word comes after a word and not necessarily before it. An example for such a case is the sentence "The police is chasing the thief" where the word "thief" is a strongly related (opposite) word for the word "police". In this sentence, both the words "thief" and "police" are related and it is worth to process the sentence in both directions.

5.2. Bidirectional RNN

The bidirectional RNN was first proposed in (Schuster & Paliwal, 1997) and further exploited in (Baldi et al., 1999). It uses two sets of states each for one of the directions in

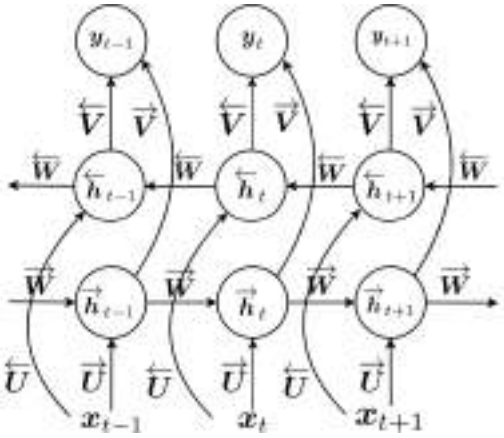


Figure 7. The bidirectional RNN.

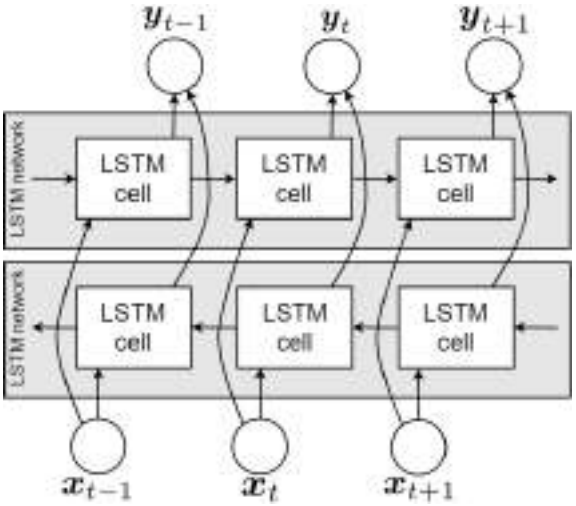


Figure 8. The bidirectional LSTM.

the sequence. Let the states for left-to-right and right-to-left processing be denoted by $\overrightarrow{\boldsymbol{h}}_t$ and $\overleftarrow{\boldsymbol{h}}_t$, respectively. In the bidirectional RNN, Eq. (4) is replaced by two equations (Graves et al., 2013):

$$\overrightarrow{\boldsymbol{h}}_t = \tanh(\overrightarrow{\boldsymbol{W}}\overrightarrow{\boldsymbol{h}}_{t-1} + \overrightarrow{\boldsymbol{U}}\boldsymbol{x}_t + \overrightarrow{\boldsymbol{b}}_i), \tag{50}$$

$$\overleftarrow{\boldsymbol{h}}_t = \tanh(\overleftarrow{\boldsymbol{W}}\overleftarrow{\boldsymbol{h}}_{t+1} + \overleftarrow{\boldsymbol{U}}\boldsymbol{x}_t + \overleftarrow{\boldsymbol{b}}_i), \tag{51}$$

and Eq. (5) is replaced by:

$$\boldsymbol{y}_t = \overrightarrow{\boldsymbol{V}}\overrightarrow{\boldsymbol{h}}_t + \overleftarrow{\boldsymbol{V}}\overleftarrow{\boldsymbol{h}}_t + \boldsymbol{b}_y, \tag{52}$$

where the arrows show the parameters for each direction of processing. The unfolding schematic of the bidirectional RNN is illustrated in Fig. 7. As this figure shows, the outputs of both directions are connected to an output layer. In some cases, this output layer may be replaced by a third multi-layer neural network. All weights of the bidirectional RNN are trained using backpropagation through time similarly to what was explained in Section 2.3. It is noteworthy that the deep variant of bidirectional RNN has been proposed in (Graves et al., 2013).

5.3. Bidirectional LSTM

Bidirectional LSTM was first proposed in (Graves & Schmidhuber, 2005a;b). As obvious from its name, the bidirectional LSTM includes two LSTM networks each of which processes the sequence from one direction. In other words, there are two LSTM networks which are fed with the sequence in opposite orders. This structure is depicted in Fig. 8. Experiments have shown that the bidirectional LSTM outperforms the unidirectional LSTM (Graves & Schmidhuber, 2005a; Graves et al., 2005). This is expected according to the justification in Section 5.1. Because of its advantages, various methods have combined the bidirectional LSTM with other methods. For example, a hybrid of the bidirectional LSTM and HMM (Ghojogh et al., 2019b) has shown its merit (Graves et al., 2005). Another example is the hybrid of bidirectional LSTM and Conditional Random Fields (CRF) (Lafferty et al., 2001) for the sequence tagging task (Huang et al., 2015). Other extensions of bidirectional LSTM networks exist such as (Peters et al., 2017).

5.4. Embeddings from Language Model (ELMo)

The Embeddings from Language Model (ELMo) network, first proposed in (Peters et al., 2018), is a language model which makes use of bidirectional LSTM networks. It is one of the successful context-aware language modeling networks. It has been widely used in various applications such as in medical interview processing (Sarzynska-Wawer et al., 2021).

The structure of ELMo is illustrated in Fig. 9. As shown in this figure, ELMo contains L layers of bidirectional LSTM networks where the output of each bidirectional LSTM is fed to the next bidirectional LSTM. in The bidirectional LSTM networks of ELMo, $\boldsymbol{V} = \boldsymbol{I}$ is set so that the outputs $\boldsymbol{y}$ becomes equal to the hidden states $\boldsymbol{h}$. At time slot t and layer l, the outputs (or hidden states) of the two directions of LSTM are concatenated together to make $\boldsymbol{h}_t^{(l)}$:

$$\boldsymbol{h}_t^{(l)} := [\overrightarrow{\boldsymbol{h}}_t^{(l)\top}, \overleftarrow{\boldsymbol{h}}_t^{(l)\top}]^\top.$$

Then, a linear combination of these hidden states of layers is considered to be the embedding vector of ELMo network at time t (Peters et al., 2018):

$$\boldsymbol{y}_t^{\text{ELMo}} := \gamma \sum_{l=1}^{L} s_l\, \boldsymbol{h}_t^{(l)}, \tag{53}$$

where γ and $\{s_l\}_{l=1}^L$ are the hyperparameter scalar weights which are determined according to the specific task (e.g.,

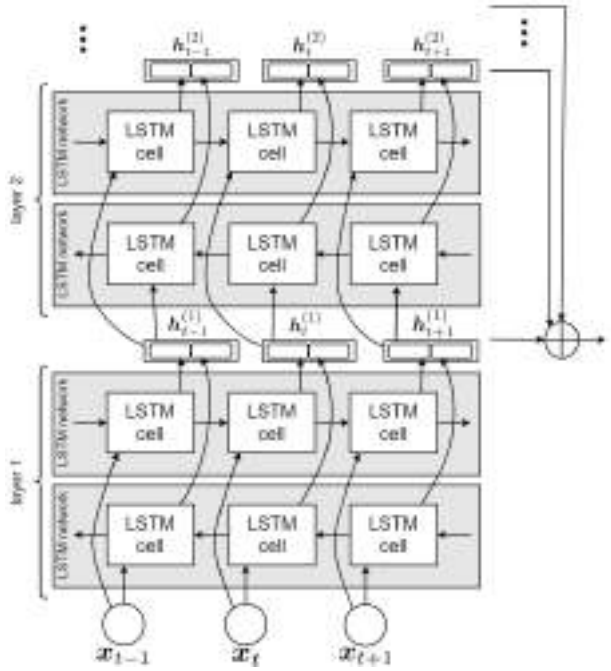


Figure 9. The ELMo network.

question answering, translation, etc) in natural language processing.

6. Conclusion

This was a tutorial paper on RNN, LSTM, and its variants. We covered dynamical system, backpropagation through time, LSTM gates and cells, history and variants of LSTM, the GRU cell, bidirectional RNN, bidirectional LSTM, and ELMo network.

References

Arjovsky, Martin, Shah, Amar, and Bengio, Yoshua. Unitary evolution recurrent neural networks. In *International conference on machine learning*, pp. 1120–1128. PMLR, 2016.

Baldi, Pierre, Brunak, Søren, Frasconi, Paolo, Soda, Giovanni, and Pollastri, Gianluca. Exploiting the past and the future in protein secondary structure prediction. *Bioinformatics*, 15(11):937–946, 1999.

Bayer, Justin, Wierstra, Daan, Togelius, Julian, and Schmidhuber, Jürgen. Evolving memory cell structures for sequence learning. In *International conference on artificial neural networks*, pp. 755–764. Springer, 2009.

Bengio, Yoshua, Frasconi, Paolo, and Simard, Patrice. The problem of learning long-term dependencies in recurrent networks. In *IEEE international conference on neural networks*, pp. 1183–1188. IEEE, 1993.

Bengio, Yoshua, Simard, Patrice, and Frasconi, Paolo. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2): 157–166, 1994.

Bengio, Yoshua, Boulanger-Lewandowski, Nicolas, and Pascanu, Razvan. Advances in optimizing recurrent networks. In *2013 IEEE international conference on acoustics, speech and signal processing*, pp. 8624–8628. IEEE, 2013.

Broer, Hendrik Wolter and Takens, Floris. *Dynamical systems and chaos*, volume 172. Springer, 2011.

Cho, Kyunghyun, Van Merriënboer, Bart, Bahdanau, Dzmitry, and Bengio, Yoshua. On the properties of neural machine translation: Encoder-decoder approaches. In *Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation (SSST-8)*, 2014.

Chung, Junyoung, Gulcehre, Caglar, Cho, KyungHyun, and Bengio, Yoshua. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.

Doetsch, Patrick, Kozielski, Michal, and Ney, Hermann. Fast and robust training of recurrent neural networks for offline handwriting recognition. In *2014 14th international conference on frontiers in handwriting recognition*, pp. 279–284. IEEE, 2014.

Gers, Felix A and Schmidhuber, Jürgen. Recurrent nets that time and count. In *Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks. IJCNN 2000. Neural Computing: New Challenges and Perspectives for the New Millennium*, volume 3, pp. 189–194. IEEE, 2000.

Gers, Felix A, Schmidhuber, Jürgen, and Cummins, Fred. Learning to forget: Continual prediction with LSTM. *Neural computation*, 12(10):2451–2471, 2000.

Gers, Felix A, Pérez-Ortiz, Juan Antonio, Eck, Douglas, and Schmidhuber, Jürgen. DEKF-LSTM. In *10th European Symposium on Artificial Neural Networks (ESANN)*, 2002.

Ghojogh, Benyamin, Karray, Fakhri, and Crowley, Mark. Eigenvalue and generalized eigenvalue problems: Tutorial. *arXiv preprint arXiv:1903.11240*, 2019a.

Ghojogh, Benyamin, Karray, Fakhri, and Crowley, Mark. Hidden Markov model: Tutorial. *Engineering Archive*, 2019b.

Ghojogh, Benyamin, Ghodsi, Ali, Karray, Fakhri, and Crowley, Mark. KKT conditions, first-order and second-order optimization, and distributed optimization: Tutorial and survey. *arXiv preprint arXiv:2110.01858*, 2021.

LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

The theory of reinforcement learning provides a normative account[1], deeply rooted in psychological[2] and neuroscientific[3] perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems[4,5], the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms[3]. While reinforcement learning agents have achieved some successes in a variety of domains[6–8], their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks[9–11] to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games[12]. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

We set out to create a single algorithm that would be able to develop a wide range of competencies on a varied range of challenging tasks—a central goal of general artificial intelligence[13] that has eluded previous efforts[8,14,15]. To achieve this, we developed a novel agent, a deep Q-network (DQN), which is able to combine reinforcement learning with a class of artificial neural network[16] known as deep neural networks. Notably, recent advances in deep neural networks[9–11], in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use one particularly successful architecture, the deep convolutional network[17], which uses hierarchical layers of tiled convolutional filters to mimic the effects of receptive fields—inspired by Hubel and Wiesel's seminal work on feedforward processing in early visual cortex[18]—thereby exploiting the local spatial correlations present in images, and building in robustness to natural transformations such as changes of viewpoint or scale.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal of the agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_{\pi} \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots \mid s_t = s,\ a_t = a,\ \pi\right],$$

which is the maximum sum of rewards r_t discounted by γ at each time-step t, achievable by a behaviour policy $\pi = P(a|s)$, after making an observation (s) and taking an action (a) (see Methods)[19].

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q) function[20]. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data distribution, and the correlations between the action-values (Q) and the target values $r + \gamma \max_{a'} Q(s', a')$. We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay[21–23] that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other stable methods exist for training neural networks in the reinforcement learning setting, such as neural fitted Q-iteration[24], these methods involve the repeated training of networks *de novo* on hundreds of iterations. Consequently, these methods, unlike our algorithm, are too inefficient to be used successfully with large neural networks. We parameterize an approximate value function $Q(s,a;\theta_i)$ using the deep convolutional neural network shown in Fig. 1, in which θ_i are the parameters (that is, weights) of the Q-network at iteration i. To perform experience replay we store the agent's experiences $e_t = (s_t, a_t, r_t, s_{t+1})$ at each time-step t in a data set $D_t = \{e_1, \ldots, e_t\}$. During learning, we apply Q-learning updates, on samples (or minibatches) of experience $(s,a,r,s') \sim U(D)$, drawn uniformly at random from the pool of stored samples. The Q-learning update at iteration i uses the following loss function:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathrm{U}(D)}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta_i^-) - Q(s,a;\theta_i)\right)^2\right]$$

in which γ is the discount factor determining the agent's horizon, θ_i are the parameters of the Q-network at iteration i and θ_i^- are the network parameters used to compute the target at iteration i. The target network parameters θ_i^- are only updated with the Q-network parameters (θ_i) every C steps and are held fixed between individual updates (see Methods).

To evaluate our DQN agent, we took advantage of the Atari 2600 platform, which offers a diverse array of tasks ($n = 49$) designed to be

[1]Google DeepMind, 5 New Street Square, London EC4A 3TW, UK.
*These authors contributed equally to this work.

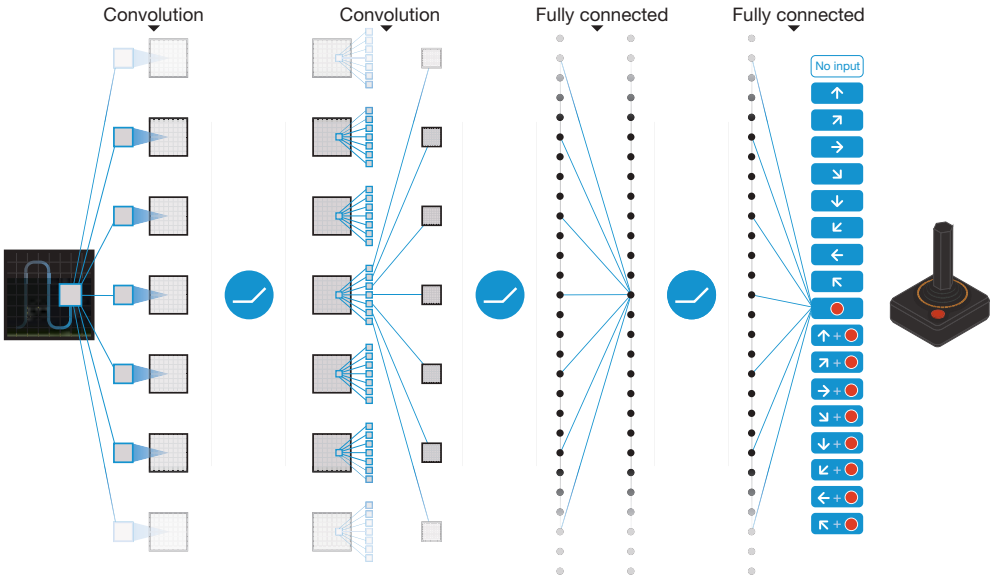


Figure 1 | Schematic illustration of the convolutional neural network. The details of the architecture are explained in the Methods. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ, followed by three convolutional layers (note: snaking blue line symbolizes sliding of each filter across input image) and two fully connected layers with a single output for each valid action. Each hidden layer is followed by a rectifier nonlinearity (that is, $\max(0,x)$).

difficult and engaging for human players. We used the same network architecture, hyperparameter values (see Extended Data Table 1) and learning procedure throughout—taking high-dimensional data (210×160 colour video at 60 Hz) as input—to demonstrate that our approach robustly learns successful policies over a variety of games based solely on sensory inputs with only very minimal prior knowledge (that is, merely the input data were visual images, and the number of actions available in each game, but not their correspondences; see Methods). Notably, our method was able to train large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner—illustrated by the temporal evolution of two indices of learning (the agent's average score-per-episode and average predicted Q-values; see Fig. 2 and Supplementary Discussion for details).

We compared DQN with the best performing methods from the reinforcement learning literature on the 49 games where results were available[12,15]. In addition to the learned agents, we also report scores for a professional human games tester playing under controlled conditions and a policy that selects actions uniformly at random (Extended Data Table 2 and Fig. 3, denoted by 100% (human) and 0% (random) on y axis; see Methods). Our DQN method outperforms the best existing reinforcement learning methods on 43 of the games without incorporating any of the additional prior knowledge about Atari 2600 games used by other approaches (for example, refs 12, 15). Furthermore, our DQN agent performed at a level that was comparable to that of a professional human games tester across the set of 49 games, achieving more than 75% of the human score on more than half of the games (29 games;

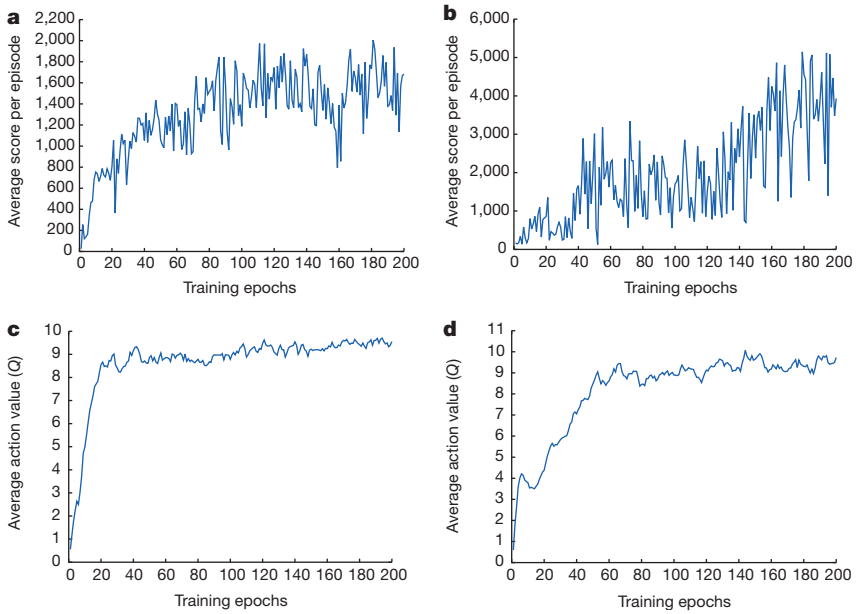


Figure 2 | Training curves tracking the agent's average score and average predicted action-value. a, Each point is the average score achieved per episode after the agent is run with ε-greedy policy ($\varepsilon = 0.05$) for 520 k frames on Space Invaders. **b**, Average score achieved per episode for Seaquest. **c**, Average predicted action-value on a held-out set of states on Space Invaders. Each point on the curve is the average of the action-value Q computed over the held-out set of states. Note that Q-values are scaled due to clipping of rewards (see Methods). **d**, Average predicted action-value on Seaquest. See Supplementary Discussion for details.

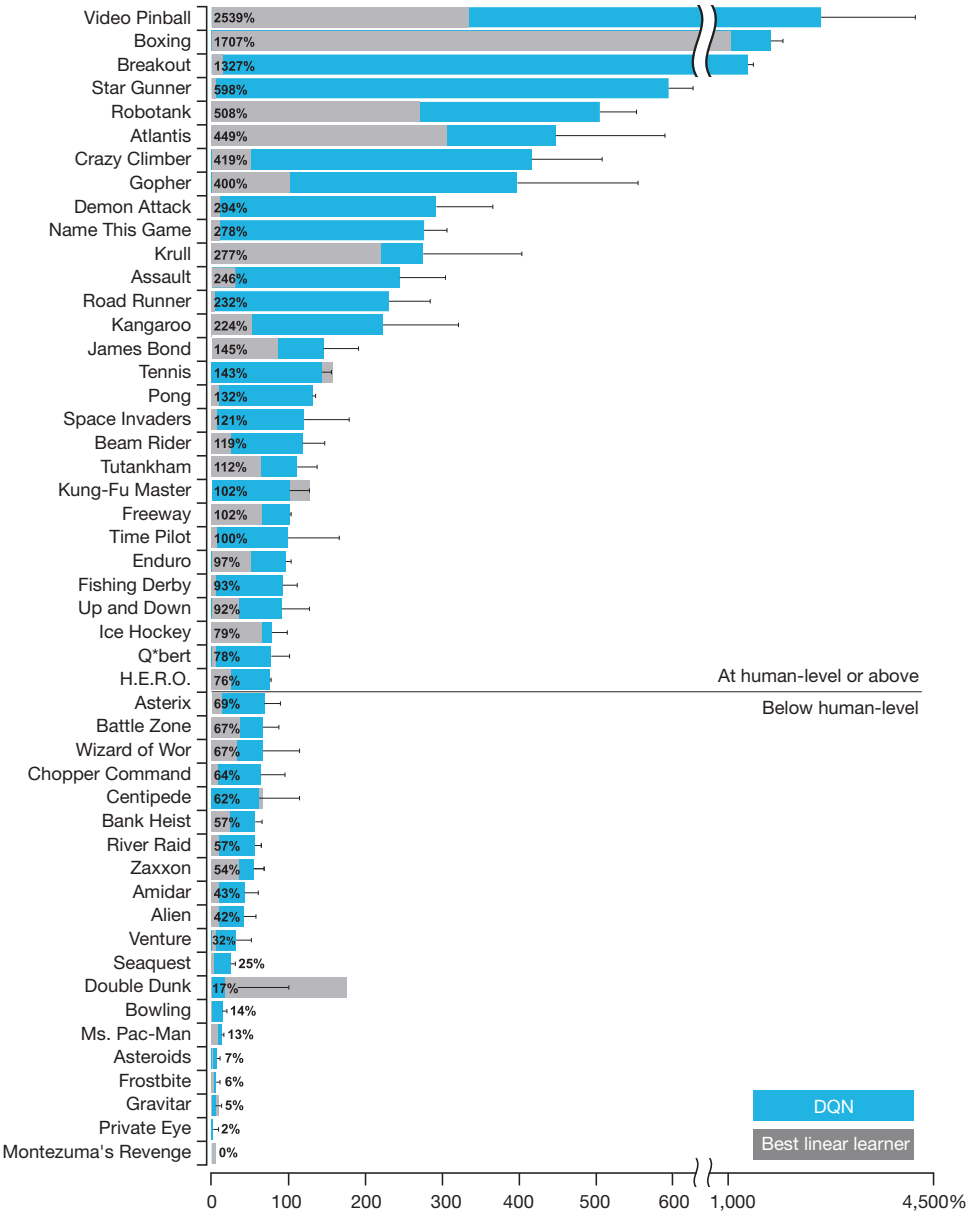


Figure 3 | Comparison of the DQN agent with the best reinforcement learning methods[15] in the literature. The performance of DQN is normalized with respect to a professional human games tester (that is, 100% level) and random play (that is, 0% level). Note that the normalized performance of DQN, expressed as a percentage, is calculated as: 100 × (DQN score − random play score)/(human score − random play score). It can be seen that DQN outperforms competing methods (also see Extended Data Table 2) in almost all the games, and performs at a level that is broadly comparable with or superior to a professional human games tester (that is, operationalized as a level of 75% or above) in the majority of games. Audio output was disabled for both human players and agents. Error bars indicate s.d. across the 30 evaluation episodes, starting with different initial conditions.

see Fig. 3, Supplementary Discussion and Extended Data Table 2). In additional simulations (see Supplementary Discussion and Extended Data Tables 3 and 4), we demonstrate the importance of the individual core components of the DQN agent—the replay memory, separate target Q-network and deep convolutional network architecture—by disabling them and demonstrating the detrimental effects on performance.

We next examined the representations learned by DQN that underpinned the successful performance of the agent in the context of the game Space Invaders (see Supplementary Video 1 for a demonstration of the performance of DQN), by using a technique developed for the visualization of high-dimensional data called 't-SNE'[25] (Fig. 4). As expected, the t-SNE algorithm tends to map the DQN representation of perceptually similar states to nearby points. Interestingly, we also found instances in which the t-SNE algorithm generated similar embeddings for DQN representations of states that are close in terms of expected reward but perceptually dissimilar (Fig. 4, bottom right, top left and middle), consistent with the notion that the network is able to learn representations that support adaptive behaviour from high-dimensional sensory inputs. Furthermore, we also show that the representations learned by DQN are able to generalize to data generated from policies other than its own—in simulations where we presented as input to the network game states experienced during human and agent play, recorded the representations of the last hidden layer, and visualized the embeddings generated by the t-SNE algorithm (Extended Data Fig. 1 and Supplementary Discussion). Extended Data Fig. 2 provides an additional illustration of how the representations learned by DQN allow it to accurately predict state and action values.

It is worth noting that the games in which DQN excels are extremely varied in their nature, from side-scrolling shooters (River Raid) to boxing games (Boxing) and three-dimensional car-racing games (Enduro).

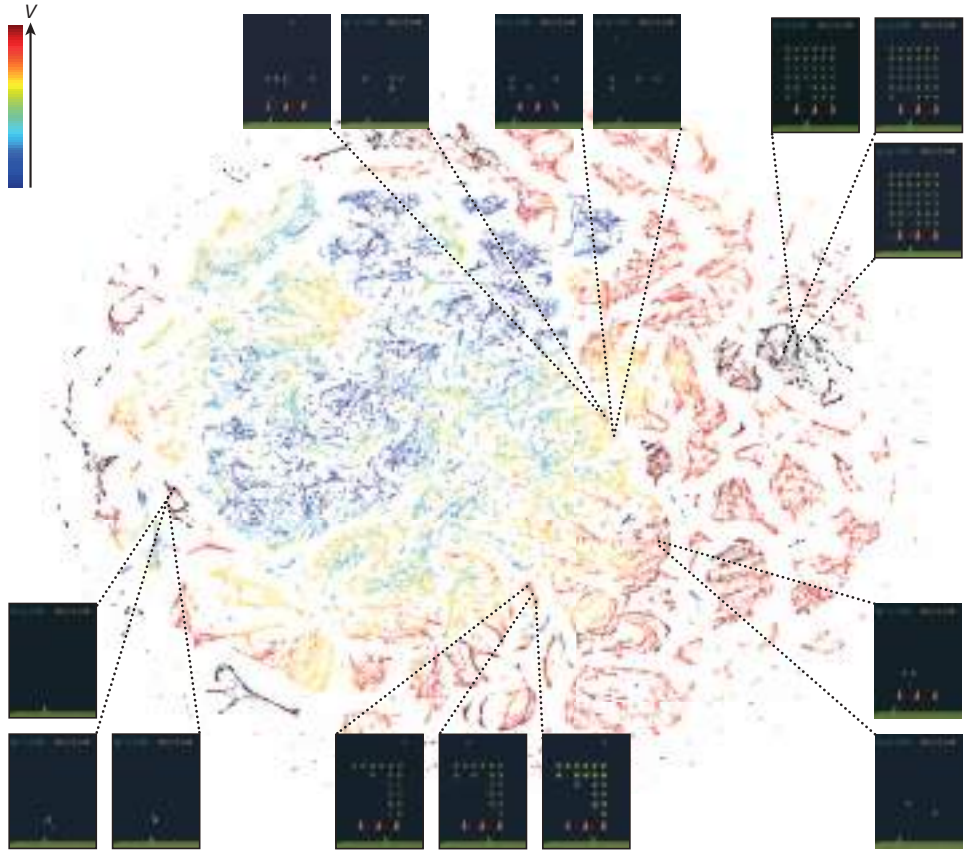


Figure 4 | Two-dimensional t-SNE embedding of the representations in the last hidden layer assigned by DQN to game states experienced while playing Space Invaders. The plot was generated by letting the DQN agent play for 2 h of real game time and running the t-SNE algorithm[25] on the last hidden layer representations assigned by DQN to each experienced game state. The points are coloured according to the state values (*V*, maximum expected reward of a state) predicted by DQN for the corresponding game states (ranging from dark red (highest *V*) to dark blue (lowest *V*)). The screenshots corresponding to a selected number of points are shown. The DQN agent predicts high state values for both full (top right screenshots) and nearly complete screens (bottom left screenshots) because it has learned that completing a screen leads to a new screen full of enemy ships. Partially completed screens (bottom screenshots) are assigned lower state values because less immediate reward is available. The screens shown on the bottom right and top left and middle are less perceptually similar than the other examples but are still mapped to nearby representations and similar values because the orange bunkers do not carry great significance near the end of a level. With permission from Square Enix Limited.

Indeed, in certain games DQN is able to discover a relatively long-term strategy (for example, Breakout: the agent learns the optimal strategy, which is to first dig a tunnel around the side of the wall allowing the ball to be sent around the back to destroy a large number of blocks; see Supplementary Video 2 for illustration of development of DQN's performance over the course of training). Nevertheless, games demanding more temporally extended planning strategies still constitute a major challenge for all existing agents including DQN (for example, Montezuma's Revenge).

In this work, we demonstrate that a single architecture can successfully learn control policies in a range of different environments with only very minimal prior knowledge, receiving only the pixels and the game score as inputs, and using the same algorithm, network architecture and hyperparameters on each game, privy only to the inputs a human player would have. In contrast to previous work[24,26], our approach incorporates 'end-to-end' reinforcement learning that uses reward to continuously shape representations within the convolutional network towards salient features of the environment that facilitate value estimation. This principle draws on neurobiological evidence that reward signals during perceptual learning may influence the characteristics of representations within primate visual cortex[27,28]. Notably, the successful integration of reinforcement learning with deep network architectures was critically dependent on our incorporation of a replay algorithm[21–23] involving the storage and representation of recently experienced transitions. Convergent evidence suggests that the hippocampus may support the physical realization of such a process in the mammalian brain, with the time-compressed reactivation of recently experienced trajectories during offline periods[21,22] (for example, waking rest) providing a putative mechanism by which value functions may be efficiently updated through interactions with the basal ganglia[22]. In the future, it will be important to explore the potential use of biasing the content of experience replay towards salient events, a phenomenon that characterizes empirically observed hippocampal replay[29], and relates to the notion of 'prioritized sweeping'[30] in reinforcement learning. Taken together, our work illustrates the power of harnessing state-of-the-art machine learning techniques with biologically inspired mechanisms to create agents that are capable of learning to master a diverse array of challenging tasks.

Online Content Methods, along with any additional Extended Data display items and Source Data, are available in the online version of the paper; references unique to these sections appear only in the online paper.

Received 10 July 2014; accepted 16 January 2015.

1. Sutton, R. & Barto, A. *Reinforcement Learning: An Introduction* (MIT Press, 1998).
2. Thorndike, E. L. *Animal Intelligence: Experimental studies* (Macmillan, 1911).
3. Schultz, W., Dayan, P. & Montague, P. R. A neural substrate of prediction and reward. *Science* **275,** 1593–1599 (1997).
4. Serre, T., Wolf, L. & Poggio, T. Object recognition with features inspired by visual cortex. *Proc. IEEE. Comput. Soc. Conf. Comput. Vis. Pattern. Recognit.* 994–1000 (2005).
5. Fukushima, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybern.* **36,** 193–202 (1980).

METHODS

Preprocessing. Working directly with raw Atari 2600 frames, which are 210×160 pixel images with a 128-colour palette, can be demanding in terms of computation and memory requirements. We apply a basic preprocessing step aimed at reducing the input dimensionality and dealing with some artefacts of the Atari 2600 emulator. First, to encode a single frame we take the maximum value for each pixel colour value over the frame being encoded and the previous frame. This was necessary to remove flickering that is present in games where some objects appear only in even frames while other objects appear only in odd frames, an artefact caused by the limited number of sprites Atari 2600 can display at once. Second, we then extract the Y channel, also known as luminance, from the RGB frame and rescale it to 84×84. The function ϕ from algorithm 1 described below applies this preprocessing to the m most recent frames and stacks them to produce the input to the Q-function, in which $m = 4$, although the algorithm is robust to different values of m (for example, 3 or 5).

Code availability. The source code can be accessed at https://sites.google.com/a/deepmind.com/dqn for non-commercial uses only.

Model architecture. There are several possible ways of parameterizing Q using a neural network. Because Q maps history–action pairs to scalar estimates of their Q-value, the history and the action have been used as inputs to the neural network by some previous approaches[24,26]. The main drawback of this type of architecture is that a separate forward pass is required to compute the Q-value of each action, resulting in a cost that scales linearly with the number of actions. We instead use an architecture in which there is a separate output unit for each possible action, and only the state representation is an input to the neural network. The outputs correspond to the predicted Q-values of the individual actions for the input state. The main advantage of this type of architecture is the ability to compute Q-values for all possible actions in a given state with only a single forward pass through the network.

The exact architecture, shown schematically in Fig. 1, is as follows. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ. The first hidden layer convolves 32 filters of 8×8 with stride 4 with the input image and applies a rectifier nonlinearity[31,32]. The second hidden layer convolves 64 filters of 4×4 with stride 2, again followed by a rectifier nonlinearity. This is followed by a third convolutional layer that convolves 64 filters of 3×3 with stride 1 followed by a rectifier. The final hidden layer is fully-connected and consists of 512 rectifier units. The output layer is a fully-connected linear layer with a single output for each valid action. The number of valid actions varied between 4 and 18 on the games we considered.

Training details. We performed experiments on 49 Atari 2600 games where results were available for all other comparable methods[12,15]. A different network was trained on each game: the same network architecture, learning algorithm and hyperparameter settings (see Extended Data Table 1) were used across all games, showing that our approach is robust enough to work on a variety of games while incorporating only minimal prior knowledge (see below). While we evaluated our agents on unmodified games, we made one change to the reward structure of the games during training only. As the scale of scores varies greatly from game to game, we clipped all positive rewards at 1 and all negative rewards at -1, leaving 0 rewards unchanged. Clipping the rewards in this manner limits the scale of the error derivatives and makes it easier to use the same learning rate across multiple games. At the same time, it could affect the performance of our agent since it cannot differentiate between rewards of different magnitude. For games where there is a life counter, the Atari 2600 emulator also sends the number of lives left in the game, which is then used to mark the end of an episode during training.

In these experiments, we used the RMSProp (see http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf) algorithm with minibatches of size 32. The behaviour policy during training was ε-greedy with ε annealed linearly from 1.0 to 0.1 over the first million frames, and fixed at 0.1 thereafter. We trained for a total of 50 million frames (that is, around 38 days of game experience in total) and used a replay memory of 1 million most recent frames.

Following previous approaches to playing Atari 2600 games, we also use a simple frame-skipping technique[15]. More precisely, the agent sees and selects actions on every kth frame instead of every frame, and its last action is repeated on skipped frames. Because running the emulator forward for one step requires much less computation than having the agent select an action, this technique allows the agent to play roughly k times more games without significantly increasing the runtime. We use $k = 4$ for all games.

The values of all the hyperparameters and optimization parameters were selected by performing an informal search on the games Pong, Breakout, Seaquest, Space Invaders and Beam Rider. We did not perform a systematic grid search owing to the high computational cost. These parameters were then held fixed across all other games. The values and descriptions of all hyperparameters are provided in Extended Data Table 1.

Our experimental setup amounts to using the following minimal prior knowledge: that the input data consisted of visual images (motivating our use of a convolutional deep network), the game-specific score (with no modification), number of actions, although not their correspondences (for example, specification of the up 'button') and the life count.

Evaluation procedure. The trained agents were evaluated by playing each game 30 times for up to 5 min each time with different initial random conditions ('no-op'; see Extended Data Table 1) and an ε-greedy policy with $\varepsilon = 0.05$. This procedure is adopted to minimize the possibility of overfitting during evaluation. The random agent served as a baseline comparison and chose a random action at 10 Hz which is every sixth frame, repeating its last action on intervening frames. 10 Hz is about the fastest that a human player can select the 'fire' button, and setting the random agent to this frequency avoids spurious baseline scores in a handful of the games. We did also assess the performance of a random agent that selected an action at 60 Hz (that is, every frame). This had a minimal effect: changing the normalized DQN performance by more than 5% in only six games (Boxing, Breakout, Crazy Climber, Demon Attack, Krull and Robotank), and in all these games DQN outperformed the expert human by a considerable margin.

The professional human tester used the same emulator engine as the agents, and played under controlled conditions. The human tester was not allowed to pause, save or reload games. As in the original Atari 2600 environment, the emulator was run at 60 Hz and the audio output was disabled: as such, the sensory input was equated between human player and agents. The human performance is the average reward achieved from around 20 episodes of each game lasting a maximum of 5 min each, following around 2 h of practice playing each game.

Algorithm. We consider tasks in which an agent interacts with an environment, in this case the Atari emulator, in a sequence of actions, observations and rewards. At each time-step the agent selects an action a_t from the set of legal game actions, $\mathcal{A} = \{1, \ldots, K\}$. The action is passed to the emulator and modifies its internal state and the game score. In general the environment may be stochastic. The emulator's internal state is not observed by the agent; instead the agent observes an image $x_t \in \mathbb{R}^d$ from the emulator, which is a vector of pixel values representing the current screen. In addition it receives a reward r_t representing the change in game score. Note that in general the game score may depend on the whole previous sequence of actions and observations; feedback about an action may only be received after many thousands of time-steps have elapsed.

Because the agent only observes the current screen, the task is partially observed[33] and many emulator states are perceptually aliased (that is, it is impossible to fully understand the current situation from only the current screen x_t). Therefore, sequences of actions and observations, $s_t = x_1, a_1, x_2, \ldots, a_{t-1}, x_t$, are input to the algorithm, which then learns game strategies depending upon these sequences. All sequences in the emulator are assumed to terminate in a finite number of time-steps. This formalism gives rise to a large but finite Markov decision process (MDP) in which each sequence is a distinct state. As a result, we can apply standard reinforcement learning methods for MDPs, simply by using the complete sequence s_t as the state representation at time t.

The goal of the agent is to interact with the emulator by selecting actions in a way that maximizes future rewards. We make the standard assumption that future rewards are discounted by a factor of γ per time-step (γ was set to 0.99 throughout), and define the future discounted return at time t as $R_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$, in which T is the time-step at which the game terminates. We define the optimal action-value function $Q^*(s,a)$ as the maximum expected return achievable by following any policy, after seeing some sequence s and then taking some action a, $Q^*(s,a) = \max_\pi \mathbb{E}[R_t | s_t = s, a_t = a, \pi]$ in which π is a policy mapping sequences to actions (or distributions over actions).

The optimal action-value function obeys an important identity known as the Bellman equation. This is based on the following intuition: if the optimal value $Q^*(s',a')$ of the sequence s' at the next time-step was known for all possible actions a', then the optimal strategy is to select the action a' maximizing the expected value of $r + \gamma Q^*(s',a')$:

$$Q^*(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^*(s',a') | s,a \right]$$

The basic idea behind many reinforcement learning algorithms is to estimate the action-value function by using the Bellman equation as an iterative update, $Q_{i+1}(s,a) = \mathbb{E}_{s'}[r + \gamma \max_{a'} Q_i(s',a') | s,a]$. Such value iteration algorithms converge to the optimal action-value function, $Q_i \rightarrow Q^*$ as $i \rightarrow \infty$. In practice, this basic approach is impractical, because the action-value function is estimated separately for each sequence, without any generalization. Instead, it is common to use a function approximator to estimate the action-value function, $Q(s,a;\theta) \approx Q^*(s,a)$. In the reinforcement learning community this is typically a linear function approximator, but

sometimes a nonlinear function approximator is used instead, such as a neural network. We refer to a neural network function approximator with weights θ as a Q-network. A Q-network can be trained by adjusting the parameters θ_i at iteration i to reduce the mean-squared error in the Bellman equation, where the optimal target values $r+\gamma \max_{a'} Q^*(s',a')$ are substituted with approximate target values $y=r+\gamma \max_{a'} Q(s',a';\theta_i^-)$, using parameters θ_i^- from some previous iteration. This leads to a sequence of loss functions $L_i(\theta_i)$ that changes at each iteration i,

$$\begin{aligned} L_i(\theta_i) &= \mathbb{E}_{s,a,r}\left[(\mathbb{E}_{s'}[y|s,a]-Q(s,a;\theta_i))^2\right] \\ &= \mathbb{E}_{s,a,r,s'}\left[(y-Q(s,a;\theta_i))^2\right]+\mathbb{E}_{s,a,r}[\mathbb{V}_{s'}[y]]. \end{aligned}$$

Note that the targets depend on the network weights; this is in contrast with the targets used for supervised learning, which are fixed before learning begins. At each stage of optimization, we hold the parameters from the previous iteration θ_i^- fixed when optimizing the ith loss function $L_i(\theta_i)$, resulting in a sequence of well-defined optimization problems. The final term is the variance of the targets, which does not depend on the parameters θ_i that we are currently optimizing, and may therefore be ignored. Differentiating the loss function with respect to the weights we arrive at the following gradient:

$$\nabla_{\theta_i} L(\theta_i) = \mathbb{E}_{s,a,r,s'}\left[\left(r+\gamma \max_{a'} Q(s',a';\theta_i^-)-Q(s,a;\theta_i)\right)\nabla_{\theta_i} Q(s,a;\theta_i)\right].$$

Rather than computing the full expectations in the above gradient, it is often computationally expedient to optimize the loss function by stochastic gradient descent. The familiar Q-learning algorithm[19] can be recovered in this framework by updating the weights after every time step, replacing the expectations using single samples, and setting $\theta_i^- = \theta_{i-1}$.

Note that this algorithm is model-free: it solves the reinforcement learning task directly using samples from the emulator, without explicitly estimating the reward and transition dynamics $P(r,s'|s,a)$. It is also off-policy: it learns about the greedy policy $a=\mathrm{argmax}_{a'} Q(s,a';\theta)$, while following a behaviour distribution that ensures adequate exploration of the state space. In practice, the behaviour distribution is often selected by an ε-greedy policy that follows the greedy policy with probability $1-\varepsilon$ and selects a random action with probability ε.

Training algorithm for deep Q-networks. The full algorithm for training deep Q-networks is presented in Algorithm 1. The agent selects and executes actions according to an ε-greedy policy based on Q. Because using histories of arbitrary length as inputs to a neural network can be difficult, our Q-function instead works on a fixed length representation of histories produced by the function ϕ described above. The algorithm modifies standard online Q-learning in two ways to make it suitable for training large neural networks without diverging.

First, we use a technique known as experience replay[23] in which we store the agent's experiences at each time-step, $e_t=(s_t,a_t,r_t,s_{t+1})$, in a data set $D_t=\{e_1,\ldots,e_t\}$, pooled over many episodes (where the end of an episode occurs when a terminal state is reached) into a replay memory. During the inner loop of the algorithm, we apply Q-learning updates, or minibatch updates, to samples of experience, $(s,a,r,s')\sim U(D)$, drawn at random from the pool of stored samples. This approach has several advantages over standard online Q-learning. First, each step of experience is potentially used in many weight updates, which allows for greater data efficiency. Second, learning directly from consecutive samples is inefficient, owing to the strong correlations between the samples; randomizing the samples breaks these correlations and therefore reduces the variance of the updates. Third, when learning on-policy the current parameters determine the next data sample that the parameters are trained on. For example, if the maximizing action is to move left then the training samples will be dominated by samples from the left-hand side; if the maximizing action then switches to the right then the training distribution will also switch. It is easy to see how unwanted feedback loops may arise and the parameters could get stuck in a poor local minimum, or even diverge catastrophically[20]. By using experience replay the behaviour distribution is averaged over many of its previous states, smoothing out learning and avoiding oscillations or divergence in the parameters. Note that when learning by experience replay, it is necessary to learn off-policy (because our current parameters are different to those used to generate the sample), which motivates the choice of Q-learning.

In practice, our algorithm only stores the last N experience tuples in the replay memory, and samples uniformly at random from D when performing updates. This approach is in some respects limited because the memory buffer does not differentiate important transitions and always overwrites with recent transitions owing to the finite memory size N. Similarly, the uniform sampling gives equal importance to all transitions in the replay memory. A more sophisticated sampling strategy might emphasize transitions from which we can learn the most, similar to prioritized sweeping[30].

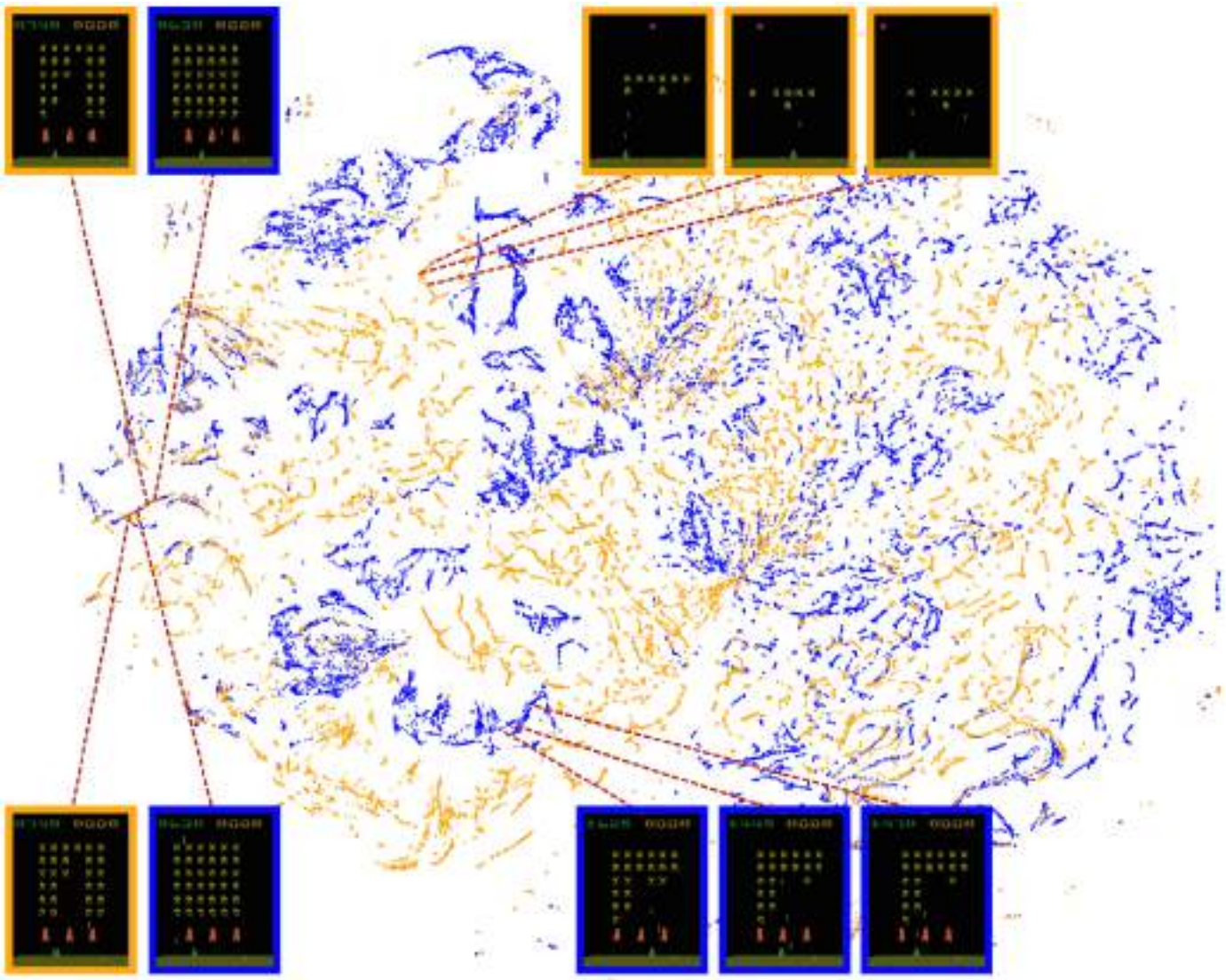
The second modification to online Q-learning aimed at further improving the stability of our method with neural networks is to use a separate network for generating the targets y_j in the Q-learning update. More precisely, every C updates we clone the network Q to obtain a target network $\hat{Q}$ and use $\hat{Q}$ for generating the Q-learning targets y_j for the following C updates to Q. This modification makes the algorithm more stable compared to standard online Q-learning, where an update that increases $Q(s_t,a_t)$ often also increases $Q(s_{t+1},a)$ for all a and hence also increases the target y_j, possibly leading to oscillations or divergence of the policy. Generating the targets using an older set of parameters adds a delay between the time an update to Q is made and the time the update affects the targets y_j, making divergence or oscillations much more unlikely.

We also found it helpful to clip the error term from the update $r+\gamma \max_{a'} Q(s',a';\theta_i^-)-Q(s,a;\theta_i)$ to be between -1 and 1. Because the absolute value loss function $|x|$ has a derivative of -1 for all negative values of x and a derivative of 1 for all positive values of x, clipping the squared error to be between -1 and 1 corresponds to using an absolute value loss function for errors outside of the $(-1,1)$ interval. This form of error clipping further improved the stability of the algorithm.

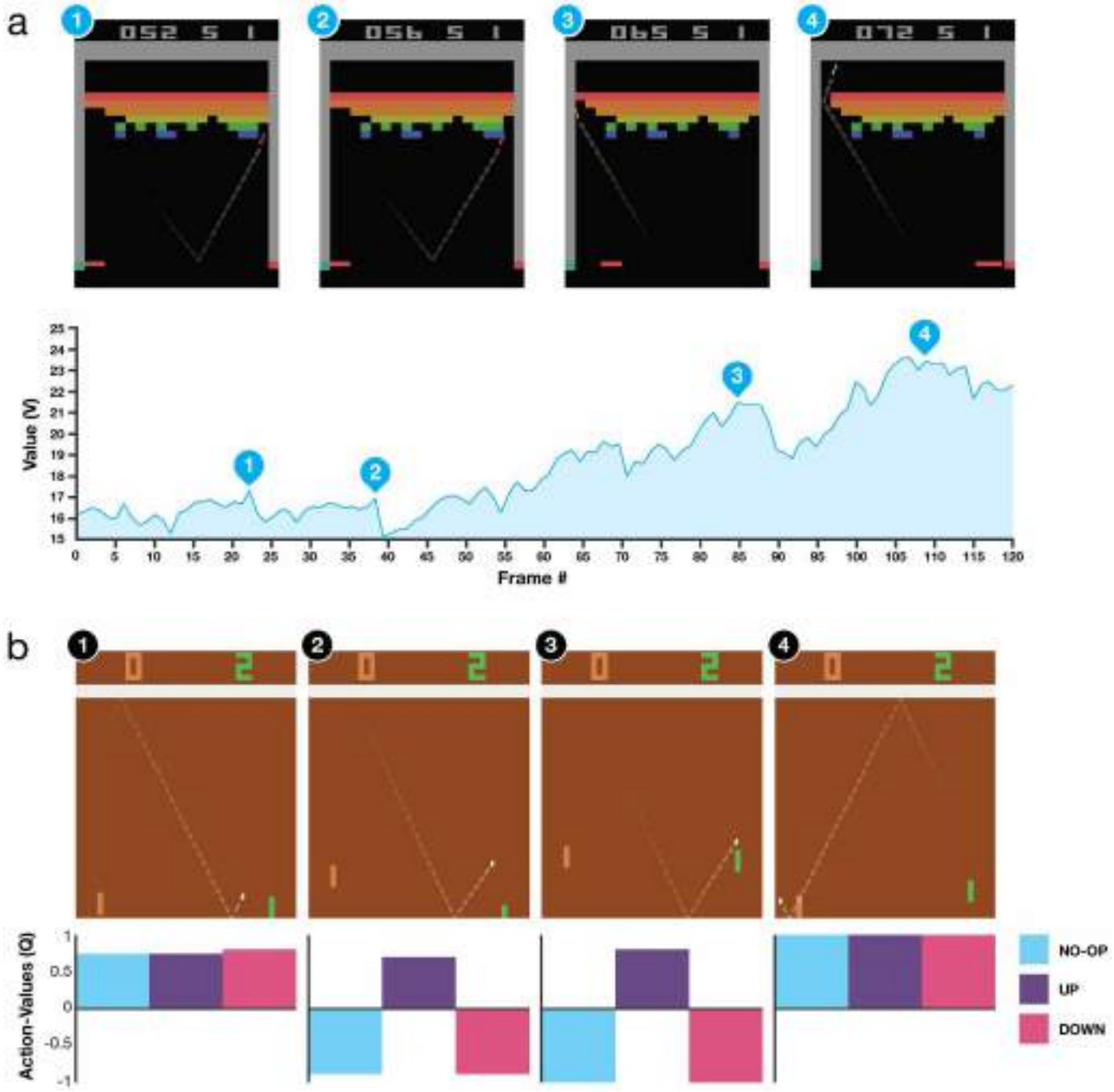
Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$
For episode = 1, M **do**
  Initialize sequence $s_1=\{x_1\}$ and preprocessed sequence $\phi_1=\phi(s_1)$
  For t = 1,T **do**
    With probability ε select a random action a_t
    otherwise select $a_t=\mathrm{argmax}_a Q(\phi(s_t),a;\theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1}=s_t,a_t,x_{t+1}$ and preprocess $\phi_{t+1}=\phi(s_{t+1})$
    Store transition $(\phi_t,a_t,r_t,\phi_{t+1})$ in D
    Sample random minibatch of transitions $(\phi_j,a_j,r_j,\phi_{j+1})$ from D
    Set $y_j=\begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j+\gamma \max_{a'} \hat{Q}(\phi_{j+1},a';\theta^-) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $\left(y_j-Q(\phi_j,a_j;\theta)\right)^2$ with respect to the network parameters θ
    Every C steps reset $\hat{Q}=Q$
  End For
End For

31. Jarrett, K., Kavukcuoglu, K., Ranzato, M. A. & LeCun, Y. What is the best multi-stage architecture for object recognition? *Proc. IEEE. Int. Conf. Comput. Vis.* 2146–2153 (2009).
32. Nair, V. & Hinton, G. E. Rectified linear units improve restricted Boltzmann machines. *Proc. Int. Conf. Mach. Learn.* 807–814 (2010).
33. Kaelbling, L. P., Littman, M. L. & Cassandra, A. R. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* **101,** 99–134 (1994).



Extended Data Figure 1 | Two-dimensional t-SNE embedding of the representations in the last hidden layer assigned by DQN to game states experienced during a combination of human and agent play in Space Invaders. The plot was generated by running the t-SNE algorithm[25] on the last hidden layer representation assigned by DQN to game states experienced during a combination of human (30 min) and agent (2 h) play. The fact that there is similar structure in the two-dimensional embeddings corresponding to the DQN representation of states experienced during human play (orange points) and DQN play (blue points) suggests that the representations learned by DQN do indeed generalize to data generated from policies other than its own. The presence in the t-SNE embedding of overlapping clusters of points corresponding to the network representation of states experienced during human and agent play shows that the DQN agent also follows sequences of states similar to those found in human play. Screenshots corresponding to selected states are shown (human: orange border; DQN: blue border).



Extended Data Figure 2 | Visualization of learned value functions on two games, Breakout and Pong. a, A visualization of the learned value function on the game Breakout. At time points 1 and 2, the state value is predicted to be ~17 and the agent is clearing the bricks at the lowest level. Each of the peaks in the value function curve corresponds to a reward obtained by clearing a brick. At time point 3, the agent is about to break through to the top level of bricks and the value increases to ~21 in anticipation of breaking out and clearing a large set of bricks. At point 4, the value is above 23 and the agent has broken through. After this point, the ball will bounce at the upper part of the bricks clearing many of them by itself. **b**, A visualization of the learned action-value function on the game Pong. At time point 1, the ball is moving towards the paddle controlled by the agent on the right side of the screen and the values of all actions are around 0.7, reflecting the expected value of this state based on previous experience. At time point 2, the agent starts moving the paddle towards the ball and the value of the 'up' action stays high while the value of the 'down' action falls to −0.9. This reflects the fact that pressing 'down' would lead to the agent losing the ball and incurring a reward of −1. At time point 3, the agent hits the ball by pressing 'up' and the expected reward keeps increasing until time point 4, when the ball reaches the left edge of the screen and the value of all actions reflects that the agent is about to receive a reward of 1. Note, the dashed line shows the past trajectory of the ball purely for illustrative purposes (that is, not shown during the game). With permission from Atari Interactive, Inc.

Extended Data Table 1 | List of hyperparameters and their values

Hyperparameter	Value	Description
minibatch size	32	Number of training cases over which each stochastic gradient descent (SGD) update is computed.
replay memory size	1000000	SGD updates are sampled from this number of most recent frames.
agent history length	4	The number of most recent frames experienced by the agent that are given as input to the Q network.
target network update frequency	10000	The frequency (measured in the number of parameter updates) with which the target network is updated (this corresponds to the parameter C from Algorithm 1).
discount factor	0.99	Discount factor gamma used in the Q-learning update.
action repeat	4	Repeat each action selected by the agent this many times. Using a value of 4 results in the agent seeing only every 4th input frame.
update frequency	4	The number of actions selected by the agent between successive SGD updates. Using a value of 4 results in the agent selecting 4 actions between each pair of successive updates.
learning rate	0.00025	The learning rate used by RMSProp.
gradient momentum	0.95	Gradient momentum used by RMSProp.
squared gradient momentum	0.95	Squared gradient (denominator) momentum used by RMSProp.
min squared gradient	0.01	Constant added to the squared gradient in the denominator of the RMSProp update.
initial exploration	1	Initial value of ϵ in ϵ-greedy exploration.
final exploration	0.1	Final value of ϵ in ϵ-greedy exploration.
final exploration frame	1000000	The number of frames over which the initial value of ϵ is linearly annealed to its final value.
replay start size	50000	A uniform random policy is run for this number of frames before learning starts and the resulting experience is used to populate the replay memory.
no-op max	30	Maximum number of "do nothing" actions to be performed by the agent at the start of an episode.

The values of all the hyperparameters were selected by performing an informal search on the games Pong, Breakout, Seaquest, Space Invaders and Beam Rider. We did not perform a systematic grid search owing to the high computational cost, although it is conceivable that even better results could be obtained by systematically tuning the hyperparameter values.

Extended Data Table 2 | Comparison of games scores obtained by DQN agents with methods from the literature[12,15] and a professional human games tester

Game	Random Play	Best Linear Learner	Contingency (SARSA)	Human	DQN (± std)	Normalized DQN (% Human)
Alien	227.8	939.2	103.2	6875	3069 (±1093)	42.7%
Amidar	5.8	103.4	183.6	1676	739.5 (±3024)	43.9%
Assault	222.4	628	537	1496	3359(±775)	246.2%
Asterix	210	987.3	1332	8503	6012 (±1744)	70.0%
Asteroids	719.1	907.3	89	13157	1629 (±542)	7.3%
Atlantis	12850	62687	852.9	29028	85641(±17600)	449.9%
Bank Heist	14.2	190.8	67.4	734.4	429.7 (±650)	57.7%
Battle Zone	2360	15820	16.2	37800	26300 (±7725)	67.6%
Beam Rider	363.9	929.4	1743	5775	6846 (±1619)	119.8%
Bowling	23.1	43.9	36.4	154.8	42.4 (±88)	14.7%
Boxing	0.1	44	9.8	4.3	71.8 (±8.4)	1707.9%
Breakout	1.7	5.2	6.1	31.8	401.2 (±26.9)	1327.2%
Centipede	2091	8803	4647	11963	8309(±5237)	63.0%
Chopper Command	811	1582	16.9	9882	6687 (±2916)	64.8%
Crazy Climber	10781	23411	149.8	35411	114103 (±22797)	419.5%
Demon Attack	152.1	520.5	0	3401	9711 (±2406)	294.2%
Double Dunk	-18.6	-13.1	-16	-15.5	-18.1 (±2.6)	17.1%
Enduro	0	129.1	159.4	309.6	301.8 (±24.6)	97.5%
Fishing Derby	-91.7	-89.5	-85.1	5.5	-0.8 (±19.0)	93.5%
Freeway	0	19.1	19.7	29.6	30.3 (±0.7)	102.4%
Frostbite	65.2	216.9	180.9	4335	328.3 (±250.5)	6.2%
Gopher	257.6	1288	2368	2321	8520 (±3279)	400.4%
Gravitar	173	387.7	429	2672	306.7 (±223.9)	5.3%
H.E.R.O.	1027	6459	7295	25763	19950 (±158)	76.5%
Ice Hockey	-11.2	-9.5	-3.2	0.9	-1.6 (±2.5)	79.3%
James Bond	29	202.8	354.1	406.7	576.7 (±175.5)	145.0%
Kangaroo	52	1622	8.8	3035	6740 (±2959)	224.2%
Krull	1598	3372	3341	2395	3805 (±1033)	277.0%
Kung-Fu Master	258.5	19544	29151	22736	23270 (±5955)	102.4%
Montezuma's Revenge	0	10.7	259	4367	0 (±0)	0.0%
Ms. Pacman	307.3	1692	1227	15693	2311(±525)	13.0%
Name This Game	2292	2500	2247	4076	7257 (±547)	278.3%
Pong	-20.7	-19	-17.4	9.3	18.9 (±1.3)	132.0%
Private Eye	24.9	684.3	86	69571	1788 (±5473)	2.5%
Q*Bert	163.9	613.5	960.3	13455	10596 (±3294)	78.5%
River Raid	1339	1904	2650	13513	8316 (±1049)	57.3%
Road Runner	11.5	67.7	89.1	7845	18257 (±4268)	232.9%
Robotank	2.2	28.7	12.4	11.9	51.6 (±4.7)	509.0%
Seaquest	68.4	664.8	675.5	20182	5286(±1310)	25.9%
Space Invaders	148	250.1	267.9	1652	1976 (±893)	121.5%
Star Gunner	664	1070	9.4	10250	57997 (±3152)	598.1%
Tennis	-23.8	-0.1	0	-8.9	-2.5 (±1.9)	143.2%
Time Pilot	3568	3741	24.9	5925	5947 (±1600)	100.9%
Tutankham	11.4	114.3	98.2	167.6	186.7 (±41.9)	112.2%
Up and Down	533.4	3533	2449	9082	8456 (±3162)	92.7%
Venture	0	66	0.6	1188	380.0 (±238.6)	32.0%
Video Pinball	16257	16871	19761	17298	42684 (±16287)	2539.4%
Wizard of Wor	563.5	1981	36.9	4757	3393 (±2019)	67.5%
Zaxxon	32.5	3365	21.4	9173	4977 (±1235)	54.1%

Best Linear Learner is the best result obtained by a linear function approximator on different types of hand designed features[12]. Contingency (SARSA) agent figures are the results obtained in ref. 15. Note the figures in the last column indicate the performance of DQN relative to the human games tester, expressed as a percentage, that is, 100 × (DQN score − random play score)/(human score − random play score).

Extended Data Table 3 | The effects of replay and separating the target Q-network

Game	With replay, with target Q	With replay, without target Q	Without replay, with target Q	Without replay, without target Q
Breakout	316.8	240.7	10.2	3.2
Enduro	1006.3	831.4	141.9	29.1
River Raid	7446.6	4102.8	2867.7	1453.0
Seaquest	2894.4	822.6	1003.0	275.8
Space Invaders	1088.9	826.3	373.2	302.0

DQN agents were trained for 10 million frames using standard hyperparameters for all possible combinations of turning replay on or off, using or not using a separate target Q-network, and three different learning rates. Each agent was evaluated every 250,000 training frames for 135,000 validation frames and the highest average episode score is reported. Note that these evaluation episodes were not truncated at 5 min leading to higher scores on Enduro than the ones reported in Extended Data Table 2. Note also that the number of training frames was shorter (10 million frames) as compared to the main results presented in Extended Data Table 2 (50 million frames).

Extended Data Table 4 | Comparison of DQN performance with linear function approximator

Game	DQN	Linear
Breakout	316.8	3.00
Enduro	1006.3	62.0
River Raid	7446.6	2346.9
Seaquest	2894.4	656.9
Space Invaders	1088.9	301.3

The performance of the DQN agent is compared with the performance of a linear function approximator on the 5 validation games (that is, where a single linear layer was used instead of the convolutional network, in combination with replay and separate target network). Agents were trained for 10 million frames using standard hyperparameters, and three different learning rates. Each agent was evaluated every 250,000 training frames for 135,000 validation frames and the highest average episode score is reported. Note that these evaluation episodes were not truncated at 5 min leading to higher scores on Enduro than the ones reported in Extended Data Table 2. Note also that the number of training frames was shorter (10 million frames) as compared to the main results presented in Extended Data Table 2 (50 million frames).

August 18, 2017 Release

OpenAI Baselines: ACKTR & A2C

Read code ↗ Read paper ↗

Illustration: Ben Barry

ACKTR can learn continuous control tasks, like moving a robotic arm to a target location, purely from low-resolution pixel inputs (left).

ACKTR (pronounced “actor”)—Actor Critic using Kronecker-factored Trust Region—was developed by researchers at the University of Toronto and New York University, and we at OpenAI have collaborated with them to release a Baselines implementation. The authors use ACKTR to learn control policies for simulated robots (with pixels as input, and continuous action spaces) and Atari agents (with pixels as input and discrete action spaces).

ACKTR combines three distinct techniques: actor-critic methods, trust region optimization for more consistent improvement, and distributed Kronecker factorization to improve sample efficiency and scalability.

ChatGPT
Sora
For Business
API Platform
Sora
API Platform
Stories
Company
News

ACKTR has better sample complexity than first-order methods such as A2C because it takes a step in the *natural gradient* direction, rather than the gradient direction (or a rescaled version as in ADAM). The natural gradient gives us the direction in parameter space that achieves the largest (instantaneous) improvement in the objective per unit of change in the output distribution of the network, as measured using the KL-divergence. By limiting the KL divergence, we ensure that the new policy does not behave radically differently than the old one, which could cause a collapse in performance.

As for computational complexity, the KFAC update used by ACKTR is only 10–25% more expensive per update step than a standard gradient update. This contrasts with methods like TRPO (i.e, Hessian-free optimization), which requires a more expensive conjugate-gradient computation.

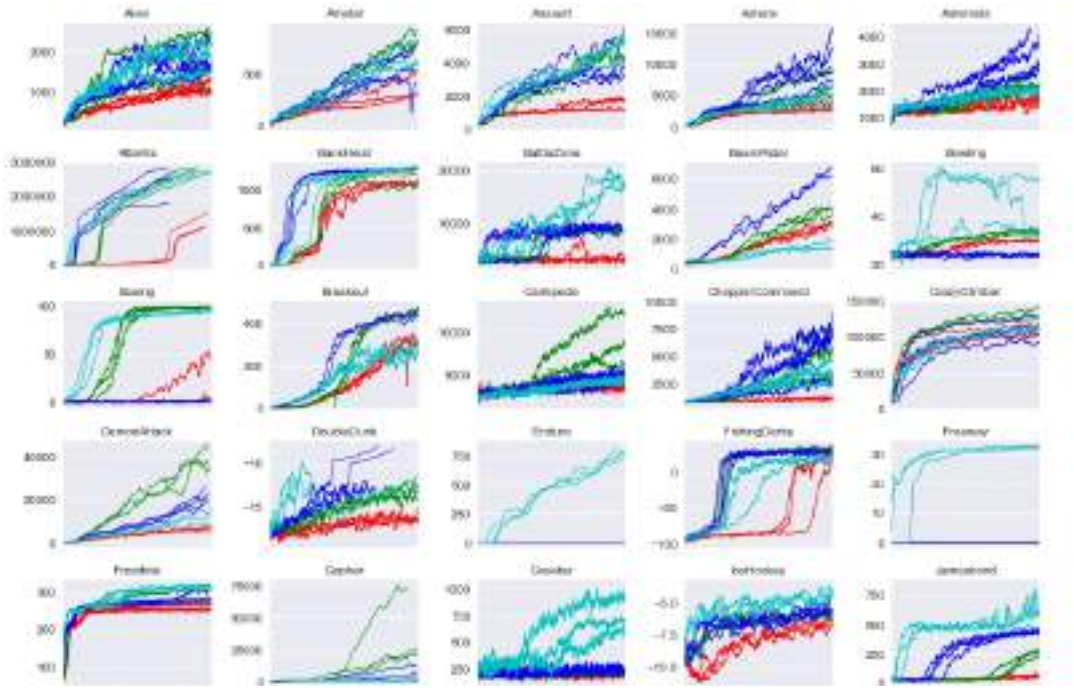
In the following video you can see comparisons at different timesteps between agents trained with ACKTR to solve the game Q-Bert and those trained with A2C. ACKTR agents get higher scores than ones trained with A2C.

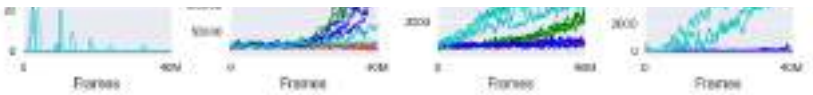
trained with other algorithms, such as A2C (left).

Baseline and benchmarks

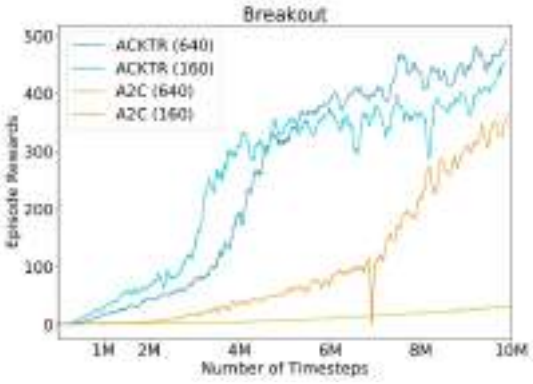
This release includes an OpenAI baseline release of ACKTR, as well as a release of A2C.

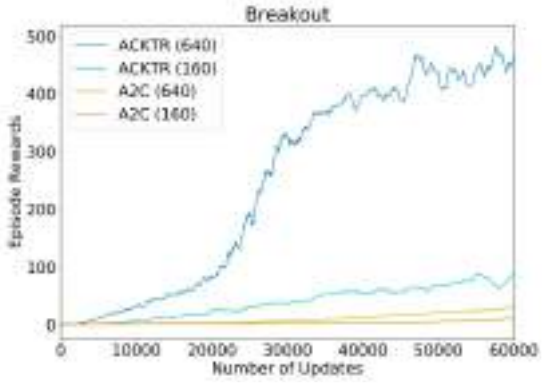
We're also publishing benchmarks that evaluate ACKTR against A2C, PPO and ACER on a range of tasks. In the following plot we show performance of ACKTR on 49 Atari games compared to other algorithm: A2C, PPO, ACER. The hyperparameters of ACKTR were tuned by the author of ACKTR solely on one game, Breakout.





ACKTR performance also scales well with batch size because it not only derives a gradient estimate from the information in each batch, but also uses the information to approximate the local curvature in the parameter space. This feature is particularly favorable for large scale distributed training, in which large batch sizes are used.





A2C and A3C

The Asynchronous Advantage Actor Critic method (A3C) has been very

ChatGPT
Sora
For Business
API Platform
Sora
API Platform
Stories
Company
News

some regularization or exploration?"), or if it was just an implementation detail that allowed for faster training with a CPU-based implementation.

As an alternative to the asynchronous implementation, researchers found you can write a synchronous, deterministic implementation that waits for each actor to finish its segment of experience before performing an update, averaging over all of the actors. One advantage of this method is that it can more effectively use of GPUs, which perform best with large batch sizes. This algorithm is naturally called A2C, short for advantage actor critic. (This term has been used in several papers.)

Our synchronous A2C implementation performs better than our asynchronous implementations—we have not seen any evidence that the noise introduced by asynchrony provides any performance benefit. This A2C implementation is more cost-effective than A3C when using single-GPU machines, and is faster than a CPU-only A3C implementation when using larger policies.

We have included code in Baselines for training feedforward convnets and LSTMs on the Atari benchmark using A2C.

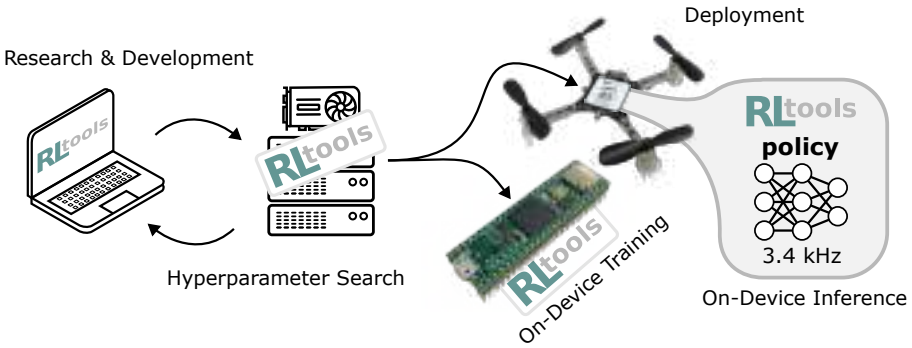
Software & Engineering

Journal of Machine Learning Research 25 (2024) 1-19 Submitted 2/24; Revised 8/24; Published 9/24

RLtools: A Fast, Portable Deep Reinforcement Learning Library for Continuous Control

Jonas Eschmann[1,2] **Dario Albani**[2] **Giuseppe Loianno**[1]
[1]**New York University** [2]**Technology Innovation Institute**
{jonas.eschmann,loiannog}@nyu.edu
dario.albani@tii.ae

Editor: Alexandre Gramfort



Abstract

Deep Reinforcement Learning (RL) can yield capable agents and control policies in several domains but is commonly plagued by prohibitively long training times. Additionally, in the case of continuous control problems, the applicability of learned policies on real-world embedded devices is limited due to the lack of real-time guarantees and portability of existing libraries. To address these challenges, we present **RLtools**, a dependency-free, header-only, pure C++ library for deep supervised and reinforcement learning. Its novel architecture allows **RLtools** to be used on a wide variety of platforms, from HPC clusters over workstations and laptops to smartphones, smartwatches, and microcontrollers. Specifically, due to the tight integration of the RL algorithms with simulation environments, **RLtools** can solve popular RL problems up to 76 times faster than other popular RL frameworks. We also benchmark the inference on a diverse set of microcontrollers and show that in most cases our optimized implementation is by far the fastest. Finally, **RLtools** enables the first-ever demonstration of training a deep RL algorithm directly on a microcontroller, giving rise to the field of Tiny Reinforcement Learning (TinyRL). The source code as well as documentation and live demos are available through our project page at https://rl.tools.

Keywords: Reinforcement Learning, Continuous Control, Deep Learning, TinyRL

1 Introduction

Continuous control is a ubiquitous and pervasive problem in a diverse set of domains such as robotics, high-frequency decision-making in financial markets or the automation of chemical plants and smart grid infrastructure. Taking advantage of the recent progress in Deep Learning (DL) that is spilling over into decision-making in the form of RL, agents derived using deep RL have already attained impressive performance in a range of decision-making problems, like games and particularly continuous control. Despite these achievements, the

real-world adoption of RL for continuous control is hindered by prohibitively long training times as well as a lack of support for the deployment of trained policies on real-world embedded devices. Long training times obstruct rapid iteration in the problem space (reward function design, hyperparameter tuning, etc.) while deployment on computationally severely limited embedded devices is necessary to control the bulk of physical systems such as: robots, automotive components, medical devices, smart grid infrastructure, etc. In non-physical systems, such as financial markets, the need for high-frequency decision-making leads to similar real-time requirements which cannot be fulfilled by current deep RL libraries. Hence, to address these challenges we present **RLtools**, a dependency-free, header-only pure C++ library for deep supervised and reinforcement learning combining the following contributions:

- **Novel Architecture**: We describe the innovations in the software design of the library which allow for unprecedented training and inference speeds on a wide variety of devices from High-Performance Computing (HPC) clusters over workstations and laptops to smartphones, smartwatches and microcontrollers.
- **Implementation**: We contribute a modular, highly portable, and efficient implementation of the aforementioned architecture in the form of open-source code, documentation, and test cases.
- **Fastest Training**: We demonstrate large speedups in terms of wall-clock training time.
- **Fastest Inference**: We demonstrate large speedups in terms of the inference time of trained policies on a diverse set of common microcontrollers.
- **TinyRL**: By utilizing **RLtools**, we successfully demonstrate, the first-ever training of a deep RL algorithm for continuous control directly on a microcontroller.

2 Related Work

Multiple deep RL frameworks and libraries have been proposed, many of which cover algorithmic research, with and without abstractions (Acme (Hoffman et al., 2020), skrl (Serrano-Munoz et al., 2023) and CleanRL (Huang et al., 2022) respectively). Other frameworks and libraries focus on comprehensiveness in terms of the number of algorithms included (RLlib (Liang et al., 2018), ReinforcementLearning.jl (Tian, 2020), MushroomRL (D'Eramo et al., 2021), Stable-Baselines3 (Raffin et al., 2021), ChainerRL (Fujita et al., 2021)), Tianshou (Weng et al., 2022), and TorchRL (Bou et al., 2024). In contrast to these aforementioned solutions, **RLtools** aims at fast iteration in the problem space in the form of e.g., reward function design (Eschmann, 2021) and hyperparameter optimization. In the problem space, the algorithmic intricacies and variety of the algorithms matter less than the robustness, training speed, and final performance as well as our understanding of how to train them reliably. From the formerly mentioned RL frameworks and libraries RLlib (Liang et al., 2018) is the most similar in terms of its mission statement being on quick iteration and deployment (cf. benchmark comparisons wrt. this goal in Section 4). By focusing on iteration in the space of problems and subsequent deployment to real-time platforms, we also draw parallels between **RLtools** and the ACADOS software (Verschueren et al., 2022) for synthesizing Model Predictive Controls (MPCs) with **RLtools** aspiring to be its RL equivalent.

3 Approach

Taking the last handful of years of progress in RL for continuous control, it can be observed that the most prominent models used as function approximators are still relatively small, fully-connected neural networks. In Appendix A we analyze the architectures used in deep RL for continuous control and justify the focus of RLtools on (small) fully-connected neural networks. Based on these observations, we conclude that the great flexibility provided by automatic differentiation frameworks like TensorFlow or PyTorch might not be necessary for applying RL to many continuous control problems. We believe that there is an advantage in trading-off the flexibility in the model architecture of the function approximators for the overall training speed. Reducing the training time and increasing the training efficiency saves energy, simplifies reproducibility and democratizes access to state of the art RL methods. Furthermore, fast training facilitates principled hyperparameter search which in turn improves comparability.

Architecture Our software architecture is guided by the previous observation and hence by maximizing the training time efficiency without sacrificing returns. Additionally, we want the software to be able to run across many different accelerators and devices (CPUs, GPUs, microcontrollers, and other accelerators) so that trained policies can also directly be deployed on microcontrollers and take advantage of device-specific instructions to run at high frequencies with hard realtime guarantees. This also entails that RLtools does not rely on any dependencies because they might not be available on the target microcontrollers.

To attain maximum performance, we integrate the different components of our library as tightly as needed while maintaining as much flexibility and modularity as possible. To enable this goal, we heavily rely on the C++ templating system. Leveraging template meta-programming, we can provide the compiler with a maximum amount of information about the structure of the code, enabling it to be optimized heavily. We particularly make sure that the size of all loops is known at compile time such that the compiler can optimize them via inlining and loop-unrolling (cf. Appendix B and F). Leveraging pure C++ without any dependencies, we implement the following major components: **Deep Learning** (MLP, backpropagation, Adam, etc.), **Reinforcement Learning** (GAE, PPO, TD3, SAC), and **Simulation** (Pendulum, Acrobot, Quadrotor, Racing Car, MuJoCo interface). We implement RLtools in a modular way by using a novel static multiple-dispatch paradigm inspired by (dynamic) multiple-dispatch which was popularized by the Julia programming language Bezanson et al. (2012). We highly recommend taking a look at the code example and explanation in Appendix B as well as the ablation study in Appendix F measuring the impact of different components and optimizations.

4 Results

Horizontal Benchmark Figure 1 and 2 show the resulting mean training times from running the PPO and SAC algorithm across ten runs on an Intel-based laptop (details in Table 6). We find that RLtools outperforms existing libraries by a wide margin. Particularly in the case of PPO where RLtools only takes 0.54 s on average (2.59 s in case of SAC).

Vertical Benchmark In Figure 3, we also present training results using RLtools on a wide variety of devices which are generally not compatible with the other RL libraries and

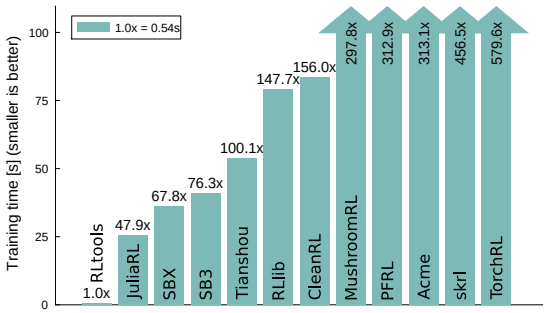


Figure 1: PPO: `Pendulum-v1` (300000 steps)

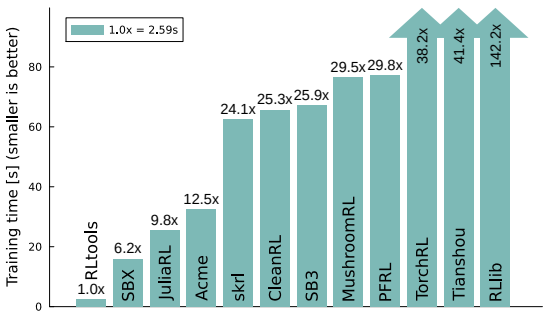


Figure 2: SAC: `Pendulum-v1` (10000 steps)

Platform	RLtools: Generic	DSP Library	RLtools: Optimized
Crazyflie	743 us (1.3 kHz)	478 us (2.1 kHz)	**293 us (3.4 kHz)**
Pixhawk 6C	133 us (7.5 kHz)	93 us (10.8 kHz)	**53 us (18.8 kHz)**
Teensy 4.1	64 us (15.5 kHz)	45 us (22.3 kHz)	**41 us (24.3 kHz)**
ESP32 (Xtensa)	4282 us (234 Hz)	**279 us (3.6 kHz)**	333 us (3 kHz)
ESP32-C3 (RISC-V)	8716 us (115 Hz)	6950 us (144 Hz)	**6645 us (150 Hz)**

Table 1: Inference times on different platforms

frameworks. Importantly, we also demonstrate the first training of a deep RL agent for continuous control on a microcontroller in form of the Teensy 4.1.

Inference on Microcontrollers Table 1 shows the inference times on microcontrollers of different compute capabilities (e.g. Crazyflie is a 27 g quadrotor with very limited resources, cf. Appendix E). The generic implementation already yields usable inference times but dispatching to the manufacturers Digital Signal Processor (DSP) library improves the performance. Finally, by optimizing the code further (e.g. through fusing the activation operators) we achieve a significant speedup even compared to the manufacturers DSP libraries.

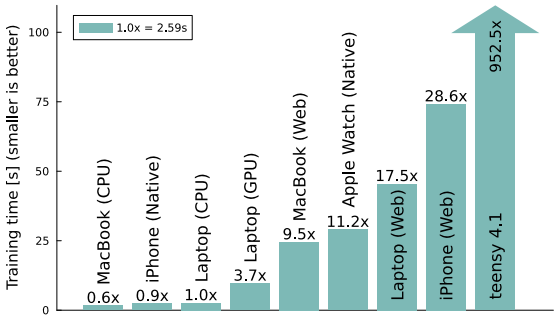


Figure 3: SAC: `Pendulum-v1` (10000 steps)

5 Conclusion

We believe RLtools fills a gap by allowing fast iteration in the problem space and subsequent real-time deployment of policies. Furthermore, RLtools facilitates the first-ever deep RL training on a microcontroller. We acknowledge the steeper learning curve of C++ (over e.g. Python) but from our experience, the faster iteration made possible by shorter training times can outweigh the added time to get started. Currently RLtools is limited to dense observations but we plan to add vision capabilities in the future. We believe that by relaxing the compute requirements and, by being fully open-source, RLtools democratizes the training of state-of-the-art RL methods and accelerates progress in RL for continuous control.

Acknowledgments and Disclosure of Funding

This work was supported by the Technology Innovation Institute, the NSF CAREER Award 2145277, and the DARPA YFA Grant D22AP00156-00. Giuseppe Loianno serves as a consultant for the Technology Innovation Institute. This arrangement has been reviewed and approved by New York University in accordance with its policy on objectivity in research.

Appendix A. Analysis of the Deep RL Landscape

Year	Name	Hidden Dim	#Params	Non-Linearity
2015	TRPO Schulman et al. (2015)	[50, 50]	1.0x	tanh
2015	GAE Schulman et al. (2016)	[100, 50, 25]	2.3x	tanh
2016	DDPG Lillicrap et al. (2016)	[400, 300]	35.3x	ReLU
2017	PPO Schulman et al. (2017)	[64, 64]	1.5x	tanh
2018	TD3 Fujimoto et al. (2018)	[400, 300]	35.3x	ReLU
2018	SAC Haarnoja et al. (2018)	[256, 256]	19.6x	ReLU
2019	SACv2 Haarnoja et al. (2019)	[256, 256]	19.6x	ReLU
2020	TQC Kuznetsov et al. (2020)	[512, 512, 512]	147.0x	ReLU
2020	D4PG&TD3Bach et al. (2020)	[256, 256]	19.6x	ReLU
2021	PPO&RMA Kumar et al. (2021)	[128, 128, 128]	9.8x	ReLU

Table 2: Selection of works that introduced impactful algorithms and the respective neural network dimensions used for their value function approximations. For the calculation of the number of parameters, an input size of 20 and an output size of 1 is assumed

In this section, we analyze the function approximator models used in the major deep RL for continuous control publications collected in Table 7. The most important observation is that over all the years the architecture (small, fully-connected neural networks) has not changed. This can be attributed to the fact that in continuous control the observations are usually dense states of the systems which do not contain any spatial or temporal regularities like images or time series that would suggest the usage of less general, more tailored network structures like Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs). This regularity, as stated in section 3, motivates our focus on optimizing and tightly integrating fully-connected neural networks as a first step. We also plan integrate recurrent and possibly convolutional layers in the future.

Appendix B. Programming Paradigm

To enable maximum performance, we are avoiding C++ Virtual Method Table (VMT) lookups by not using an object-oriented paradigm but a rather functional paradigm heavily based on templating and method overloading resembling a static, compile-time defined

```
// file: implementation_generic.h
template <typename DEVICE, auto M, auto N, auto K>
void multiply(DEVICE device, Matrix<M, K> a, Matrix<K, N> b, Matrix<M, N> result){
    // Generic code for matrix multiplication
    ...
}
// file: implementation_microcontroller.h
template <auto M, auto N, auto K>
void multiply(MICROCONTROLLER device, Matrix<M, K> a, Matrix<K, N> b, Matrix<M, N>
    result){
    // Optimized code for matrix multiplication on a particular microcontroller (e.g
    . based on DSP extensions)
    ...
}
// file: implementation_gpu.h
template <auto M, auto N, auto K>
void multiply(GPU device, Matrix<M, K> a, Matrix<K, N> b, Matrix<M, N> result){
    // Optimized GPU code for matrix multiplication (e.g. CUDA kernel launch)
    ...
}
template<typename DEVICE, typename OBJECT_A, typename OBJECT_B, typename OBJECT_C>
void algorithm(DEVICE device, OBJECT_A a, OBJECT_B b, OBJECT_C c){
    ...
    multiply(device, a, b, c);
    ...
}

// usage
GPU device;
Matrix<10, 10> a, b, result;
... // malloc and initialize randomly on the GPU device using tag dispatch as well
algorithm(device, a, b, result);
```

Figure 4: Toy example for tag dispatch towards different implementations of elementary matrix operations

interpretation of the multiple dispatch paradigm. Multiple dispatch has been popularized by the Julia programming language Bezanson et al. (2012) and is based on advanced function overloading.

Leveraging multiple dispatch, higher-level functions like the forward or backward pass of a fully-connected neural network just specify the actions that should be taken on the different sub-components/layers and the actual implementation used is dependent on the type of the arguments. In this way, it is simple to share code between GPU and CPU implementations by just implementing the lower-level primitives for the respective device types and then signaling the implementations through the argument type (i.e. using the tag dispatch technique). A toy example for this is displayed in Figure 4. In this case, some `algorithm` is using a matrix multiplication operation on two objects. During the implementation of the `algorithm`, we do

not need to care about the type of the operands and just let them be specified by wildcard template parameters. When this function is called by the user, the compiler infers the template parameters and dispatches the call to the appropriate implementation. If the user does not have a GPU available he simply does not include the `implementation_gpu.h` and hence has no dependency on further dependencies that the GPU implementation would entail (e.g., the CUDA toolkit). In the case where there is no specialized implementation for a particular hardware, the compiler will fall back to the generic implementation which in this example could simply consist of a nested loop. The generic implementations are pure C++ and are guaranteed to have no dependencies. We can also see that the compiler will check the dimensions of the operands automatically at compile time such that the `algorithm` can not be called with incompatible shapes. To create more complex dispatch behaviors and operand type checking C++ features like `static_assert` and `enable_if` can be leveraged through the Substitution Failure Is Not An Error (SFINAE) mechanism. In this way, we can maintain composability while still providing all the structure to the compiler at compile-time.

In the case of Julia, this leads to unparalleled composability which manifests in a small number of people being able to implement and maintain e.g. a deep learning library (Flux Innes (2018)) that is competitive with PyTorch and TensorFlow which are backed by much more resources. In contrast to Julia, which reaches almost native performance while performing the multiple dispatch resolution at runtime, we make sure that all the function calls can be resolved at compile time. Additionally, Julia is not suited for our purposes because it does not fit to run on microcontrollers due to its runtime size and stochastic, non-realtime behavior due to the garbage collection-based memory management. Nevertheless, in our benchmark presented later in this manuscript, we found that Julia is one of the closest competitors when it comes to training performance. Furthermore, we find it important to emphasize that we focus on building a library not a framework.[1] The main feature of frameworks is that they restrict the freedoms of the user to make a small set of tasks easier to accomplish. In certain, repetitive problem settings this might be justified, but in many cases, the overhead coming with the steep learning curves and finding workarounds after bumping into the tight restrictions of frameworks is not worth it. The major conceptual difference is that frameworks provide a context from which they invoke the user's code while in the case of libraries, the user is entirely in control and invokes the components he needs. If not specifically made interoperable, the contexts provided by frameworks are usually incompatible while with libraries this is not generally the case.

In our implementation, this for example concretely manifests in the way function approximators are used in the RL algorithms. By using templating, any function approximator type can be specified by the user at compile time. As long as he also provides the required `forward` and `backward` functions.

As demonstrated in Figure 4 we establish the convention of making a device-dependent context available in each function via tag dispatch to simplify the usage of different compute devices like accelerators or microcontrollers.

1. **Write Libraries, Not Frameworks** [link]

Appendix C. Benchmark Details

Parameter	Value
Actor structure	$[64, 64]$
Critic structure	$[64, 64]$
Activation function	Rectified Linear Unit (ReLU)
Batch size	256
Number of environments	4
Steps per environment	1024
Number of epochs	2
Total number of (environment) steps	300000
Discount factor γ	0.9
Generalized Advantage Estimation (GAE) λ	0.95
ϵ clip	0.2
Entropy coefficient β	0
Advantage normalization	`true`
Adam α	1×10^{-3}
Adam β_1	0.9
Adam β_2	0.999
Adam ϵ	1×10^{-7}

Table 3: `Pendulum-v1` PPO parameters (Figure 1)

Parameter	Value
Actor structure	$[64, 64]$
Critic structure	$[64, 64]$
Activation function	ReLU
Batch size	100
Total number of (environment) steps	10000
Replay buffer size	10000
Discount factor γ	0.99
Entropy bonus coefficient (learned, initial value) α	0.5
Polyak β	0.99
Adam α	1×10^{-3}
Adam β_1	0.9
Adam β_2	0.999
Adam ϵ	1×10^{-7}

Table 4: `Pendulum-v1` SAC parameters (Figure 2)

Parameter	Value
Input dimensionality	13
Policy structure	$[64, 64]$
Output dimensionality	4
Activation function	ReLU

Table 5: On-device inference parameters (Table 1)

Label	Details
RLtools / Laptop (CPU) / Laptop (Web) / Baseline	Intel i9-10885H
Laptop (GPU)	Intel i9-10885H + Nvidia T2000
MacBook (CPU) / MacBook (Web)	MacBook Pro (M3 Pro)
iPhone (Native) / iPhone (Web)	iPhone 14
Apple Watch (Native)	Apple Watch Series 4

Table 6: `Pendulum-v1` SAC devices (Figure 3)

Appendix D. Deep Reinforcement Learning Frameworks and Libraries

Name ↓	Platform	Stars / Citations
Acme (Hoffman et al., 2020)	JAX	3316 / 219
CleanRL (Huang et al., 2022)	PyTorch	4030 / 86
MushroomRL (D'Eramo et al., 2021)	TF/PyTorch	749 / 61
PFRL (Fujita et al., 2021)	PyTorch	1125 / 122
ReinforcementLearning.jl (JuliaRL) (Tian, 2020)	Flux.jl (Julia)	543 / n/a
RLlib + `ray` (Liang et al., 2018)	PyTorch	29798 / 828
Stable Baselines3 (SB3) (Raffin et al., 2021)	PyTorch	7396 / 1149
Stable Baselines JAX (SBX) (Raffin et al., 2021)	JAX	223 / n/a
Tianshou (Weng et al., 2022)	PyTorch	7139 / 133
TorchRL (Weng et al., 2022)	PyTorch	1691 / 5

Table 7: Overview over different RL libraries/frameworks, the deep learning platform they build upon, and their popularity in terms of Github stars and publication citations (data as of 2024-02-07)

Appendix E. Embedded Platforms

1. **Crazyflie**: A small, open-source quadrotor which only weighs 27 g including the battery. The Crazyflie's main processor is a STM32F405 microcontroller using the ARM Cortex-M4 architecture, featuring 192 KB of Random Access Memory (RAM) and running at 168 MHz.

2. **Pixhawk 6C**: We use a Pixracer Pro, a Flight Controller Unit (FCU) that belongs to the family of Pixhawk FCUs and implements the Pixhawk 6C standard. Hence, the PixRacer Pro supports the common PX4 firmware Meier et al. (2015) and can be used in many different vehicle types (aerial, ground, marine) but is predominantly used in multirotor vehicles of varying sizes. The main processor used in the Pixhawk 6C standard is a STM32H743 using the ARM Cortex-M7 architecture. The PixRacer Pro runs at 460 MHz and comes with 1024 KB of RAM.

3. **Teensy 4.1**: A general-purpose embedded device powered by an i.MX RT1060 ARM Cortex-M7 microcontroller with 1024 KB on-chip and 16 MB off-chip RAM that is running at 600 MHz.

4. **ESP32**: One of the most common microcontrollers for Internet of Things (IoT) and edge devices due to its built-in Wi-Fi and Bluetooth. Close to 1 billion devices built around this chip and its predecessor have been sold worldwide. Hence it is widely available and relatively cheap (around $5 for a development kit). For our purposes,

the ESP32 is interesting because it deviates from the previous platforms in that its processor is based on the Xtensa LX7 architecture. In addition to the original version of the ESP32 based on the Xtensa architecture, we also evaluate the ESP32-C3 version based on the RISC-V architecture.

Appendix F. Ablation Study

We conduct an ablation study to investigate the contribution of different components and optimizations to the fast wall-clock training time achieved by RLtools. Figure 5 shows the resulting training times after removing different components and optimizations from the setup. The "Baseline" is exactly the same setup used in the `Pendulum-v1` (SAC) training in the other experiments in Section 4. We simulate the slowness of the Python environment by slowing down the C++ implementation by the average time required for a step in the Python implementation. We can observe that the C++ implementation of the `Pendulum-v1` dynamics has a measurable, but not dominating impact on the training time. Additionally, we ablate the different optimization levels `-O0`, `-O1`, `-O2` and `-O3` (used in the Baseline) of the C++ compiler. We can observe that the compiler optimizations have a sizable impact on the training time. When removing all optimizations (`-O0`) RLtools is roughly between ACME and CleanRL (cf. Figure 2). Furthermore, the "No Fast Math" configuration tests removing the `-ffast-math` from the compiler options, and the "BLAS" Basic Linear Algebra Subprograms (BLAS) option removes the Intel oneMKL matrix multiplication kernels. In the case of "AVX/AVX2" we disable the Advanced Vector Extensions (AVX) that are used for Single Instruction, Multiple Data (SIMD) operations. We notice that due to the design of RLtools (refer to Appendix B) which allows the sizes of all loops and data structures to be known at compile-time the compiler is able to better reason about the code and hence make heavy use of vectorized/SIMD operations. We observe that $2276 + 1430 = 3706$ (AVX + Streaming SIMD Extensions (SSE), an older set of vectorized instructions) out of 11243 machine-code instructions in total refer to registers of the vector extensions. Unfortunately (for the sake of measurement), when turning off AVX, the compiler replaces the instructions with SSE instructions (5406 out of 11243 in this case) which we could not turn off because of some dependency in `libstdc++`. Still, the number of SSE instructions demonstrates the compiler-friendliness that RLtools' architecture entails.

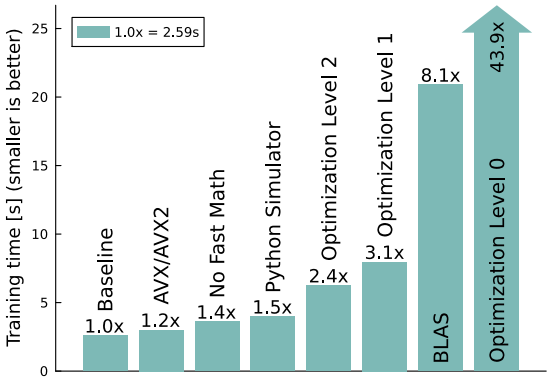


Figure 5: Ablation study. The "Baseline" contains all optimizations.

Appendix G. Convergence Study

To make sure the implementations of the supported RL algorithms (PPO, TD3, and SAC) are correct, we conduct a convergence study where we compare the learning curves across different environments with learning curves of other implementations. We make sure that per environment the same hyperparameters are used across all implementations and run each setup for multiple seeds (100 for `Pendulum-v1` and 30 for `Hopper-v1`). For each of the seeds at every evaluation step, we perform 100 episodes with random initial states.

By comparing different sets of 100 seeds each, we found that, even for a large number of seeds, outliers have a significant impact on the mean final return. Hence, as also recommended by Agarwal et al. (2021), we report the Inter Quantile Mean (IQM) which discards the lowest and highest quantile to remove the impact of outliers on the statistics. We still aim at capturing as much of the final return distribution by only discarding the lower upper 5 % for the calculation of the IQM μ. We use the same inter-quantile set for the calculation of the standard deviation σ. To make sure that the environments are identical in this convergence study, instead of re-implementing the environments in C++ using the **RLtools** interface, we built a Python wrapper for **RLtools** such that we can use the original environments from the Gymnasium (Towers et al.) suite. The Python wrapper makes **RLtools** easier to use but sacrifices in terms of performance if the environment/simulator is implemented in Python (as shown in Appendix F).

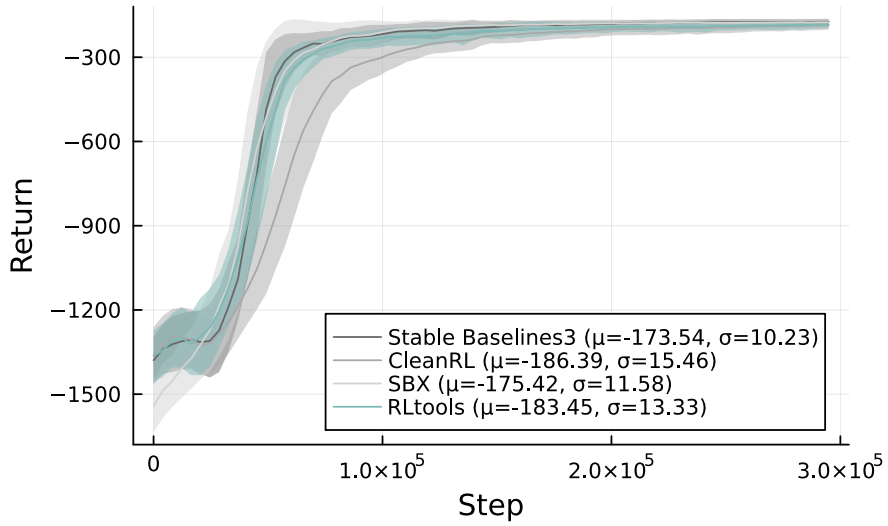


Figure 6: PPO `Pendulum-v1`

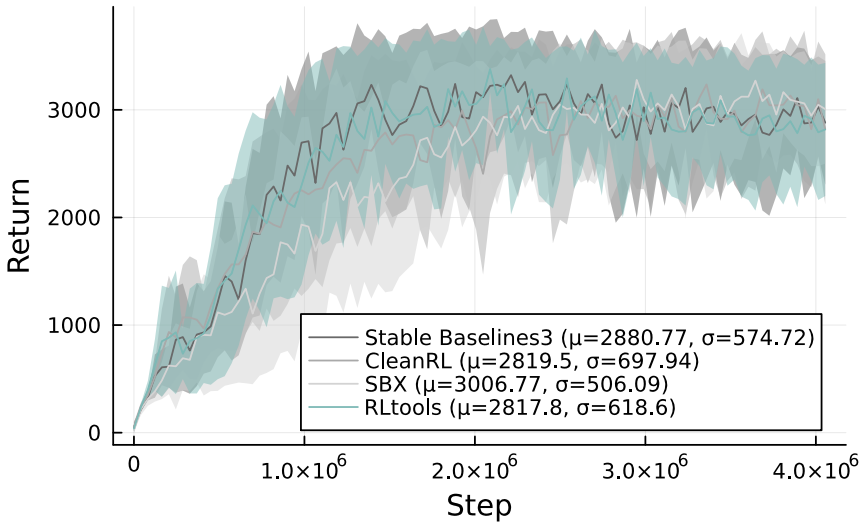


Figure 7: PPO `Hopper-v4`

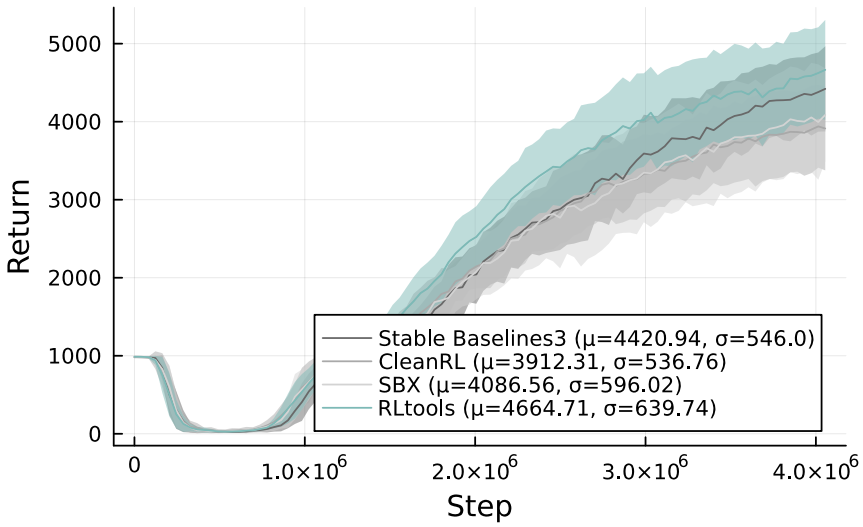


Figure 8: PPO `Ant-v4`

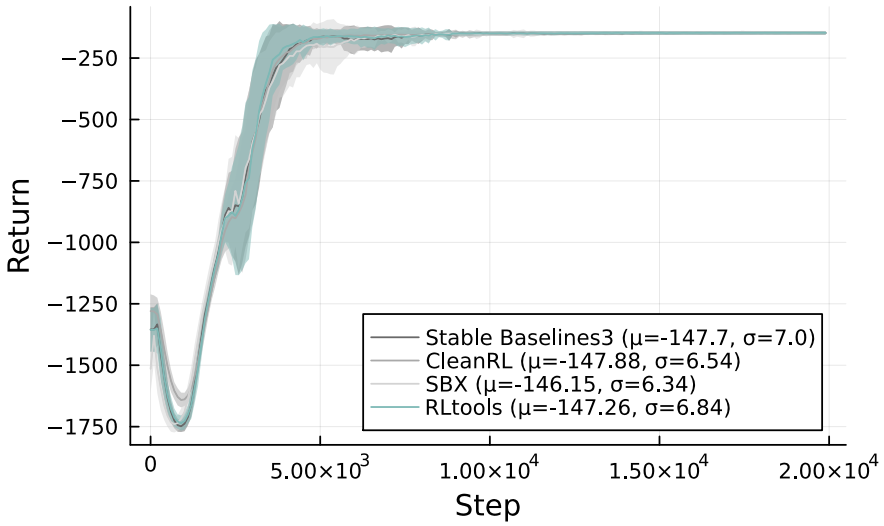


Figure 9: SAC `Pendulum-v1`

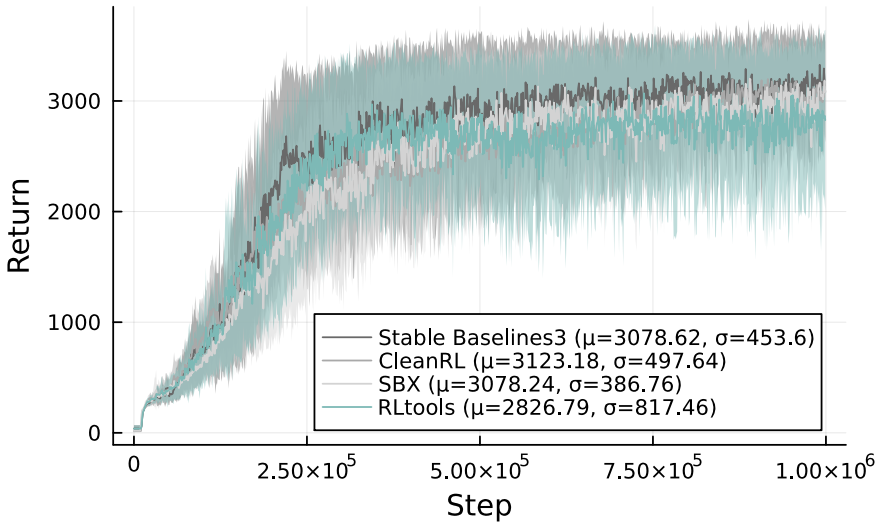


Figure 10: SAC `Hopper-v4`

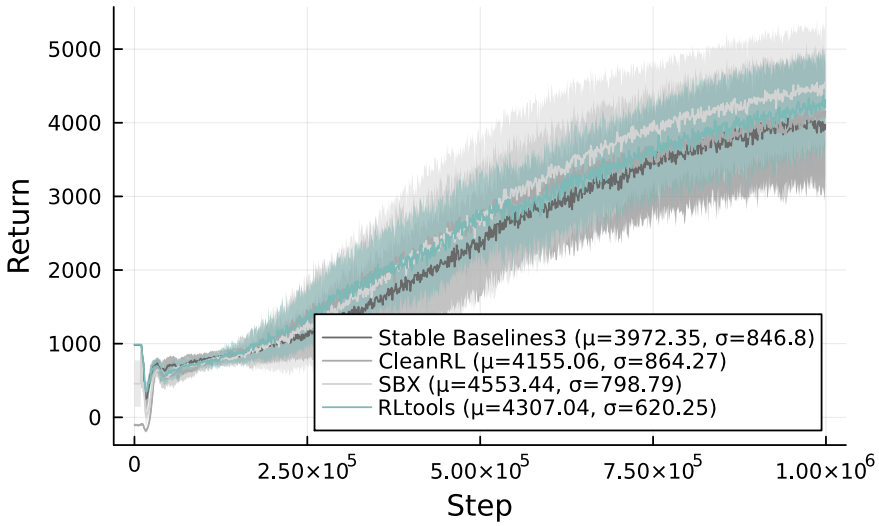


Figure 11: SAC `Ant-v4`

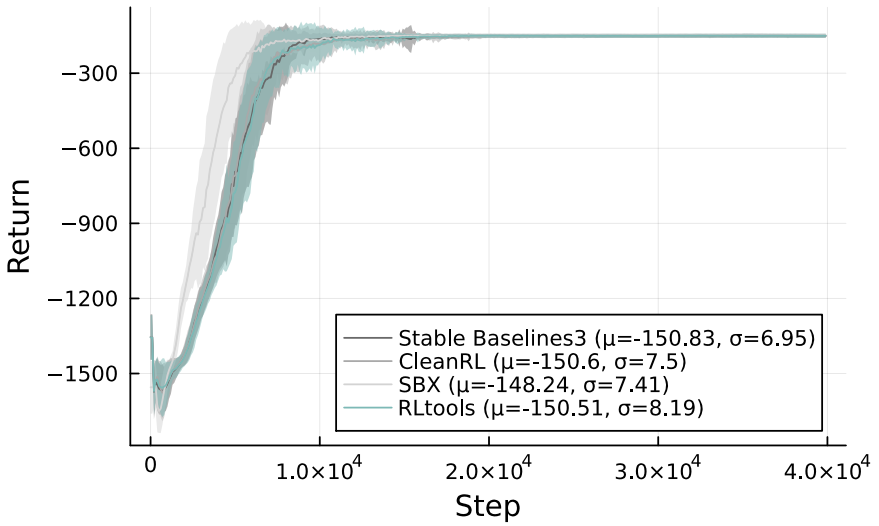


Figure 12: TD3 `Pendulum-v1`

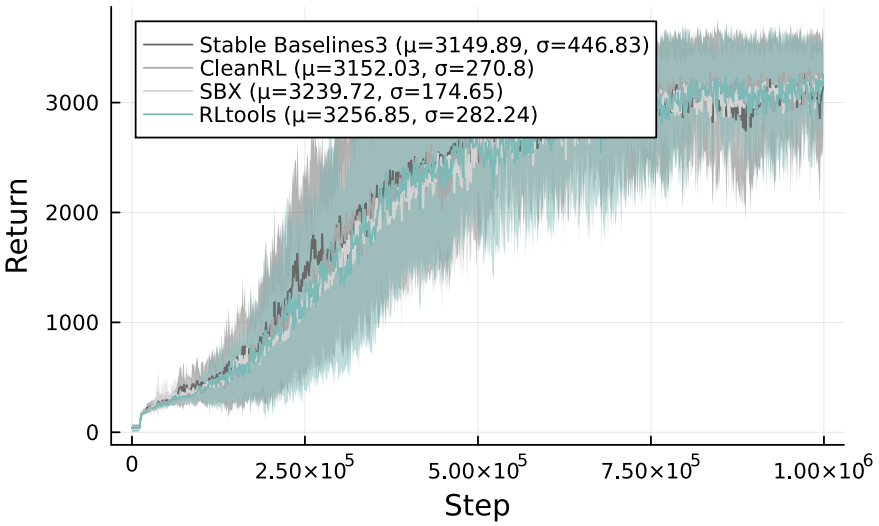


Figure 13: TD3 `Hopper-v4`

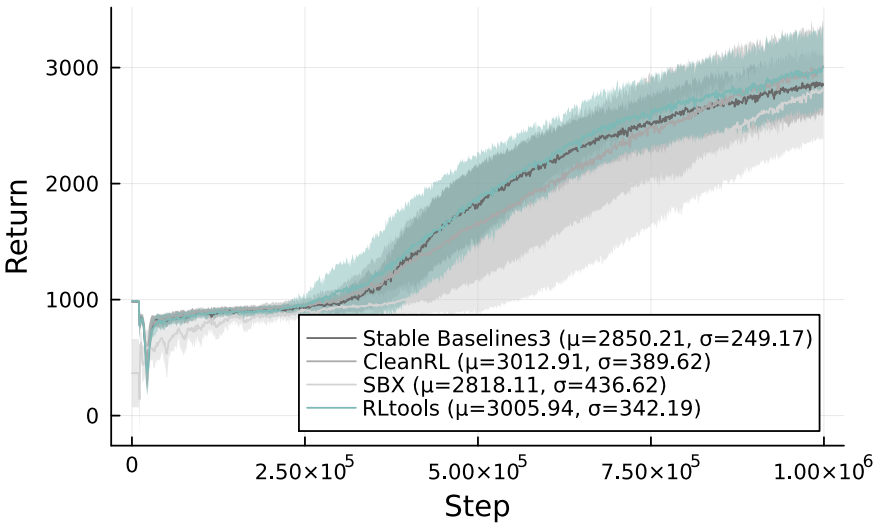


Figure 14: TD3 `Ant-v4`

Prioritized Level Replay

Minqi Jiang [1 2] **Edward Grefenstette** [1 2] **Tim Rocktäschel** [1 2]

Abstract

Environments with procedurally generated content serve as important benchmarks for testing systematic generalization in deep reinforcement learning. In this setting, each *level* is an algorithmically created environment instance with a unique configuration of its factors of variation. Training on a prespecified subset of levels allows for testing generalization to unseen levels. What can be learned from a level depends on the current policy, yet prior work defaults to uniform sampling of training levels independently of the policy. We introduce *Prioritized Level Replay* (PLR), a general framework for selectively sampling the next training level by prioritizing those with higher estimated learning potential when revisited in the future. We show TD-errors effectively estimate a level's future learning potential and, when used to guide the sampling procedure, induce an emergent curriculum of increasingly difficult levels. By adapting the sampling of training levels, PLR significantly improves sample efficiency and generalization on Procgen Benchmark—matching the previous state-of-the-art in test return—and readily combines with other methods. Combined with the previous leading method, PLR raises the state-of-the-art to over 76% improvement in test return relative to standard RL baselines.

1. Introduction

Deep reinforcement learning (RL) easily overfits to training experiences, making generalization a key open challenge to widespread deployment of these methods. Procedural content generation (PCG) environments have emerged as a promising class of problems with which to probe and address this core weakness (Risi & Togelius, 2020; Chevalier-Boisvert et al., 2018; Cobbe et al., 2020a; Juliani et al., 2019; Zhong et al., 2020; Küttler et al., 2020). Unlike singleton environments, like the Arcade Learning Environment games (Bellemare et al., 2013), PCG environments take on algorithmically created configurations at the start of each training episode, potentially varying the layout, asset appearances, or even game dynamics. Each environment instance generated this way is called a *level*. In mapping an identifier, such as a random seed, to each level, PCG environments allow us to measure a policy's generalization to held-out test levels. In this paper, we assume only a blackbox generation process that returns a level given an identifier. We avoid the strong assumption of control over the generation procedure itself, explored by several prior works (see Section 6). Further, we assume levels follow common latent dynamics, so that in aggregate, experiences collected in individual levels reveal general rules governing the entire set of levels.

Despite its humble origin in games, the PCG abstraction proves far-reaching: Many control problems, such as teaching a robotic arm to stack blocks in a specific formation, easily conform to PCG. Here, each level may consist of a unique combination of initial block positions, arm state, and target formation. In a vastly different domain, Hanabi (Bard et al., 2020) also conforms to PCG, where levels map to initial deck orderings. These examples illustrate the generality of the PCG abstraction: Most challenging RL problems entail generalizing across instances (or levels) differing along some underlying factors of variation and thereby can be aptly framed as PCG. This highlights the importance of effective deep RL methods for PCG environments.

Many techniques have been proposed to improve generalization in the PCG setting (see Section 6), requiring changes to the model architecture, learning algorithm, observation space, or environment structure. Notably, these approaches default to uniform sampling of training levels. We instead hypothesize that the variation across levels implies that at each point of training, each level likely holds different potential for an agent to learn about the structure shared across levels to improve generalization. Inspired by this insight and selective sampling methods from active learning, we investigate whether sampling the next training level weighed by its learning potential can improve generalization.

We introduce Prioritized Level Replay (PLR), illustrated in Figure 1, a method for sampling training levels that exploits

[1]Facebook AI Research, London, United Kingdom [2]University College London, London, United Kingdom. Correspondence to: Minqi Jiang <msj@fb.com>.

Proceedings of the 38^{th} International Conference on Machine Learning, PMLR 139, 2021.

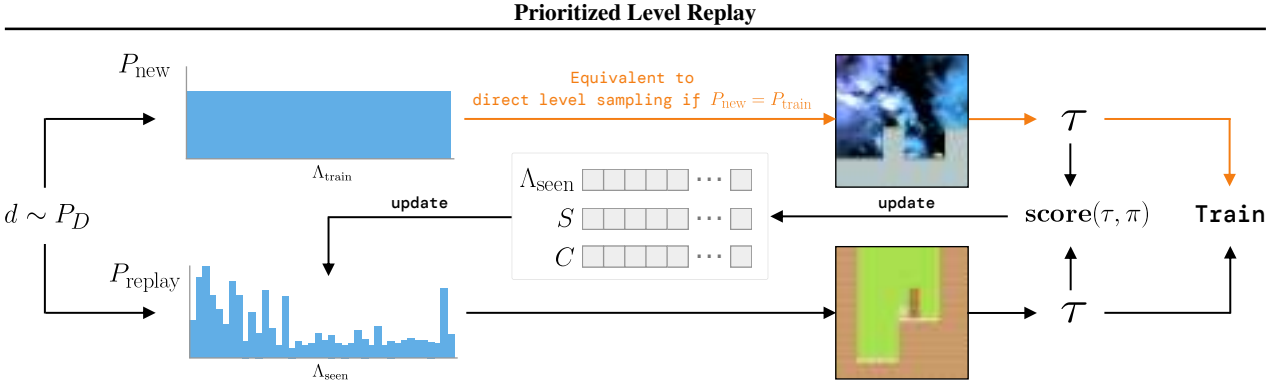


Figure 1. Overview of Prioritized Level Replay. The next level is either sampled from a distribution with support over unseen levels (top), which could be the environment's (perhaps implicit) full training-level distribution, or alternatively, sampled from the replay distribution, which prioritizes levels based on future learning potential (bottom). In either case, a trajectory τ is sampled from the next level and used to update the replay distribution. This update depends on the lists of previously seen levels Λ_{seen}, their latest estimated learning potentials S, and last sampled timestamps C.

the differences in learning potential among levels to improve both sample efficiency and generalization. PLR selectively samples the next training level based on an estimated learning potential of replaying each level anew. During training, our method updates scores estimating each level's learning potential as a function of the agent's policy and temporal-difference (TD) errors collected along the last trajectory sampled on that level. Our method then samples the next training level from a distribution derived from a normalization procedure over these level scores. PLR makes no assumptions about how the policy is updated, so it can be used in tandem with any RL algorithm and combined with many other methods such as data augmentation. Our method also does not assume any external, predefined ordering of levels by difficulty or other criteria, but instead derives level scores dynamically during training based on how the policy interacts with the environment. The only requirements are as follows—satisfied by almost any problem that can be framed as PCG, including RL environments implemented as seeded simulators: (i) Some notion of "level" exists, such that levels follow common latent dynamics; (ii) such levels can be sampled from the environment in an identifiable way; and (iii) given a level identifier, the environment can be set to that level to collect new experiences from it.

While previous works in off-policy RL devised effective methods to directly reuse *past* experiences for training (Schaul et al., 2016; Andrychowicz et al., 2017), PLR uses past experiences to inform the collection of *future* experiences by estimating how much replaying each level anew will benefit learning. Our method can thus be seen as a forward-view variation of prioritized experience replay, and an online counterpart to this off-policy method.

This paper makes the following contributions[1]: (i) We introduce a computationally-efficient algorithm for sampling levels during training based on an estimate of the future learning potential of collecting new experiences from each level; (ii) we show our method significantly improves generalization on 10 of 16 environments in Procgen Benchmark and two challenging MiniGrid environments; (iii) we demonstrate our method combines with the previous leading method to set a new state-of-the-art on Procgen Benchmark; and (iv) we show our method induces an implicit curriculum over training levels in sparse reward settings.

2. Background

In this paper, we refer to a *PCG environment* as any computational process that, given a level identifier (e.g. an index or a random seed), generates a *level*, defined as an environment instance exhibiting a unique configuration of its underlying factors of variation, such as layout, asset appearances, or specific environment dynamics (Risi & Togelius, 2020). For example, MiniGrid's MultiRoom environment instantiates mazes with varying numbers of rooms based on the seed (Chevalier-Boisvert et al., 2018). We refer to sampling a new trajectory generated from the agent's latest policy acting on a given level l as *replaying* that level l.

The level diversity of PCG environments makes them useful testbeds for studying the robustness and generalization ability of RL agents, measured by agent performance on unseen test levels. The standard test evaluation protocol for PCG environments consists of training the agent on a finite number of training levels Λ_{train}, and evaluating performance on unseen test levels Λ_{test}, drawn from the set of all levels. Training levels are sampled from an arbitrary distribution $P_{\text{train}}(l|\Lambda_{\text{train}})$. We call this training process *direct level sampling*. A common variation of this protocol sets Λ_{train} to the set of all levels, though in practice, the agent will still only effectively see a finite set of levels after training for a finite

[1]Our code is available at https://github.com/facebookresearch/level-replay.

number of steps. In the case of a finite training set, typically $P_{\text{train}}(l|\Lambda_{\text{train}}) = \textbf{Uniform}(l; \Lambda_{\text{train}})$. See Appendix C for the pseudocode outlining this procedure.

PCG environments naturally lend themselves to curriculum learning. Prior works have shown that directly altering levels to match their difficulty to the agent's abilities can improve generalization (Justesen et al., 2018; Dennis et al., 2020; Chevalier-Boisvert et al., 2018; Zhong et al., 2020). These findings further suggest the levels most useful for improving an agent's policy vary throughout the course of training. In this work, we consider how to automatically discover a curriculum that improves generalization for a general black-box PCG environment—crucially, without assuming any knowledge or control of how levels are generated (beyond providing the random seed or other indicial level identifier).

3. Prioritized Level Replay

In this section, we present *Prioritized Level Replay* (PLR), an algorithm for selectively sampling the next training level given the current policy, by prioritizing levels with higher estimated learning potential when replayed (that is, revisited). PLR is a drop-in replacement for the experience-collection process used in a wide range of RL algorithms. Algorithm 1 shows how it is straightforward to incorporate PLR into a generic policy-gradient training loop. For clarity, we focus on training on batches of complete trajectories (see Appendix C for pseudocode of PLR with T-step rollouts).

Our method, illustrated in Figure 1 and fully specified in Algorithm 2, induces a dynamic, nonparametric sampling distribution $P_{\text{replay}}(l|\Lambda_{\text{seen}})$ over previously visited training levels Λ_{seen} that prioritizes visited levels with higher learning potential based on properties of the agent's past trajectories. We refer to $P_{\text{replay}}(l|\Lambda_{\text{seen}})$ as the *replay distribution*. Throughout training, our method updates this replay distribution according to a heuristic score, assigning greater weight to visited levels with higher *future* learning potential. Using dynamic arrays S and C of equal length to Λ_{seen}, PLR tracks level scores $S_i \in S$ for each visited training level l_i based on the latest episode trajectory on l_i, as well as the episode count $C_i \in C$ at which each level $l_i \in \Lambda_{\text{seen}}$ was last sampled. Our method updates P_{replay} after each terminated episode by computing a mixture of two distributions, P_S, based on the level scores, and P_C, based on how long ago each level was last sampled:

$$P_{\text{replay}} = (1-\rho)\cdot P_S + \rho \cdot P_C, \tag{1}$$

where the staleness coefficient $\rho \in [0, 1]$ is a hyperparameter. We discuss how we compute level scores S_i, parameterizing the scoring distribution P_S, and the staleness distribution P_C, in Sections 3.1 and 3.2, respectively.

PLR chooses the next level at the start of every training episode by first sampling a replay-decision from a Bernoulli (or similar) distribution $P_D(d)$ to determine whether to replay a level sampled from the replay distribution $P_{\text{replay}}(l|\Lambda_{\text{seen}})$ or to experience a new, unseen level from Λ_{train}, according to some distribution $P_{\text{new}}(l|\Lambda_{\text{train}} \setminus \Lambda_{\text{seen}})$. In practice, for the case of a finite number of training levels, we implement P_{new} as a uniform distribution over the remaining unseen levels. For the case of a countably infinite number of training levels, we simulate P_{new} by sampling levels from P_{train} until encountering an unseen level. In our experiments based on a finite number of training levels, we opt to naturally anneal $P_D(d=1)$ as $|\Lambda_{\text{seen}}|/|\Lambda_{\text{train}}|$, so replay occurs more often as more training levels are visited.

The following sections describes how Prioritized Level Replay updates the replay distribution $P_{\text{replay}}(l|\Lambda_{\text{seen}})$, namely through level scoring and staleness-aware prioritization.

Algorithm 1 Policy-gradient training loop with PLR

Input: Training levels Λ_{train}, policy π_θ,
policy update function $\mathcal{U}(\mathcal{B}, \theta) \to \theta'$, and batch size N_b.
Initialize level scores S and level timestamps C
Initialize global episode counter $c \leftarrow 0$
Initialize the ordered set of visited levels $\Lambda_{\text{seen}} = \varnothing$
Initialize experience buffer $\mathcal{B} = \varnothing$
while training **do**
  $\mathcal{B} \leftarrow \varnothing$
  while collecting experiences **do**
    $\mathcal{B} \leftarrow \mathcal{B} \cup$ **collect_experiences**$(\Lambda_{\text{train}}, \Lambda_{\text{seen}}, \pi_\theta, S, C, c)$
    Using Algorithm 2
  end while
  $\theta \leftarrow \mathcal{U}(\mathcal{B}, \theta)$ *Update policy using collected experiences*
end while

Algorithm 2 Experience collection with PLR

Input: Training levels Λ_{train}, visited levels Λ_{seen}, policy π,
global level scores S, global level timestamps C, and
global episode counter c.
Output: A sampled trajectory τ
$c \leftarrow c + 1$
Sample replay decision $d \sim P_D(d)$
if $d = 0$ **and** $|\Lambda_{\text{train}} \setminus \Lambda_{\text{seen}}| > 0$ **then**
  Define new index $i \leftarrow |S| + 1$
  Sample $l_i \sim P_{\text{new}}(l|\Lambda_{\text{train}}, \Lambda_{\text{seen}})$ *Sample an unseen level*
  Add l_i to Λ_{seen}
  Add initial value $S_i = 0$ to S and $C_i = 0$ to C
else
  Sample $l_i \sim (1-\rho)\cdot P_S(l|\Lambda_{\text{seen}}, S) + \rho \cdot P_C(l|\Lambda_{\text{seen}}, C, c)$
  Sample a level for replay
end if
Sample $\tau \sim P_\pi(\tau|l_i)$
Update score $S_i \leftarrow \textbf{score}(\tau, \pi)$ and timestamp $C_i \leftarrow c$

3.1. Scoring Levels for Learning Potential

After collecting each complete episode trajectory τ on level l_i using policy π, our method assigns l_i a score $S_i = \textbf{score}(\tau, \pi)$ measuring the learning potential of replaying l_i in the future. We employ a function of the TD-error at timestep t, $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, as a proxy

for this learning potential. The expectation of the TD-error over next states is equivalent to the advantage estimate, and therefore higher-magnitude TD-errors imply greater discrepancy between expected and actual returns, making δ_t a useful measure of the learning potential in revisiting a particular state transition. To prioritize the learning potential of future experiences resulting from replaying a level, we use a scoring function based on the *average magnitude* of the Generalized Advantage Estimate (GAE; Schulman et al., 2016) over each of T time steps in the latest trajectory τ from that level:

$$S_i = \textbf{score}(\tau, \pi) = \frac{1}{T}\sum_{t=0}^{T}\left|\sum_{k=t}^{T}(\gamma\lambda)^{k-t}\delta_k\right|. \quad (2)$$

While the GAE at time t is most commonly expressed as the discounted sum of all 1-step TD-errors starting at t as in Equation 2, it is equivalent to an exponentially-discounted sum of all k-step TD-errors from t, with discount factor λ. By considering all k-step TD-errors, the GAE mitigates the bias introduced by the bootstrap term in 1-step TD-errors. The discount factor λ then controls the trade-off between bias and variance. Our scoring function considers the absolute value of the GAE, as we assume the learning potential grows with the magnitude of the TD-error irrespective of its sign. This also avoids opposite signed errors canceling out.

Another useful interpretation of Equation 2 comes from observing that the GAE magnitude at t is equivalent to the L1 value loss $|\hat{V}_t - V_t|$ under a policy-gradient algorithm that uses GAE for its own advantage estimates (and therefore value targets $\hat{V}_t$), as done in state-of-the-art implementations of PPO (Schulman et al., 2017) used in our experiments. Unless otherwise indicated, PLR refers to the instantiation of our algorithm with L1 value loss as the scoring function.

We further formulate the *Value Correction Hypothesis* to motivate our approach: In sparse reward settings, prioritizing the sampling of training levels with greatest average absolute value loss leads to a curriculum that improves both sample efficiency and generalization. We reason that on threshold levels (i.e. those at the limit of the agent's current abilities) the agent will see non-stationary returns (or value targets)—and therefore incur relatively high value errors—until it learns to solve them consistently. In contrast, levels beyond the agent's current abilities tend to result in stationary value targets signaling failure and therefore low value errors, until the agent learns useful, transferable behaviors from threshold levels. Prioritizing levels by value loss then naturally guides the agent along the expanding threshold of its ability—without the need for any externally provided measure of difficulty. We believe that learning behaviors systematically aligned with the inherent complexities of the environment in this way may lead to better generalization, and will seek to verify this empirically in Section 5.2.

While we provide principled motivations for our specific choice of scoring function, we emphasize that in general, the scoring function can be any approximation of learning potential based on trajectory values. Note that candidate scoring functions should asymptotically decrease with frequency of level visitation to avoid mode collapse of P_{replay} to a limited set of levels and possible overfitting. In Section 4, we compare our choice of the GAE magnitude, or equivalently, the L1 value loss, to alternative TD-error-based and uncertainty-based scoring approaches, listed in Table 1.

Given level scores, we use normalized outputs of a prioritization function h evaluated over these scores and tuned using a temperature parameter β to define the score-prioritized distribution $P_S(\Lambda_{\text{train}})$ over the training levels, under which

$$P_S(l_i|\Lambda_{\text{seen}}, S) = \frac{h(S_i)^{1/\beta}}{\sum_j h(S_j)^{1/\beta}}. \quad (3)$$

The function h defines how differences in level scores translate into differences in prioritization. The temperature parameter β allows us to tune how much $h(S)$ ultimately determines the resulting distribution. We make the design choice of using rank prioritization, for which $h(S_i) = 1/\text{rank}(S_i)$, where $\text{rank}(S_i)$ is the rank of level score S_i among all scores sorted in descending order. We also experimented with proportional prioritization ($h(S_i) = S_i$) as well as greedy prioritization (the level with the highest score receives probability 1), both of which tend to perform worse.

3.2. Staleness-Aware Prioritization

As the scores used to parameterize P_S are a function of the state of the policy at the time the associated level was last played, they come to reflect a gradually more off-policy measure the longer they remain without an update through replay. We mitigate this drift towards "off-policy-ness" by explicitly mixing the sampling distribution with a staleness-prioritized distribution P_C:

$$P_C(l_i|\Lambda_{\text{seen}}, C, c) = \frac{c - C_i}{\sum_{C_j \in C} c - C_j} \quad (4)$$

which assigns probability mass to each level l_i in proportion to the level's *staleness* $c - C_i$. Here, c is the count of total episodes sampled so far in training and C_i (referred to as the level's timestamp) is the episode count at which l_i was last sampled. By pushing support to levels with staler scores, P_C ensures no score drifts too far off-policy.

Plugging Equations 3 and 4 into Equation 1 gives us a replay distribution that is calculated as

$$P_{\text{replay}}(l_i) = (1-\rho)\cdot P_S(l_i|\Lambda_{\text{seen}}, S) + \rho \cdot P_C(l_i|\Lambda_{\text{seen}}, C, c).$$

Thus, a level has a greater chance of being sampled when its score is high or it has not been sampled for a long time.

4. Experimental Setting

We evaluate PLR on several PCG environments with various combinations of scoring functions and prioritization schemes, and compare to the most common direct level sampling baseline of $P_{\text{train}}(l|\Lambda_{\text{train}}) = \textbf{Uniform}(l; \Lambda_{\text{train}})$. We train and test on all 16 environments in the Procgen Benchmark on easy and hard difficulties, but focus discussion on the easy results, which allow direct comparison to several prior studies. We compare to UCB-DrAC (Raileanu et al., 2021), the state-of-the-art image augmentation method on this benchmark, and mixreg (Wang et al., 2020a), a recently introduced data augmentation method. We also compare to TSCL Window (Matiisen et al., 2020), which resembles PLR with an alternative scoring function using the slope of recent returns and no staleness sampling. For fair comparison, we also evaluate a custom TSCL Window variant that mixes in the staleness distribution P_C weighted by $\rho > 0$. Further, to demonstrate the ease of combining PLR with other methods, we evaluate UCB-DrAC using PLR for sampling training levels. Finally, we test the Value Correction Hypothesis on two challenging MiniGrid environments.

We measure episodic *test returns* per game throughout training, as well as the performance of the final policy over 100 unseen test levels of each game relative to PPO with uniform sampling. We also evaluate the mean normalized episodic test return and mean generalization gap, averaged over all games (10 runs each). We normalize returns according to Cobbe et al. (2019) and compute the generalization gap as train returns minus test returns. Thus, a larger gap indicates more overfitting, making it an apt measure of generalization. We assess statistical significance at $p = 0.05$, using the Welch t-test.

In line with the standard baseline for these environments, all experiments use PPO with GAE for training. For Procgen, we use the same ResBlock architecture as Cobbe et al. (2020a) and train for 25M total steps on 200 levels on the easy setting as in the original baselines. For MiniGrid, we use a 3-layer CNN architecture based on Igl et al. (2019), and provide approximately 1000 levels of each difficulty per environment during training. Detailed descriptions of the environments, architectures, and hyperparameters used in our experiments (and how they were set or obtained) can be found in Appendix A. See Table 1 for the full set of scoring functions investigated in our experiments.

Additionally, we extend PLR to support training on an unbounded number of levels by tracking a rolling, finite buffer of the top levels so far encountered by learning potential. Appendix B.3 reports the results of this extended PLR algorithm when training on the full level distribution of the MiniGrid environments featured in our main experiments.

Table 1. Scoring functions investigated in this work.

Scoring function	$\textbf{score}(\tau, \pi)$
Policy entropy	$\frac{1}{T}\sum_{t=0}^{T}\sum_{a}\pi(a,s_t)\log\pi(a,s_t)$
Policy min-margin	$\frac{1}{T}\sum_{t=0}^{T}(\max_a \pi(a,s_t) - \max_{a\neq\max_a\pi(a,s_t)}\pi(a,s_t))$
Policy least-confidence	$\frac{1}{T}\sum_{t=0}^{T}(1-\max_a\pi(a,s_t))$
1-step TD error	$\frac{1}{T}\sum_{t=0}^{T}\lvert\delta_t\rvert$
GAE	$\frac{1}{T}\sum_{t=0}^{T}\sum_{k=t}^{T}(\gamma\lambda)^{k-t}\delta_k$
GAE magnitude (L1 value loss)	$\frac{1}{T}\sum_{t=0}^{T}\left\lvert\sum_{k=t}^{T}(\gamma\lambda)^{k-t}\delta_k\right\rvert$

5. Results and Discussion

Our main findings are that (i) PLR significantly improves both sample efficiency and generalization, attaining the highest normalized mean test and train returns and mean reduction in generalization gap on Procgen out of all individual methods evaluated, while matching UCB-DrAC in test improvement relative to PPO; (ii) alternative scoring functions lead to inconsistent improvements across environments; (iii) PLR combined with UCB-DrAC sets a new state-of-the-art on Procgen; and (iv) PLR induces an implicit curriculum over training levels, which substantially aids training in two challenging MiniGrid environments.

5.1. Procgen Benchmark

Our results, summarized in Figure 2, show PLR with rank prioritization ($\beta = 0.1$, $\rho = 0.1$) leads to the largest statistically significant gains in mean normalized test and train returns and reduction in generalization gap compared to uniform sampling, outperforming all other methods besides UCB-DrAC + PLR. PLR combined with UCB-DrAC sees the most drastic improvements in these metrics. As reported in Table 2, UCB-DrAC + PLR yields a 76% improvement in mean test return relative to PPO with uniform sampling, and a 28% improvement relative to the previous state-of-the-art set by UCB-DrAC. While PLR with rank prioritization leads to statistically significant gains in test return on 10 of 16 environments and proportional prioritization, on 11 of 16 games, we prefer rank prioritization: While we find the two comparable in mean normalized returns, Figure 3 shows rank prioritization results in higher mean *unnormalized* test returns and a significantly lower mean generalization gap, averaged over all environments.

Further, Figure 3 shows that gains only occur when P_{replay} considers *both* level scores and staleness ($0 < \rho < 1$), highlighting the importance of staleness-based sampling in keeping scores from drifting off-policy. Lastly, we also

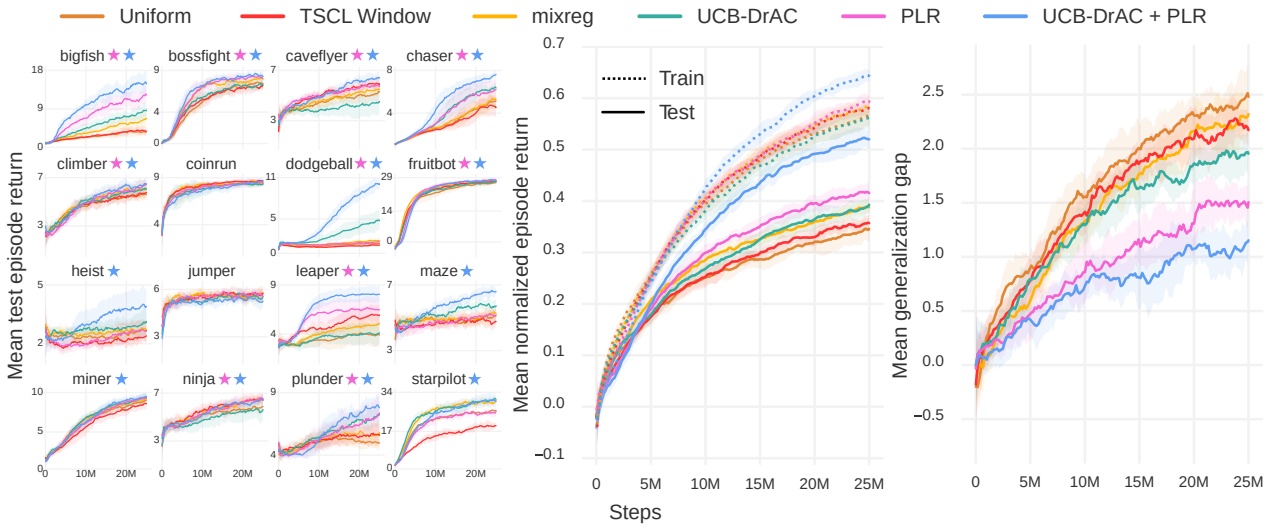


Figure 2. Left: Mean episodic test returns (10 runs) of each method. Each colored ★ indicates statistically significant ($p < 0.05$) gains in final test performance or sample complexity along the curve, relative to uniform sampling, for the PLR-based method of the same color. Center: Mean normalized train and test returns averaged across all games. Right: Mean generalization gaps averaged across all games.

benchmarked PLR on the hard setting against the same set of methods, where it again leads with 35% greater test returns relative to uniform sampling and 83% greater test returns when combined with UCB-DrAC. Figures 10–18 and Table 4 in Appendix B report additional details on these results.

The alternative scoring metrics based on TD-error and classifier uncertainty perform inconsistently across games. While certain games, such as BigFish, see improved sample-efficiency and generalization under various scoring functions, others, such as Ninja, see no improvement or worse, degraded performance. See Figure 3 for an example of this inconsistent effect across games. We find the best-performing variant of TSCL Window does not incorporate staleness information ($\rho = 0$) and similarly leads to inconsistent outcomes across games at test time, notably significantly worsening performance on StarPilot, as seen in Figure 2, and increasing the generalization gap on some environments as revealed in Figure 15 of Appendix B.

5.2. MiniGrid

We provide empirical support for the Value Correction Hypothesis (defined in Section 3) on two challenging MiniGrid environments, whose levels fall into discrete difficulties (e.g. by number of rooms to be traversed). In both, PLR with rank prioritization significantly improves sample efficiency and generalization over uniform sampling, demonstrating our method also works well in discrete state spaces. We find a staleness coefficient of $\rho = 0.3$ leads to the best test performance on MiniGrid. The top row of Figure 4 summarizes these results.

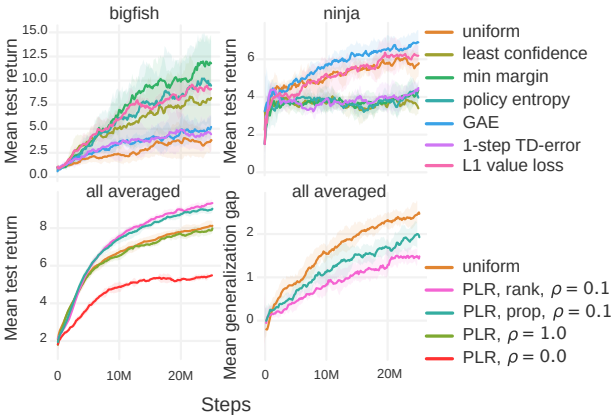


Figure 3. Top: Two example Procgen environments, between which all scoring functions except L1 value loss show inconsistent improvements to test performance (rank prioritization, $\beta = 0.1$, $\rho = 0.3$). This inconsistency holds across settings in our grid search. Bottom: Mean unnormalized episodic test returns (left) and mean generalization gap (right) for various PLR settings.

To test our hypothesis, we bin each level into its corresponding difficulty, expressed as ascending, discrete values (note that PLR does not have access to this privileged information). In the bottom row of Figure 4, we see how the expected difficulty of levels sampled using PLR changes during training for each environment. We observe that as P_{replay} is updated, levels become sampled according to an implicit curriculum over the training levels that prioritizes progressively harder levels. Of particular note, PLR seems to struggle to discover a useful curriculum for around the first 4,000 updates on ObstructedMazeGamut-Medium, at which point it discovers

Table 2. Test returns of policies trained using each method with its best hyperparameters. Following Raileanu et al. (2021), the reported mean and standard deviations per environment are computed by evaluating the final policy's average return on 100 test episodes, aggregated across multiple training runs (10 runs for Procgen Benchmark and 3 for MiniGrid, each initialized with a different training seed). Normalized test returns per run are computed by dividing the average test return per run for each environment by the corresponding average test return of the uniform-sampling baseline over all runs. We then report the means and standard deviations of normalized test returns aggregated across runs. We report the normalized return statistics for Procgen and MiniGrid environments separately. Bolded methods are not significantly different from the method with highest mean, unless all are, in which case none are bolded.

Environment	Uniform	TSCL	mixreg	UCB-DrAC	PLR	UCB-DrAC + PLR
BigFish	3.7 ± 1.2	4.3 ± 1.3	6.9 ± 1.6	8.7 ± 1.1	10.9 ± 2.8	$\mathbf{14.3 \pm 2.1}$
BossFight	7.7 ± 0.4	7.4 ± 0.8	8.1 ± 0.7	7.7 ± 0.7	$\mathbf{8.9 \pm 0.4}$	$\mathbf{8.8 \pm 0.8}$
CaveFlyer	5.4 ± 0.8	$\mathbf{6.3 \pm 0.6}$	6.0 ± 0.6	4.6 ± 0.9	$\mathbf{6.3 \pm 0.5}$	$\mathbf{6.8 \pm 0.7}$
Chaser	5.2 ± 0.7	4.9 ± 1.0	5.7 ± 1.1	6.8 ± 0.9	6.9 ± 1.2	$\mathbf{8.0 \pm 0.6}$
Climber	5.9 ± 0.6	6.0 ± 0.8	$\mathbf{6.6 \pm 0.7}$	$\mathbf{6.4 \pm 0.9}$	$\mathbf{6.3 \pm 0.8}$	$\mathbf{6.8 \pm 0.7}$
CoinRun	8.6 ± 0.4	$\mathbf{9.2 \pm 0.2}$	8.6 ± 0.3	8.6 ± 0.4	8.8 ± 0.5	$\mathbf{9.0 \pm 0.4}$
Dodgeball	1.7 ± 0.2	1.2 ± 0.4	1.8 ± 0.4	5.1 ± 1.6	1.8 ± 0.5	$\mathbf{10.3 \pm 1.4}$
FruitBot	27.3 ± 0.9	27.1 ± 1.6	27.7 ± 0.8	27.0 ± 1.3	28.0 ± 1.3	27.6 ± 1.5
Heist	2.8 ± 0.9	2.5 ± 0.6	2.7 ± 0.4	3.2 ± 0.7	2.9 ± 0.5	$\mathbf{4.9 \pm 1.3}$
Jumper	5.7 ± 0.4	6.1 ± 0.6	6.1 ± 0.3	5.6 ± 0.5	5.8 ± 0.5	5.9 ± 0.3
Leaper	4.2 ± 1.3	6.4 ± 1.2	5.2 ± 1.1	4.4 ± 1.4	6.8 ± 1.2	$\mathbf{8.7 \pm 1.0}$
Maze	5.5 ± 0.4	5.0 ± 0.3	5.4 ± 0.5	6.2 ± 0.5	5.5 ± 0.8	$\mathbf{7.2 \pm 0.8}$
Miner	8.7 ± 0.7	8.9 ± 0.6	9.5 ± 0.4	$\mathbf{10.1 \pm 0.6}$	$\mathbf{9.6 \pm 0.6}$	$\mathbf{10.0 \pm 0.5}$
Ninja	6.0 ± 0.4	$\mathbf{6.8 \pm 0.5}$	$\mathbf{6.9 \pm 0.5}$	5.8 ± 0.8	$\mathbf{7.2 \pm 0.4}$	$\mathbf{7.0 \pm 0.5}$
Plunder	5.1 ± 0.6	5.9 ± 1.1	5.7 ± 0.5	$\mathbf{7.8 \pm 0.9}$	$\mathbf{8.7 \pm 2.2}$	$\mathbf{7.7 \pm 0.9}$
StarPilot	26.8 ± 1.5	19.8 ± 3.4	$\mathbf{32.7 \pm 1.5}$	$\mathbf{31.7 \pm 2.4}$	27.9 ± 4.4	29.6 ± 2.2
Normalized test returns (%)	100.0 ± 4.5	103.0 ± 3.6	113.8 ± 2.8	129.8 ± 8.2	128.3 ± 5.8	$\mathbf{176.4 \pm 6.1}$
MultiRoom-N4-Random	0.80 ± 0.04	–	–	–	$\mathbf{0.81 \pm 0.01}$	–
ObstructedMazeGamut-Easy	0.53 ± 0.04	–	–	–	$\mathbf{0.85 \pm 0.04}$	–
ObstructedMazeGamut-Med	0.65 ± 0.01	–	–	–	$\mathbf{0.73 \pm 0.07}$	–
Normalized test returns (%)	100.0 ± 2.5	–	–	–	$\mathbf{124.3 \pm 4.7}$	–

a curriculum that gradually assigns more weight to harder levels. This curriculum enables PLR with access to only $6{,}000$ training levels to attain even higher mean test returns than the uniform-sampling baseline with access to the full set of training levels, of which there are roughly 4 billion (so our training levels constitute 0.00015% of the total number).

We further tested an extended version of PLR that trains on the full level distribution on these two environments by tracking a buffer of levels with the highest estimated learning potential. We find it outperforms uniform sampling with access to the full level distribution. These additional results are presented in Appendix B.3.

6. Related Work

Several methods for improving generalization in deep RL adapt techniques from supervised learning, including stochastic regularization (Igl et al., 2019; Cobbe et al., 2020a), data augmentation (Kostrikov et al., 2020; Raileanu et al., 2021; Wang et al., 2020a), and feature distillation (Igl et al., 2020; Cobbe et al., 2020b). In contrast, PLR modifies only how the next training level is sampled, thereby easily combining with any model or RL algorithm.

The selective-sampling performed by PLR makes it a form of active learning (Cohn et al., 1994; Settles, 2009). Our work also echoes ideas from Graves et al. (2017), who train a multi-armed bandit to choose the next task in multi-task supervised learning, so to maximize gradient-based progress signals. Sharma et al. (2018) extend these ideas to multi-task RL, but add the additional requirement of knowing a maximum target return for each task a priori. Zhang et al. (2020b) use an ensemble of value functions for selective goal sampling in the off-policy continuous control setting, requiring prior knowledge of the environment structure to generate candidate goals. Unlike PLR, these methods assume the ability to sample tasks or levels based on their structural properties, an assumption that does not typically hold for PCG simulators. Instead, our method automatically uncovers similarly difficult levels, giving rise to a curriculum without prior knowledge of the environment.

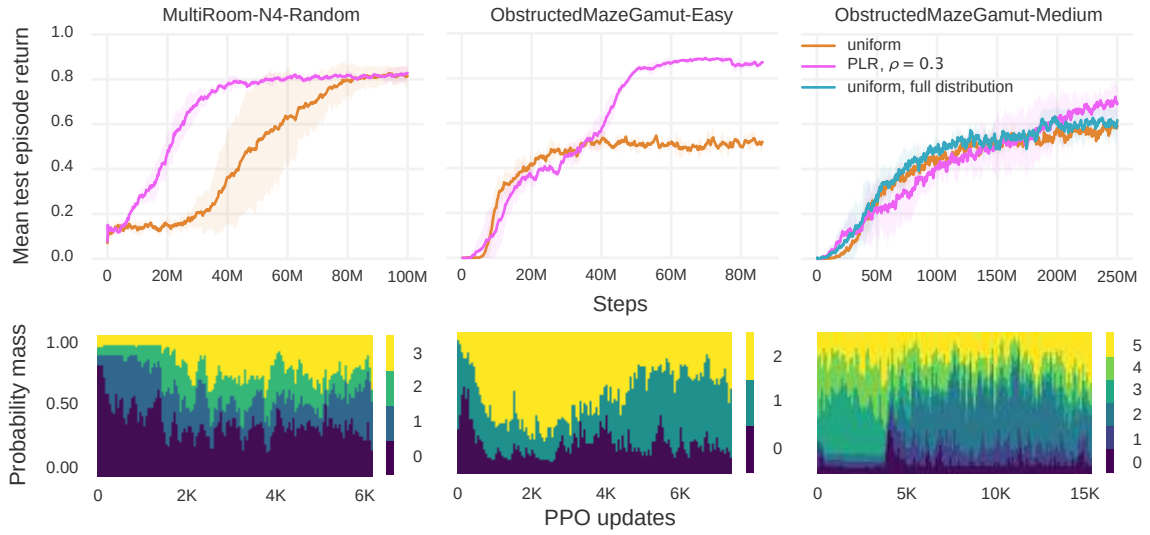


Figure 4. Top: Mean episodic test returns of PLR and the uniform-sampling baseline on MultiRoom-N4-Random (4 runs), ObstructedMazeGamut-Easy (3 runs), and ObstructedMazeGamut-Medium (3 runs). Bottom: The probability mass assigned to levels of varying difficulty over the course of training in a single, randomly selected run for the respective environment.

A recent theme in the PCG setting explores adaptively generating levels to facilitate learning (Sukhbaatar et al., 2017; Wang et al., 2019; 2020b; Khalifa et al., 2020; Akkaya et al., 2019; Campero et al., 2020; Dennis et al., 2020). Unlike these approaches, our method does not assume control over level generation, requiring only the ability to replay previously visited levels. These methods also require parameterizing level generation with additional learning modules. In contrast, our approach does not require such extensions of the environment, for example including teacher-specific action spaces in the case of Campero et al. (2020). Most similar to our method, Matiisen et al. (2020) proposes a teacher-student curriculum learning (TSCL) algorithm that samples training levels by considering the change in episodic returns per level, though they neither design nor test the method for generalization. As shown in Section 5.1, it provides inconsistent benefits at test time. Further, unlike TSCL, PLR does not assume access to all levels at the start of training, and as we show in Appendix B.3, PLR can be extended to improve sample efficiency and generalization by training on an unbounded number of training levels.

Like our method, Schaul et al. (2016) and Kapturowski et al. (2019) use TD-errors to estimate learning potential. While these methods make use of TD-errors to prioritize learning from *past* experiences, our method uses such estimates to prioritize revisiting levels for generating entirely new *future* experiences for learning.

Generalization requires sufficient exploration of environment states and dynamics. Thus, recent exploration strategies (e.g. Raileanu & Rocktäschel, 2020; Campero et al., 2020; Zhang et al., 2020a; Zha et al., 2021) shown to benefit simple PCG settings are complementary to the aims of this work. However, as these studies focus on PCG environments with low-dimensional state spaces, whether such methods can be successfully applied to more complex PCG environments like Procgen Benchmark remains to be seen. If so, they may potentially combine with PLR to yield additive improvements. We believe the interplay between such exploration methods and PLR to be a promising direction for future research.

7. Conclusion and Future Work

We introduced Prioritized Level Replay (PLR), an algorithm for selectively sampling the next training level in PCG environments based on the estimated learning potential of revisiting each level for the current policy. We showed that our method remarkably improves both the sample efficiency and generalization of deep RL agents in PCG environments, including the majority of environments in Procgen Benchmark and two challenging MiniGrid environments. We further combined PLR with the prior leading method to set a new state-of-the-art on Procgen Benchmark. Further, on MiniGrid environments, we showed PLR induces an emergent curriculum of increasingly more difficult levels.

The flexibility of the PCG abstraction makes PLR applicable to many problems of practical importance, for example, robotic object manipulation tasks, where domain randomized environment instances map to the notion of levels. We believe PLR may even be applicable to singleton environments, given a procedure for generating variations of the underlying MDP as a function of a level identifier, for ex-

ample, by varying the starting positions of entities. Another natural extension of PLR is to adapt the method to operate in the goal-conditioned setting, by incorporating goals into the level parameterization.

Despite the wide applicability of PCG and consequently PLR, not all problem domains can be effectively represented in seed-based simulation. Many real world problems require transfer into domains too complex to be adequately captured by simulation, such as car driving, where realizing a completely faithful simulation would entail solving the very same control problem of interest, creating a chicken-and-egg dilemma. Further, environment resets are not universally available, such as in the continual learning setting, where the agent interacts with the environment without explicit episode boundaries—arguably, a more realistic interaction model for a learning agent deployed in the wild.

Still, pre-training in simulation with resets can nevertheless benefit such settings, where the target domain is rife with open-ended complexity and where resets are unavailable, especially as training through real-world interactions can be slow, expensive, and precarious. In fact, in practice, we almost exclusively train deep RL policies in simulation for these reasons. As PLR provides a simple method to more fully exploit the simulator for improved test-time performance, we believe PLR can also be adapted to improve learning in these settings.

We further note that while we empirically demonstrated that the L1 value loss acts as a highly effective scoring function, there likely exist even more potent choices. Directly learning such functions may reveal even better alternatives. Lastly, combining PLR with various exploration strategies may further improve test performance in hard exploration environments. We look forward to investigating each of these promising directions in future work, prioritized accordingly, by learning potential.

Acknowledgements

We thank Roberta Raileanu, Heinrich Küttler, and Jakob Foerster for useful discussions and feedback on this work, and our anonymous reviewers, for their recommendations on improving this paper.

References

Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., Petron, A., Paino, A., Plappert, M., Powell, G., Ribas, R., Schneider, J., Tezak, N., Tworek, J., Welinder, P., Weng, L., Yuan, Q., Zaremba, W., and Zhang, L. Solving rubik's cube with a robot hand. *CoRR*, abs/1910.07113, 2019. URL http://arxiv.org/abs/1910.07113.

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, O. P., and Zaremba, W. Hindsight experience replay. In *Advances in neural information processing systems*, pp. 5048–5058, 2017.

Bard, N., Foerster, J. N., Chandar, S., Burch, N., Lanctot, M., Song, H. F., Parisotto, E., Dumoulin, V., Moitra, S., Hughes, E., Dunning, I., Mourad, S., Larochelle, H., Bellemare, M. G., and Bowling, M. The hanabi challenge: A new frontier for AI research. *Artif. Intell.*, 280:103216, 2020. doi: 10.1016/j.artint.2019.103216. URL https://doi.org/10.1016/j.artint.2019.103216.

Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, Jun 2013. ISSN 1076-9757. doi: 10.1613/jair.3912. URL http://dx.doi.org/10.1613/jair.3912.

Campero, A., Raileanu, R., Küttler, H., Tenenbaum, J. B., Rocktäschel, T., and Grefenstette, E. Learning with AMIGo: Adversarially Motivated Intrinsic Goals. *CoRR*, abs/2006.12122, 2020. URL https://arxiv.org/abs/2006.12122.

Chevalier-Boisvert, M., Bahdanau, D., Lahlou, S., Willems, L., Saharia, C., Nguyen, T. H., and Bengio, Y. BabyAI: First steps towards grounded language learning with a human in the loop. *CoRR*, abs/1810.08272, 2018. URL http://arxiv.org/abs/1810.08272.

Chevalier-Boisvert, M., Willems, L., and Pal, S. Minimalistic gridworld environment for OpenAI Gym. https://github.com/maximecb/gym-minigrid, 2018.

Cobbe, K., Klimov, O., Hesse, C., Kim, T., and Schulman, J. Quantifying generalization in reinforcement learning. In *International Conference on Machine Learning*, pp. 1282–1289. PMLR, 2019.

Cobbe, K., Hesse, C., Hilton, J., and Schulman, J. Leveraging Procedural Generation to Benchmark Reinforcement Learning. In *International Conference on Machine Learning*, pp. 2048–2056. PMLR, November 2020a. URL http://proceedings.mlr.press/v119/cobbe20a.html. ISSN: 2640-3498.

Cobbe, K., Hilton, J., Klimov, O., and Schulman, J. Phasic Policy Gradient. *CoRR*, abs/2009.04416, 2020b. URL https://arxiv.org/abs/2009.04416.

Cohn, D., Atlas, L., and Ladner, R. Improving generalization with active learning. *Machine learning*, 15(2): 201–221, 1994.

A. Experiment Details and Hyperparameters

A.1. Procgen Benchmark

Procgen Benchmark consists of 16 PCG environments of varying styles, exhibiting a diversity of gameplay similar to that of the ALE benchmark. Game levels are determined by a random seed and can vary in navigational layout, visual appearance, and starting positions of entities. All Procgen environments share the same discrete 15-dimensional action space and produce $64 \times 64 \times 3$ RGB observations. (Cobbe et al., 2020a) provides a comprehensive description of each of the 16 environments. State-of-the-art RL algorithms, like PPO, lead to significant generalization gaps between test and train performance in all games, making Procgen a useful benchmark for assessing generalization performance.

We follow the standard protocol for testing generalization performance on Procgen: We train an agent for each game on a finite number of levels, N_{train}, and sample test levels from the full distribution of levels. Normalized test returns are computed as $(R - R_{\text{min}})/(R_{\text{max}} - R_{\text{min}})$, where R is the unnormalized return and each game's minimum return, R_{min}, and maximum return, R_{max}, are provided in Cobbe et al. (2020a), which uses this same normalization.

To make the most efficient use of our computational resources, we perform hyperparameter sweeps on the easy setting. This also makes our results directly comparable to most prior works benchmarked on Procgen, which have likewise focused on the easy setting. In Procgen easy, our experiments use the recommended settings of $N_{\text{train}} = 200$ and 25M steps of training, as well as the same ResNet policy architecture and PPO hyperparameters shared across all games as in Cobbe et al. (2020a) and Raileanu et al. (2021). We find 25M steps to be sufficient for uncovering differences in generalization performance among our methods and standard baselines. Moreover, under this setup, we find Procgen training runs require much less wall-clock time than training runs on the two MiniGrid environments of interest over an equivalent number of steps needed to uncover differences in generalization performance. Therefore we survey the empirical differences across various settings of PLR on Procgen easy rather than MiniGrid.

To find the best hyperparameters for PLR, we evaluate each combination of the scoring function choices in Table 1 with both rank and proportional prioritization, performing a coarse grid search for each pair over different settings of the temperature parameter β in $\{0.1, 0.5, 1.0, 1.4, 2.0\}$ and the staleness coefficient ρ in $\{0.1, 0.3, 1.0\}$. For each setting, we run 4 trials across all 16 of games of the Procgen Benchmark, evaluating based on mean unnormalized test return across games. In our TD-error-based scoring functions, we set γ and λ equal to the same respective values used by the GAE in PPO during training. We found PLR offered the most pronounced gains at $\beta = 0.1$ and $\rho = 0.1$ on Procgen, but these benefits also held for higher values ($\beta = 0.5$ and $\rho = 0.3$), though to a lesser degree.

For UCB-DrAC, we make use of the best-reported hyperparameters on the easy setting of Procgen in Raileanu et al. (2021), listed in Table 3.

We found the default setting of mixreg's $\alpha = 0.2$, as used by Wang et al. (2020a) in the hard setting, performs poorly on the easy setting. Instead, we conducted a grid search over α in $\{0.001, 0.005, 0.01, 0.05, 0.1, 0.2, 0.8, 0.2, 0.5, 0.8, 1\}$.

Since the TSCL Window algorithm was not previously evaluated on Procgen Benchmark, we perform a grid search over different settings for both Boltzmann and ϵ-greedy variants of the algorithm to determine the best hyperparameter settings for Procgen easy. We searched over window size K in $\{10, 100, 1000, 10000\}$, bandit learning rate α in $\{0.01, 0.1, 0.5, 1.0\}$, random exploration probability ϵ in $\{0.0, 0.01, 0.1, 0.5\}$ for the ϵ-greedy variant, and temperature τ in $\{0.1, 0.5, 1.0\}$ for the Boltzmann variant. Additionally, for a fairer comparison to PLR we further evaluated a variant of TSCL Window that, like PLR, incorporates the staleness distribution, by additionally searching over values of the staleness coefficient ρ in $\{0.0, 0.1, 0.3, 0.5\}$, though we ultimately found that TSCL Window performed best without staleness sampling ($\rho = 0$).

See Table 3 for a comprehensive overview of the hyperparameters used for PPO, UCB-DrAC, mixreg, and TSCL Window, shared across all Procgen environments to generate our reported results on Procgen easy.

The evaluation protocol on the hard setting entails training on 500 levels over 200M steps (Cobbe et al., 2020a), making it more compute-intensive than the easy setting. To save on computational resources, we make use of the same hyperparameters found in the easy setting for each method on Procgen hard, with one exception: As our PPO implementation does not use multi-GPU training, we were unable to quadruple our GPU actors as done in Cobbe et al. (2020a) and Wang et al. (2020a). Instead, we resorted to doubling the number of environments in our single actor to 128, resulting in mini-batch sizes half as large as used in these two prior works. As such, our baseline results on hard are not directly comparable to theirs. We found setting mixreg's $\alpha = 0.2$ as done in Wang et al. (2020a) led to poor performance under this reduced batch size. We conducted an additional grid search, finding $\alpha = 0.01$ to perform best, as on Procgen easy.

A.2. MiniGrid

The MiniGrid suite (Chevalier-Boisvert et al., 2018) features a series of highly structured environments of increasing difficulty. Each environment features a task in a grid world

Table 3. Hyperparameters used for training on Procgen Benchmark and MiniGrid environments.

Parameter	Procgen	MiniGrid
PPO		
γ	0.999	0.999
λ_{GAE}	0.95	0.95
PPO rollout length	256	256
PPO epochs	3	4
PPO minibatches per epoch	8	8
PPO clip range	0.2	0.2
PPO number of workers	64	64
Adam learning rate	5e-4	7e-4
Adam ϵ	1e-5	1e-5
return normalization	yes	yes
entropy bonus coefficient	0.01	0.01
value loss coefficient	0.5	0.5
PLR		
Prioritization	rank	rank
Temperature, β, 0.1	0.1	0.1
Staleness coefficient, ρ	0.1	0.3
UCB-DrAC		
Window size, K	10	-
Regularization coefficient, α_r	0.1	-
UCB exploration coefficient, c	0.1	-
mixreg		
Beta shape, α	0.01	-
TSCL Window		
Bandit exploration strategy	ϵ-greedy	-
Window size, K	10	-
Bandit learning rate, α	1.0	-
Exploration probability, ϵ	0.5	-

setting, and as in Procgen, environment levels are determined by a seed. Harder levels require the agent to perform longer action sequences over a combinatorially-rich set of game entities, on increasingly larger grids. The clear ordering of difficulty over subsets of MiniGrid environments allows us to track the relative difficulty of levels sampled by PLR over the course of training.

MiniGrid environments share a discrete 7-dimensional action space and produce a 3-channel integer state encoding of the 7×7 grid immediately including and in front of the agent. However, following the training setup in Igl et al. (2019), we modify the environment to produce an $N \times M \times 3$ encoding of the full grid, where N and M vary according to the maximum grid dimensions of each environment. Full observability makes generalization harder, requiring the agent to generalize across different level layouts in their entirety.

We evaluate PLR with rank prioritization on two MiniGrid environments whose levels are uniformly distributed across several difficulty settings. Training on levels of varying difficulties helps agents make use of the easier levels as stepping stones to learn useful behaviors that help the agent make progress on harder levels. However, under the uniform-sampling baseline, learning may be inefficient, as the training process does not selectively train the agent on levels of increasing difficulty, leading to wasted training steps when a difficult level is sampled early in training. On the contrary, if PLR scores levels according to the time-averaged L1 value loss of recently experienced level trajectories, the average difficulty of the sampled levels should adapt to the agent's current abilities, following the reasoning outlined in the Value Correction Hypothesis, stated in Section 3.

As in Igl et al. (2019), we parameterize the agent policy as a 3-layer CNN with 16, 32, and 32 channels, with a final hidden layer of size 64. All kernels are 2×2 and use a stride of 1. For the ObstructedMazeGamut environments, we increase the number of channels of the final CNN layer to 64. We follow the same high-level generalization evaluation protocol used for Procgen, training the agent on a fixed set of 4000 levels for MultiRoom-N4-Random, 3000 levels for ObstructedMazeGamut-Easy, and 6000 levels for ObstructedMazeGamut-Medium, and testing on the full level distribution. We chose these values for $|\Lambda_{\text{train}}|$ to ensure roughly 1000 training levels of each difficulty setting of each environment. We model our PPO parameters on those used by Igl et al. (2019) in their MiniGrid experiments. We performed a grid search to find that PLR with rank prioritization, $\beta = 0.1$, and $\rho = 0.3$ learned most quickly on the MultiRoom environment, and used this setting for all our MiniGrid experiments. Table 3 summarizes these hyperparameter choices.

The remainder of this section provides more details about the various MiniGrid environments used in this work.

MultiRoom-N4-Random This environment requires the agent to navigate through 1, 2, 3, or 4 rooms respectively to reach a goal object, resulting in a natural ordering of levels over four levels of difficulty. The agent always starts at a random position in the furthest room from the goal object, facing a random direction. The goal object is also initialized to a random position within its room. See Figure 5 for screenshots of example levels.

ObstructedMazeGamut-Easy This environment consists of levels uniformly distributed across the first three difficulty settings of the ObstructedMaze environment, in which the agent must locate and pick up the key in order to unlock the door to pick up a goal object in a second room. The agent and goal object are always initialized in random positions in different rooms separated by the locked door.

Figure 5. Example levels of each of the four difficulty levels of MultiRoom-N4-Random, in order of increasing difficulty from left to right. The agent (red triangle) must reach the goal (green square).

The second difficulty setting further requires the agent to first uncover the key from under a box before picking up the key. The third difficulty level further requires the agent to first move a ball blocking the door before entering the door. See Figure 6 for screenshots of example levels.

Figure 6. Example levels of each of the three difficulty levels of ObstructedMazeGamut-Easy, in order of increasing difficulty from left to right. The agent must find the key, which may be hidden under a box, to unlock a door, which may be blocked by an obstacle, to reach the goal object (blue circle).

ObstructedMazeGamut-Hard This environment consists of levels uniformly distributed across the first six difficulty levels of the ObstructedMaze environment. Harder levels corresponding to the fourth, fifth, and sixth difficulty settings include two additional rooms with no goal object to distract the agent. Each instance of these harder levels also contain two pairs of keys of different colors, each opening a door of the same color. The agent always starts one room away from the randomly positioned goal object. Each of the two keys is visible in the fourth difficulty setting and doors are unobstructed. The fifth difficulty setting hides the keys under boxes, and the sixth again places obstacles that must be removed before entering two of the doors, one of which is always the door to the goal-containing room. See Figure 7 for example screenshots.

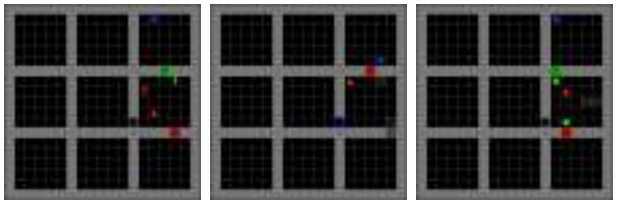


Figure 7. Example levels in increasing difficulty from left to right of each additional difficulty setting introduced by ObstructedMazeGamut-Hard in addition to those in ObstructedMazeGamut-Easy.

B. Additional Experimental Results

B.1. Extended Results on Procgen Benchmark

We present an overview of the improvements in test performance of each method across all 16 Procgen Benchmark games over 10 runs in Figure 14. For each game, Figure 15 further shows how the generalization gap changes over the course of training under each method tested. We show in Figures 16 and 17, the mean test episodic returns on the Procgen Benchmark (easy) for PLR with rank and proportional prioritization, respectively. In both of these plots, we can see that using only staleness ($\rho = 1$) or only L1 value loss scores ($\rho = 0$) is considerably worse than direct level sampling. Thus, we only observe gains compared to the baseline when both level scores and staleness are used for the sampling distribution. Comparing Figures 16 with 17 we find that PLR with proportional instead of rank prioritization provides statistically significant gains over uniform level sampling on an additional game (CoinRun), but rank prioritization leads to slightly larger mean improvements on several games.

Figure 18 shows that when PLR improves generalization performance, it also either matches or improves training sample efficiency, suggesting that when beneficial to test performance, the representations learned via the auto-curriculum induced by PLR prove similarly useful on training levels. However we see that our method reduces training sample efficiency on two games on which our method does not improve generalization performance. Since our method does not discover useful auto-curricula for these games, it is likely that uniformly sampling levels at training time allows the agent to better memorize useful behaviors on each of the training levels compared to the selective sampling performed by our method.

Finally, we also benchmarked PLR and UCB-DrAC + PLR against uniform sampling, TSCL Window, mixreg, and UCB-DrAC on Procgen hard across 5 runs per environment. Due to the high computational cost of the evaluation protocol for Procgen hard, which entails 200M training steps, we directly use the best hyperparameters found in the easy setting for each method. The results in Figure 10 show the two PLR-based methods significantly outperform all other methods in terms of normalized mean train and test episodic return, as well as reduction in mean generalization gap, attaining even greater margins of improvement than in the easy setting. As summarized by Table 4, the gains of PLR and UCB + PLR in mean normalized test return relative to uniform sampling in the hard setting are comparable to those in the easy setting. We provide plots of episodic test return over training for each individual environment in Figure 12.

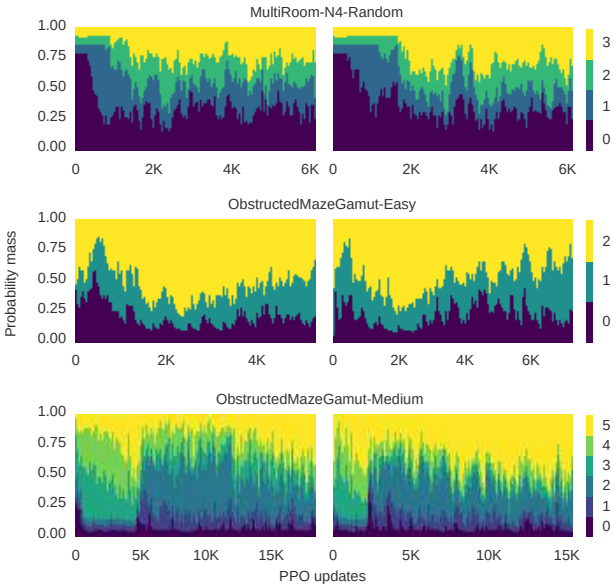


Figure 8. PLR consistently induces emergent curricula from easier to harder levels during training. Left and right correspond to two additional training runs independent of that in Figure 4.

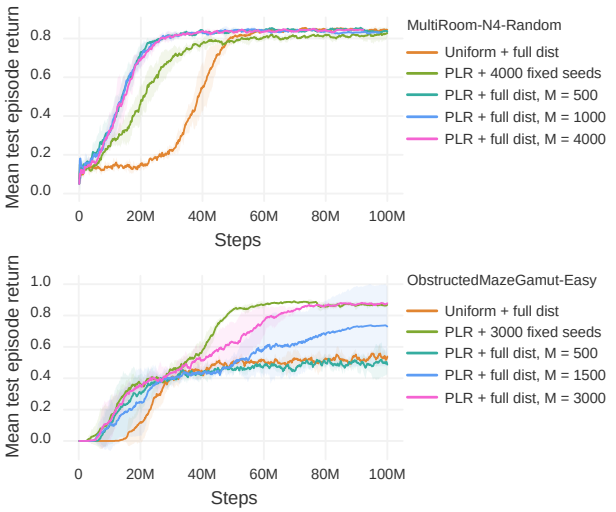


Figure 9. Mean test episodic returns on MultiRoom-N4-Random (top) and ObstructedMazeGamut-Easy (bottom) with access to the full level distribution at training. Plots are averaged over 3 runs. We set P_D to a Bernoulli parameterized as $p = 0.5$ for MultiRoom-N4-Random and $p = 0.95$ for ObstructedMazeGamut-Easy (found via grid search). As with all MiniGrid experiments using PLR, we use rank prioritization, $\beta = 0.1$, and $\rho = 0.3$.

B.2. Extended Results on Minigrid

To demonstrate that PLR consistently induces an emergent curriculum, we present plots showing the change in probability mass over different difficulty bins for additional training runs in Figure 8. Like in Figure 4, we see the probability mass assigned by P_{replay} gradually shifts from easier to harder levels over the course of training.

B.3. Training on the Full Level Distribution

While assessing generalization performance calls for using a fixed set of training levels, ideally our method can also make use of the full level distribution if given access to it. We take advantage of an unbounded number of training levels by modifying the list structures for storing scores and timestamps (see Algorithm 1 and 2) to track the top M levels by learning potential in our finite level buffer. When the lists are full, we set the next level for replacement to be

$$l_{\text{min}} = \arg\min_{l} P_{\text{replay}}(l).$$

When the outcome of the Bernoulli P_D entails sampling a new level l, the score and timestamps of l replace those of l_{min} only if the score of l_{min} is lower than that of l. In this way, PLR keeps a running buffer throughout training of the top M levels appraised to have the highest learning potential for replaying anew.

Figure 9 shows that with access to the full level distribution at training, PLR improves sample efficiency and generalization performance in both environments compared to uniform sampling on the full distribution. In MultiRoom-N4-Random, the value M makes little difference to test performance, and training with PLR on the full level distribution leads to a policy outperforming one trained with PLR on a fixed set of training levels. However, on ObstructedMazeGamut-Easy, a smaller M leads to worse test performance. Nevertheless, for all but $M = 500$, including the case of a fixed set of 3000 training levels, PLR leads to better mean test performance than uniform sampling on the full level distribution.

C. Algorithms

In this section, we provide detailed pseudocode for how PLR can be used for experience collection when using T-step rollouts. Algorithm 3 presents the extension of the generic policy-gradient training loop presented in Algorithm 1 to the case of T-step rollouts, and Algorithm 4 presents an implementation of experience collection in this setting (extending Algorithm 2). Note that when using T-step rollouts in the training loop, rollouts may start and end between episode boundaries. To compute level scores on full trajectories segmented across rollouts, we compute scores of partial episodes according to Equation 2, and record these partial scores alongside the partial episode step count in a separate buffer $\tilde{S}$. The function **score** then technically, optionally takes the additional input $\tilde{S}$ (through an abuse of notation) as an additional argument to stitch together this partial information into scores of full episodic trajectories.

Table 4. Comparison of test scores of PPO with PLR against PPO with uniform-sampling on the hard setting of Procgen Benchmark. Following (Raileanu et al., 2021), reported figures represent the mean and standard deviation of average test scores over 100 episodes aggregated across 5 runs, each initialized with a unique training seed. For each run, a normalized average return is computed by dividing the average test return for each game by the corresponding average test return of the uniform-sampling baseline over all 500 test episodes of that game, followed by averaging these normalized returns over all 16 games. The final row reports the mean and standard deviation of the normalized returns aggregated across runs. Bolded methods are not significantly different from the method with highest mean, unless all are, in which case none are bolded.

Environment	Uniform	TSCL	mixreg	UCB-DrAC	PLR	UCB-DrAC + PLR
BigFish	9.7 ± 1.8	$\mathbf{11.9 \pm 2.5}$	$\mathbf{12.0 \pm 2.5}$	10.9 ± 1.6	$\mathbf{15.3 \pm 3.6}$	$\mathbf{15.5 \pm 2.8}$
BossFight	$\mathbf{9.6 \pm 0.2}$	8.4 ± 0.7	$\mathbf{9.3 \pm 0.9}$	9.0 ± 0.2	$\mathbf{9.7 \pm 0.4}$	$\mathbf{9.5 \pm 1.1}$
CaveFlyer	3.5 ± 0.8	6.3 ± 0.6	4.0 ± 1.0	2.6 ± 0.8	6.4 ± 0.6	$\mathbf{8.0 \pm 0.9}$
Chaser	5.9 ± 0.5	6.2 ± 1.0	$\mathbf{6.5 \pm 0.8}$	$\mathbf{7.0 \pm 0.6}$	$\mathbf{6.8 \pm 2.2}$	$\mathbf{7.6 \pm 0.2}$
Climber	5.3 ± 1.1	5.2 ± 0.7	5.7 ± 0.7	6.1 ± 1.0	7.4 ± 0.6	7.6 ± 1.8
CoinRun	4.5 ± 0.4	5.8 ± 0.8	$\mathbf{6.2 \pm 1.0}$	5.2 ± 1.0	$\mathbf{6.8 \pm 0.6}$	$\mathbf{7.1 \pm 0.5}$
Dodgeball	3.9 ± 0.6	1.9 ± 0.9	4.7 ± 1.0	9.9 ± 1.2	7.4 ± 1.3	$\mathbf{12.4 \pm 0.7}$
FruitBot	$\mathbf{11.9 \pm 4.2}$	13.1 ± 2.3	$\mathbf{14.7 \pm 2.2}$	$\mathbf{15.6 \pm 3.7}$	$\mathbf{16.7 \pm 1.0}$	$\mathbf{12.9 \pm 5.1}$
Heist	1.5 ± 0.4	0.9 ± 0.3	1.2 ± 0.4	1.1 ± 0.3	1.3 ± 0.4	2.6 ± 2.2
Jumper	3.2 ± 0.3	3.2 ± 0.3	3.3 ± 0.4	2.9 ± 0.9	3.5 ± 0.5	3.3 ± 0.8
Leaper	7.1 ± 0.3	$\mathbf{7.5 \pm 0.5}$	$\mathbf{7.5 \pm 0.5}$	3.8 ± 1.6	$\mathbf{7.4 \pm 0.2}$	$\mathbf{8.2 \pm 0.7}$
Maze	3.6 ± 0.7	3.8 ± 0.6	3.9 ± 0.5	4.4 ± 0.2	4.0 ± 0.4	$\mathbf{6.2 \pm 0.4}$
Miner	12.8 ± 1.4	11.7 ± 0.9	13.3 ± 1.6	$\mathbf{16.1 \pm 0.6}$	11.3 ± 0.7	$\mathbf{15.3 \pm 0.8}$
Ninja	5.2 ± 0.1	5.9 ± 0.8	5.0 ± 1.0	5.2 ± 1.0	$\mathbf{6.1 \pm 0.6}$	$\mathbf{6.9 \pm 0.3}$
Plunder	3.2 ± 0.1	5.4 ± 1.1	3.7 ± 0.4	7.8 ± 1.1	8.6 ± 2.7	$\mathbf{17.5 \pm 1.3}$
StarPilot	5.5 ± 0.6	2.1 ± 0.4	6.9 ± 0.6	$\mathbf{11.2 \pm 1.7}$	5.4 ± 0.8	$\mathbf{12.3 \pm 1.5}$
Normalized test returns (%)	100.0 ± 2.0	103.9 ± 3.5	110.6 ± 3.9	126.6 ± 3.0	135.0 ± 6.1	$\mathbf{182.9 \pm 8.2}$

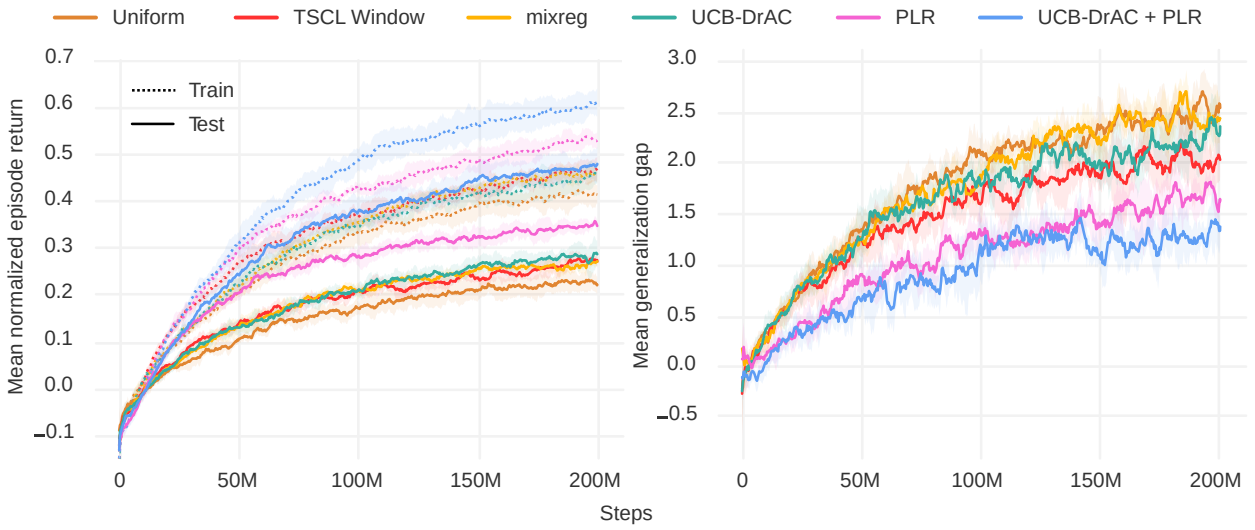


Figure 10. Left: Mean normalized train and test episode returns on Procgen Benchmark (hard). Right: Corresponding generalization gaps during training. All curves are averaged across all environments over 5 runs. The shaded area indicates one standard deviation around the mean. PLR-based methods statistically significantly outperform all others in both train and test returns. Only the PLR-based methods statistically significantly reduce the generalization gap ($p < 0.05$).

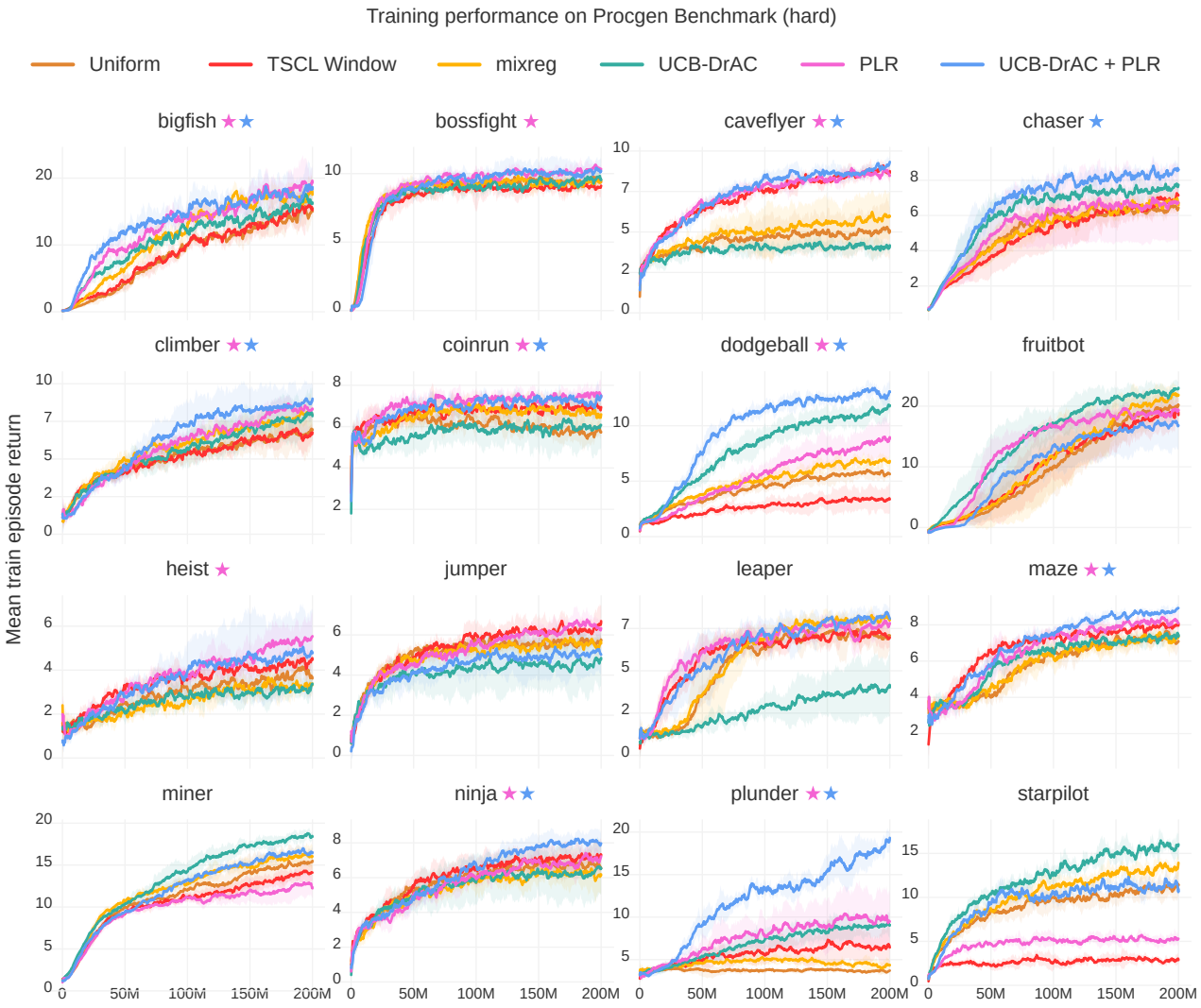


Figure 11. Mean train episode returns (5 runs) on Procgen Benchmark (hard), using the best hyperparameters found on the easy setting. The shaded area indicates one standard deviation around the mean. A ★ indicates statistically significant improvement over the uniform-sampling baseline by the PLR-based method of the matching color ($p < 0.05$). Note that while PLR reduces training performance on StarPilot, it performs comparably to the uniform-sampling baseline at test time, indicating less overfitting to training levels.

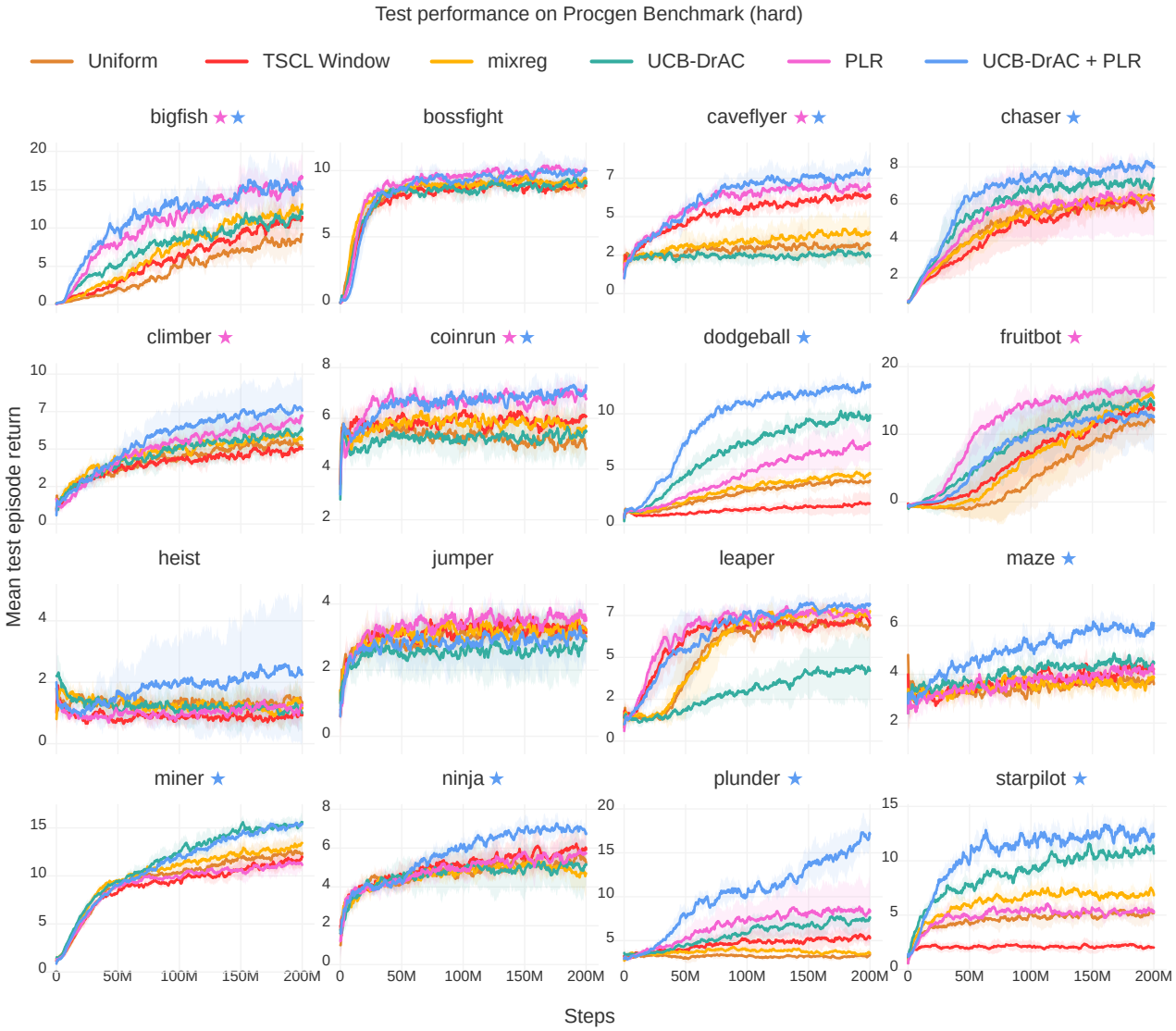


Figure 12. Mean test episode returns (5 runs) on Procgen Benchmark (hard), using best hyperparameters found on the easy setting. The shaded area indicates one standard deviation around the mean. A ★ indicates statistically significant improvement over the uniform-sampling baseline by the PLR-based method of the matching color ($p < 0.05$).

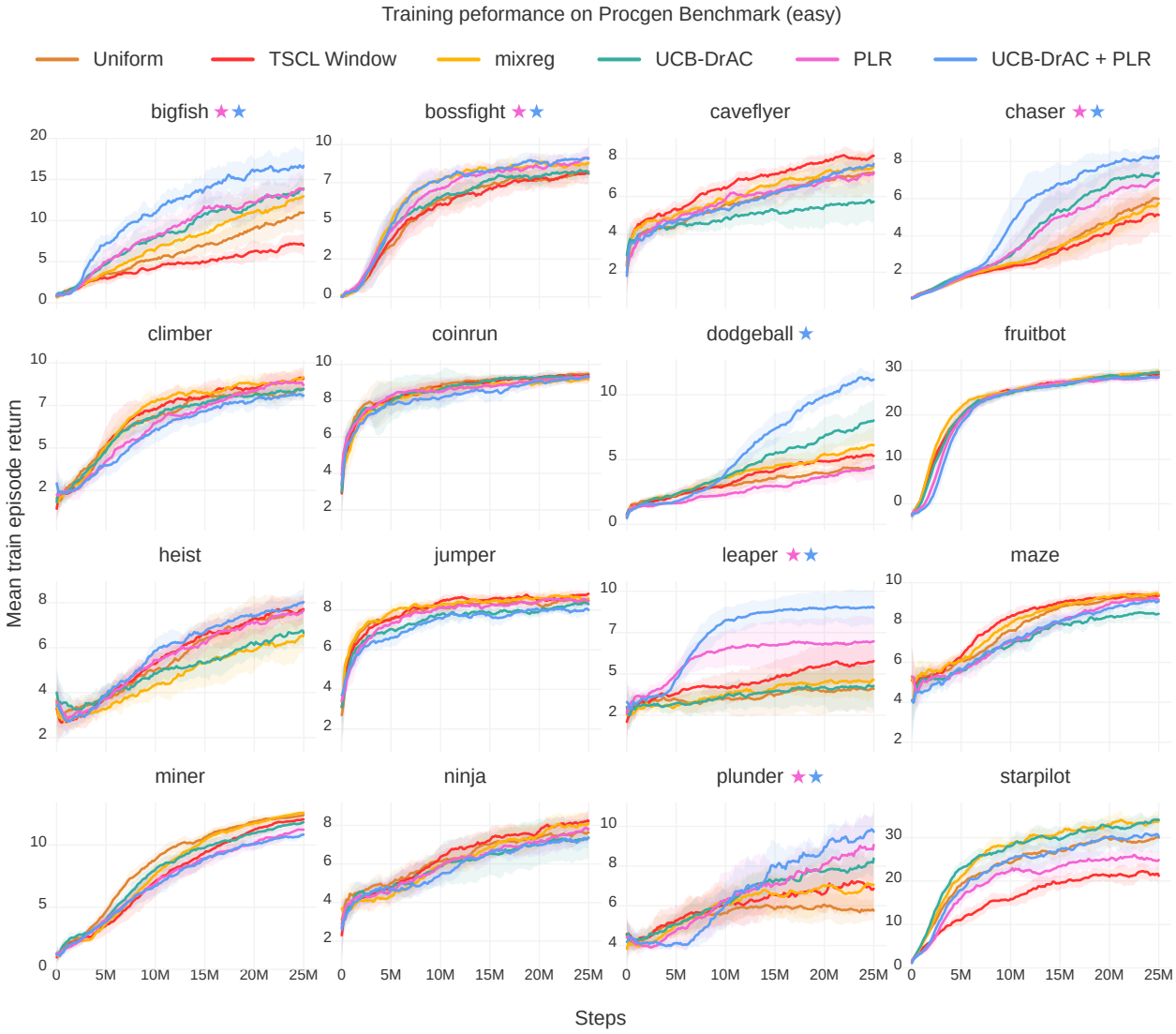


Figure 13. Mean train episode returns (5 runs) on Procgen Benchmark (easy). The shaded area indicates one standard deviation around the mean. A ★ indicates statistically significant improvement over the uniform-sampling baseline by the PLR-based method of the matching color ($p < 0.05$). PLR tends to improve or match training sample efficiency. Note that while PLR reduces training performance on StarPilot, it performs comparably to the uniform-sampling baseline at test time, indicating less overfitting to training levels.

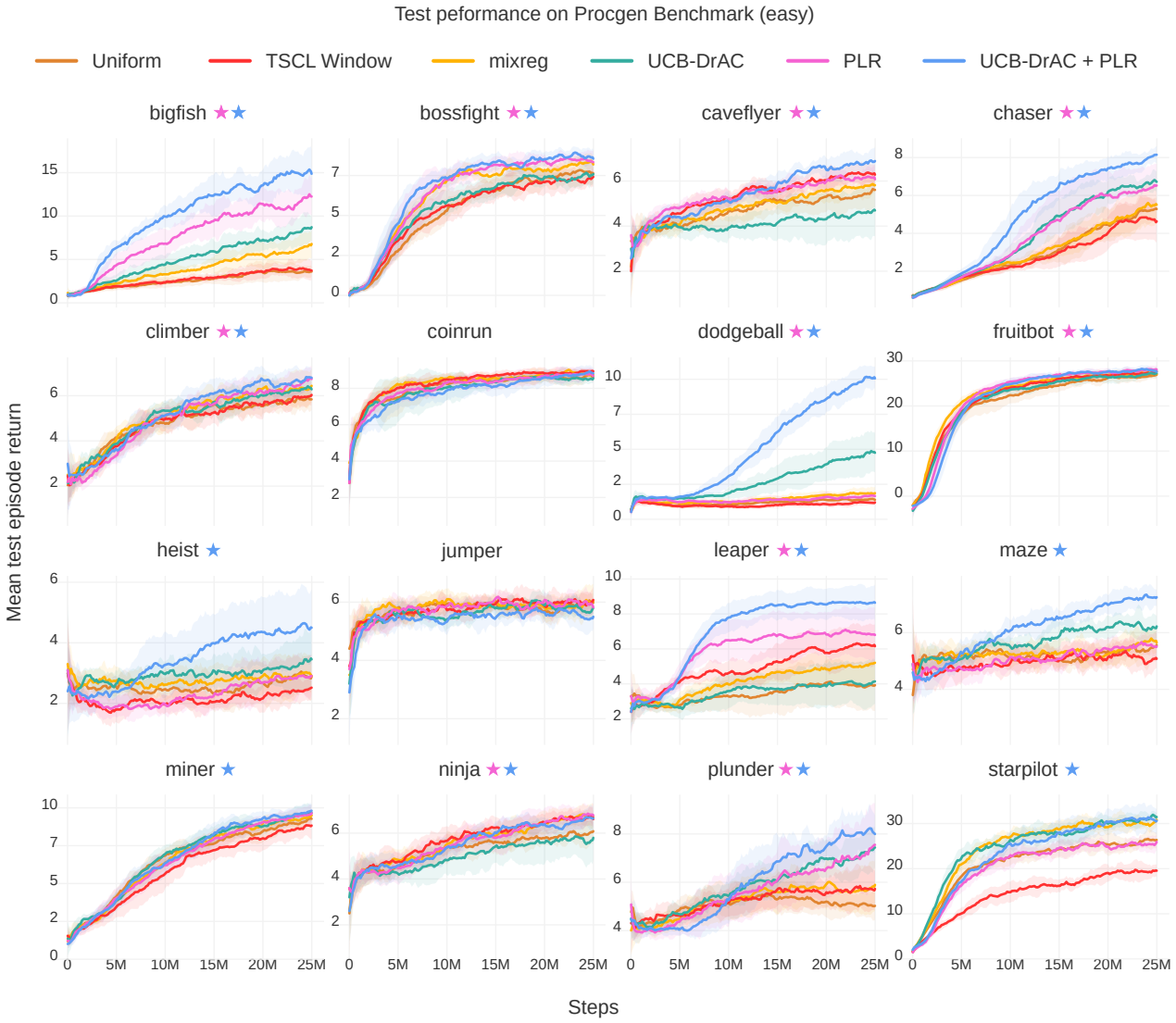


Figure 14. Mean test episode return (10 runs) on each Procgen Benchmark game (easy). The shaded area indicates one standard deviation around the mean. PLR-based methods consistently match or outperform uniform sampling with statistically significance ($p < 0.05$), indicated by a ★ of the corresponding color. We see that TSCL results in inconsistent outcomes across games, notably drastically lower test returns on StarPilot.

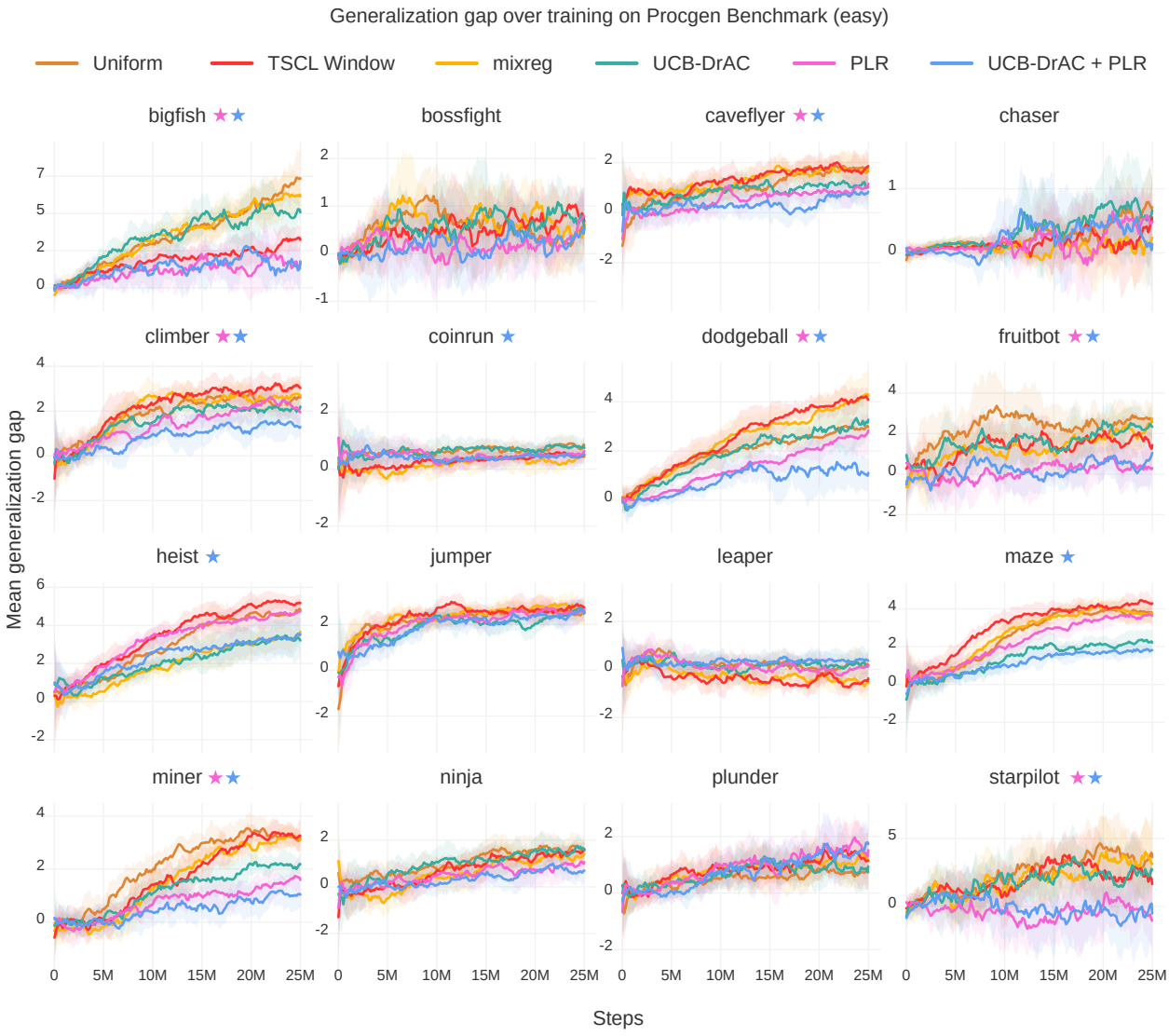


Figure 15. Mean generalization gaps throughout training (10 runs) on each Procgen Benchmark game (easy). The shaded area indicates one standard deviation around the mean. A ★ indicates the method of matching color results in a statistically significant ($p < 0.05$) reduction in generalization gap compared to the uniform-sampling baseline. By itself, PLR significantly reduces the generalization gap on 7 games, and UCB-DrAC, on 5 games. This number jumps to 10 of 16 games when these two methods are combined. TSCL only significantly reduces generalization gap on 2 of 16 games relative to uniform sampling, while increasing it on others, most notably on Dodgeball.

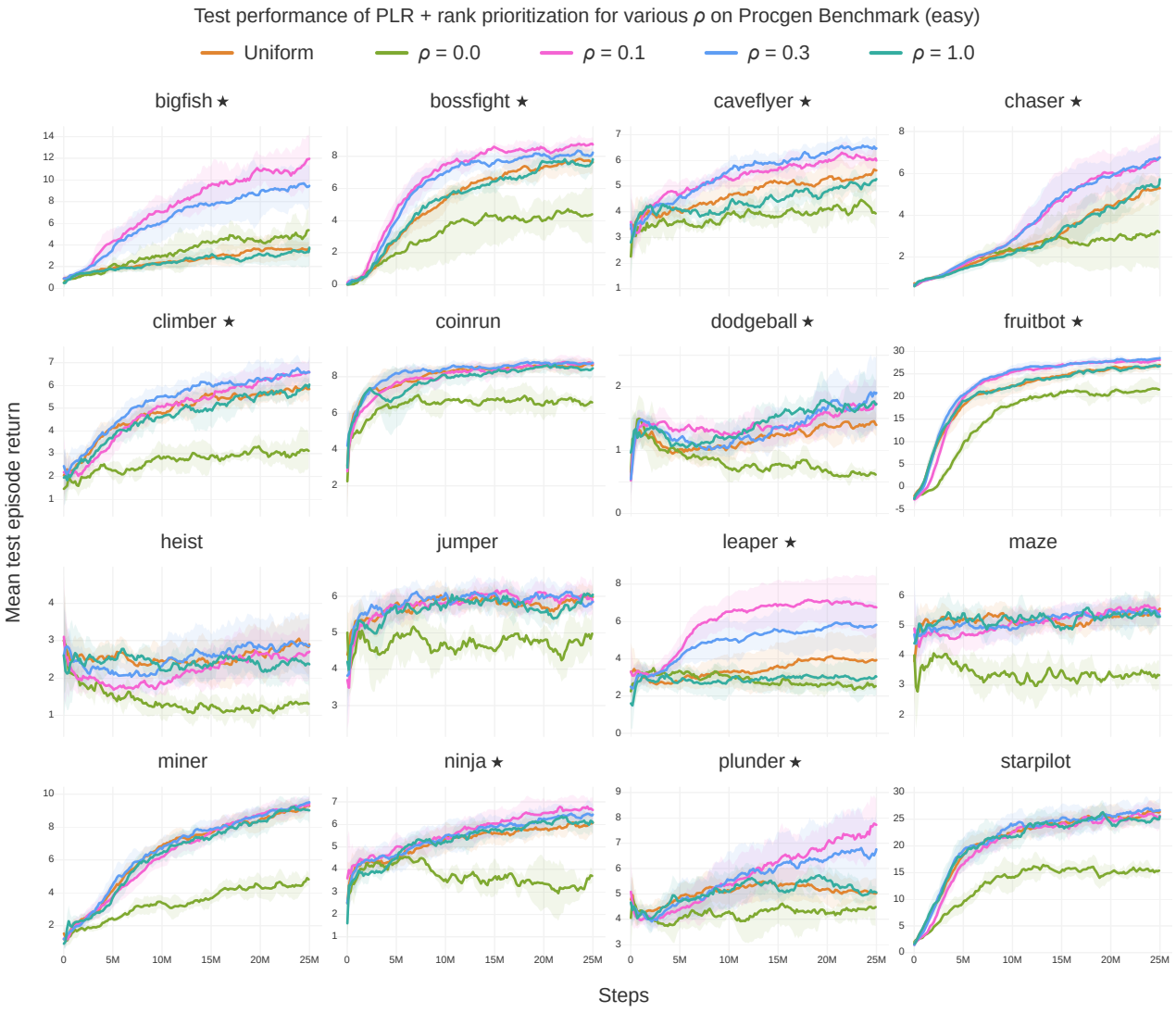


Figure 16. Mean test episode returns (10 runs) on the Procgen Benchmark (easy) for PLR with rank prioritization and $\beta = 0.1$ across a range of staleness coefficient values, ρ. The replay distribution must consider both the L1 value-loss and staleness values to realize improvements to generalization and sample efficiency. The shaded area indicates one standard deviation around the mean. A ★ next to the game name indicates that $\rho = 0.1$ exhibits statistically significantly better final test returns or sample efficiency along the test curve ($p < 0.05$), which we observe in 10 of 16 games.

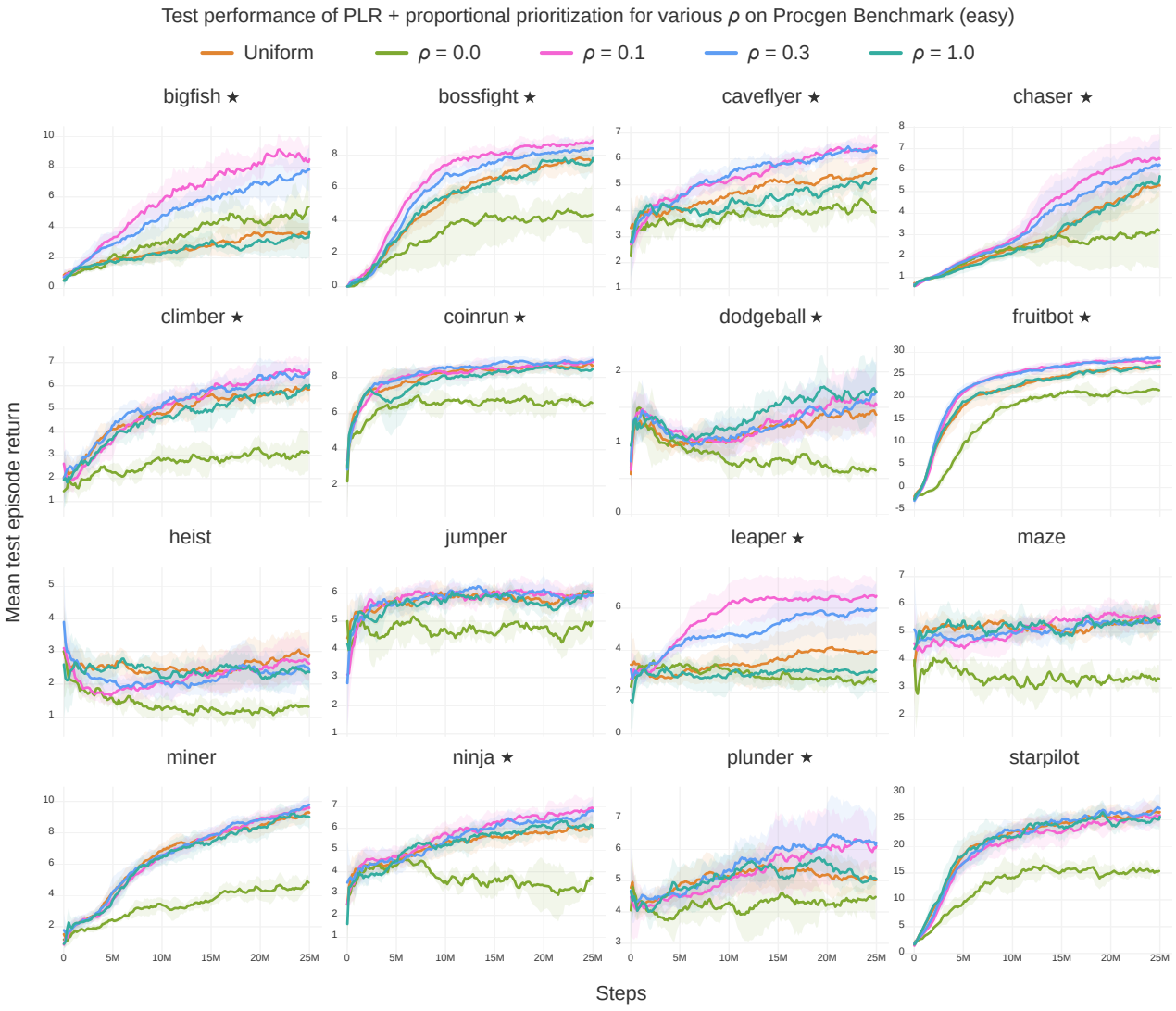


Figure 17. Mean test episode returns (10 runs) on the Procgen Benchmark (easy) for PLR with proportional prioritization and $\beta = 0.1$ across a range of values of ρ. As in the case of rank prioritization, the replay distribution must consider both the L1 value loss score and staleness values in order to realize performance improvements. The shaded area indicates one standard deviation around the mean. A ★ next to the game name indicates the condition $\rho = 0.1$ exhibits statistically significantly better final test returns or sample efficiency along the test curve ($p < 0.05$), which we observe in 11 of 16 games.

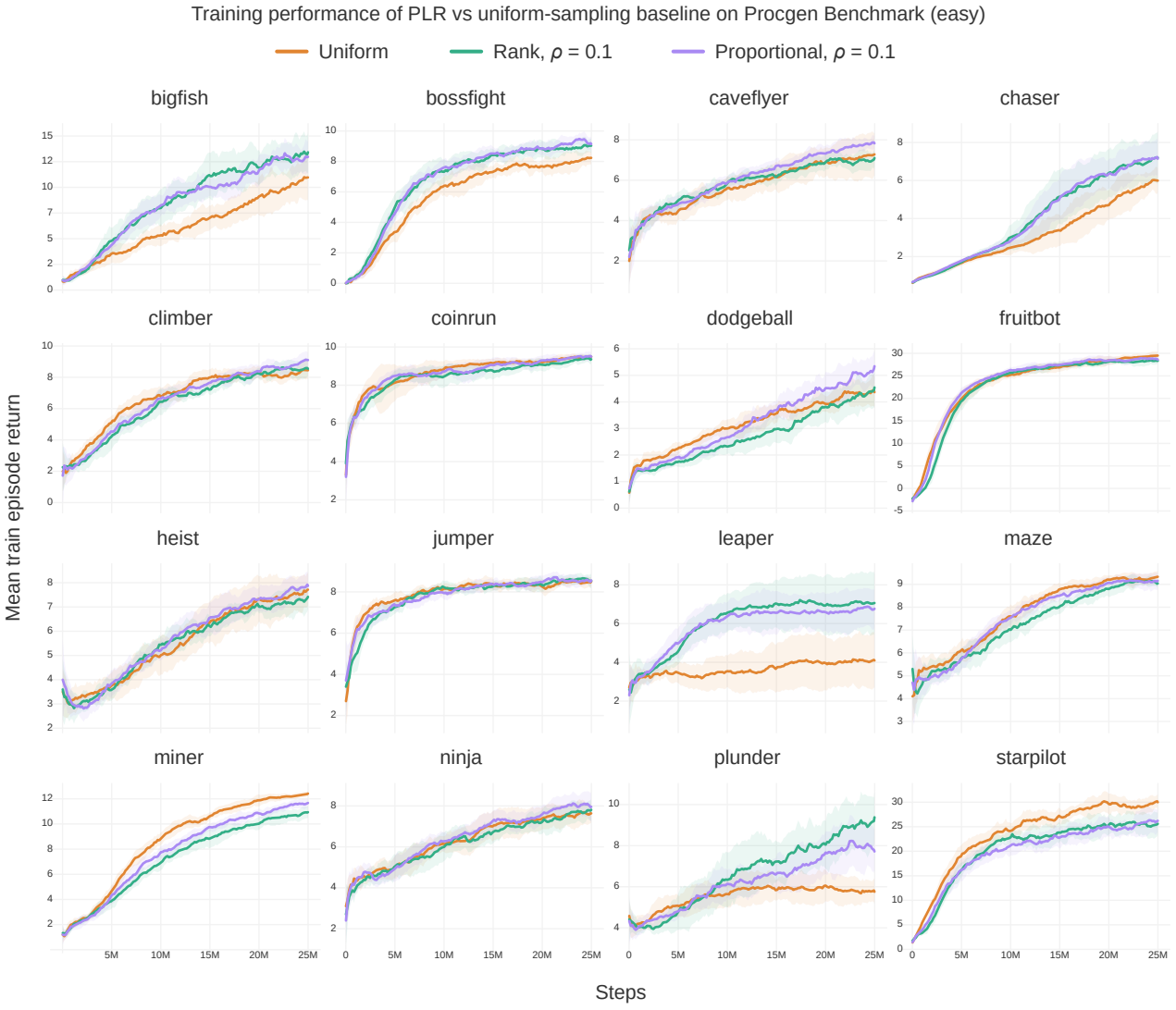


Figure 18. Mean training episode returns (10 runs) on the Procgen Benchmark for (easy) PLR with $\beta = 0.1$, $\rho = 0.1$, and each of rank and proportional prioritization. On some games, PLR improves both training sample efficiency and generalization performance (e.g. BigFish and Chaser), while on others, only generalization performance (e.g. CaveFlyer with rank prioritization). The shaded area indicates one standard deviation around the mean.

Algorithm 3 Generic T-step policy-gradient training loop with prioritized level replay

Input: Training levels Λ_{train} of an environment, policy π_θ, rollout length T, number of updates N_u, batch size N_b, policy update function $\mathcal{U}(\mathcal{B}, \theta) \rightarrow \theta'$.

Initialize level scores S, partial level scores $\tilde{S}$, and level timestamps C
Initialize global episode count $c \leftarrow 0$
Initialize set of visited levels $\Lambda_{\text{seen}} = \varnothing$
Initialize experience buffer $\mathcal{B} = \varnothing$
Initialize N_b parallel environment instances E, each set to a random level in $\in \Lambda_{\text{train}}$
for $u = 1$ **to** N_u **do**
  $\mathcal{B} \leftarrow \varnothing$
  for $k = 1$ **to** N_b **do**
    $\mathcal{B} \leftarrow \mathcal{B} \cup$ **collect_experiences**$(k, E, \Lambda_{\text{train}}, \Lambda_{\text{seen}}, \pi_\theta, T, S, \tilde{S}, C, c)$ *Using Algorithm 4*
  end for
  $\theta \leftarrow \mathcal{U}(\mathcal{B}, \theta)$
end for

Algorithm 4 Collect T-step rollouts with prioritized level replay

Input: Actor index k, batch environments E, training levels Λ_{train}, visited levels Λ_{seen}, current level l, policy π_θ, rollout length T, scoring function $\mathbf{score}$, level scores S, partial scores $\tilde{S}$, staleness values C, and global episode count c.
Output: Experience buffer $\mathcal{B}$

Initialize $\mathcal{B} = \varnothing$, and set current level $l_i = E_k$
Observe current state s_0, termination flag d_0
if d_0 **then**
  Define new index $i \leftarrow |S| + 1$
  Choose current level $l_i \leftarrow \mathbf{sampleNextLevel}(\Lambda_{\text{train}}, S, C, c)$ and $E_k \leftarrow l_i$
  Update level timestamp $C_i \leftarrow c$
  Observe initial state s_0
end if
Choose $a_0 \sim \pi_\theta(\cdot|s_0)$
t = 1
Initialize episodic trajectory buffer $\tau = \varnothing$
while $t < T$ **do**
  Observe (s_t, r_t, d_t)
  $\mathcal{B} \leftarrow \mathcal{B} \cup (s_{t-1}, a_{t-1}, s_t, r_t, d_t, \log \pi_\theta(a))$
  $\tau \leftarrow \tau \cup (s_{t-1}, a_{t-1}, s_t, r_t, d_t, \log \pi_\theta(a))$
  if d_t **then**
    Update level score $S_i \leftarrow \mathbf{score}(\tau, \pi_\theta, \tilde{S}_i)$ and partial score $\tilde{S}_i \leftarrow 0$
    $\tau \leftarrow \varnothing$
    Define new index $i \leftarrow |S| + 1$
    Update current level $l_i \leftarrow$ **sampleNextLevel**$(\Lambda_{\text{train}}, S, C, c)$ and $E_k \leftarrow l_i$
    Update level timestamp $C_i \leftarrow c$
  end if
  Choose $a_{t+1} \sim \pi_\theta(\cdot|s_t)$
  $t \leftarrow t + 1$
end while
if not d_t **then**
  $\tilde{S}_i \leftarrow (\textbf{score}(\tau, \pi_\theta), |\tau|)$ *Track partial time-averaged score and $|\tau|$*
end if

function sampleNextLevel$(\Lambda_{\text{train}}, S, C, c)$
  $c \leftarrow c + 1$
  Sample replay decision $d \sim P_D(d)$
  if $d = 0$ **and** $|\Lambda_{\text{train}} \setminus \Lambda_{\text{seen}}| > 0$ **then**
    Define new index $i \leftarrow |S| + 1$
    Sample $l_i \sim P_{\text{new}}(l|\Lambda_{\text{train}}, \Lambda_{\text{seen}})$ *Sample an unseen level, if any*
    Add l_i to Λ_{seen}, add initial value $S_i = 0$ to S and $C_i = 0$ to C
  else
    Sample $l_i \sim (1 - \rho) \cdot P_S(l|\Lambda_{\text{seen}}, S) + \rho \cdot P_C(l|\Lambda_{\text{seen}}, C, c)$ *Sample a level for replay*
  end if
  return l_i
end function

PySCF: The Python-based Simulations of Chemistry Framework

Qiming Sun*[1], Timothy C. Berkelbach[2], Nick S. Blunt[3,4], George H. Booth[5], Sheng Guo[1,6], Zhendong Li[1], Junzi Liu[7], James D. McClain[1,6], Elvira R. Sayfutyarova[1,6], Sandeep Sharma[8], Sebastian Wouters[9], and Garnet Kin-Lic Chan†[1]

[1]Division of Chemistry and Chemical Engineering, California Institute of Technology, Pasadena CA 91125, USA
[2]Department of Chemistry and James Franck Institute, University of Chicago, Chicago, Illinois 60637, USA
[3]Chemical Science Division, Lawrence Berkeley National Laboratory, Berkeley, California 94720, USA
[4]Department of Chemistry, University of California, Berkeley, California 94720, USA
[5]Department of Physics, King's College London, Strand, London WC2R 2LS, United Kingdom
[6]Department of Chemistry, Princeton University, Princeton, New Jersey 08544, USA
[7]Institute of Chemistry Chinese Academy of Sciences, Beijing 100190, P. R. China
[8]Department of Chemistry and Biochemistry, University of Colorado Boulder, Boulder, CO 80302, USA
[9]Brantsandpatents, Pauline van Pottelsberghelaan 24, 9051 Sint-Denijs-Westrem, Belgium

Abstract

PySCF is a general-purpose electronic structure platform designed from the ground up to emphasize code simplicity, so as to facilitate new method development and enable flexible

*osirpt.sun@gmail.com
†gkc1000@gmail.com

computational workflows. The package provides a wide range of tools to support simulations of finite-size systems, extended systems with periodic boundary conditions, low-dimensional periodic systems, and custom Hamiltonians, using mean-field and post-mean-field methods with standard Gaussian basis functions. To ensure ease of extensibility, PySCF uses the Python language to implement almost all of its features, while computationally critical paths are implemented with heavily optimized C routines. Using this combined Python/C implementation, the package is as efficient as the best existing C or Fortran based quantum chemistry programs. In this paper we document the capabilities and design philosophy of the current version of the PySCF package.

1 INTRODUCTION

The Python programming language is playing an increasingly important role in scientific computing. As a high level language, Python supports rapid development practices and easy program maintenance. While programming productivity is hard to measure, it is commonly thought that it is more efficient to prototype new ideas in Python, rather than in traditional low-level compiled languages such as Fortran or C/C++. Further, through the use of the many high-quality numerical libraries available in Python – such as NumPy[1], SciPy[2], and MPI4Py[3] – Python programs can perform at competitive levels with optimized Fortran and C/C++ programs, including on large-scale computing architectures.

There have been several efforts in the past to incorporate Python into electronic structure programs. Python has been widely adopted in a scripting role: the Psi4[4] quantum chemistry package uses a custom "Psithon" dialect to drive the underlying C++ implementation, while general simulation environments such as ASE[5] and PyMatGen[6] provide Python frontends to multiple quantum chemistry and electronic structure packages, to organize complex workflows[7]. Python has also proved popular for implementing symbolic second-quantized algebra and code generation tools, such as the Tensor Contraction Engine[8] and the SecondQuantizationAlgebra library[9,10].

In the above cases, Python has been used as a supporting language, with the underlying quantum chemistry algorithms implemented in a compiled language. However, Python has also seen some use as a primary implementation language for electronic structure methods. PyQuante[11] was an early attempt to implement a Gaussian-based quantum chemistry code in Python, although it did not achieve speed or functionality competitive with typical packages. Another early effort was the GPAW[12] code, which implements the projector augmented wave formalism for density functional theory, and which is still under active development in multiple groups. Nonetheless, it is probably fair to say that using Python as an implementation language, rather than a supporting language, remains the exception rather than the rule in modern quantum chemistry and electronic structure software efforts.

With the aim of developing a new highly functional, high-performance computing toolbox for the quantum chemistry of molecules and materials implemented primarily in the

Python language, we started the open-source project "Python-based Simulations of Chemistry Framework" (PySCF) in 2014. The program was initially ported from our quantum chemistry density matrix embedding theory (DMET) project[13] and contained only the Gaussian integral interface, a basic Hartree-Fock solver, and a few post-Hartree-Fock components required by DMET. In the next 18 months, multi-configurational self-consistent-field (MCSCF), density functional theory and coupled cluster theory, as well as relevant modules for molecular properties, were added into the package. In 2015, we released the first stable version, PySCF 1.0, wherein we codified our primary goals for further code development: to produce a package that emphasizes simplicity, generality, and efficiency, in that order. As a result of this choice, most functions in PySCF are written purely in Python, with a very limited amount of C code only for the most time-critical parts. The various features and APIs are designed and implemented in the simplest and most straightforward manner, so that users can easily modify the source code to meet their own scientific needs and workflow. However, although we have favored algorithm accessibility and extensibility over performance, we have found that the efficient use of numerical Python libraries allows PySCF to perform at least as fast as the best existing quantum chemistry implementations. In this article, we highlight the current capabilities and design philosophy of the PySCF package.

2 CAPABILITIES

Molecular electronic structure methods are typically the main focus of quantum chemistry packages. We have put significant effort towards the production of a stable, feature-rich and efficient molecular simulation environment in PySCF. In addition to molecular quantum chemistry methods, PySCF also provides a wide range of quantum chemistry methods for extended systems with periodic boundary conditions (PBC). Table 1 lists the main electronic structure methods available in the PySCF package. More detailed descriptions are presented in Section 2.1 - Section 2.7. Although not listed in the table, many auxiliary tools for method development are also part of the package. They are briefly documented in Section 2.8 - Section 2.13.

2.1 Self-consistent field methods

Self-consistent field (SCF) methods are the starting point for most electronic structure calculations. In PySCF, the SCF module includes implementations of Hartree-Fock (HF) and density functional theory (DFT) for restricted, unrestricted, closed-shell and open-shell Slater determinant references. A wide range of predefined exchange-correlation (XC) functionals are supported through a general interface to the Libxc[14] and Xcfun[15] functional libraries. Using the interface, as shown in Figure 1, one can easily customize the XC functionals in DFT calculations. PySCF uses the Libcint[16] Gaussian integral library, written by one of us (QS) as its integral engine. In its current implementation, the SCF program can handle over 5000 basis functions on a single symmetric multiprocessing (SMP) node without any approximations to the integrals. To obtain rapid convergence in the SCF iterations, we have also developed a second order co-iterative augmented Hessian (CIAH) algorithm for orbital optimization[17]. Using the direct SCF technique with the CIAH algorithm, we are able to converge a Hartree-Fock calculation for the open-shell molecule Fe(II)-porphine (2997 AOs) on a 16-core node in one day.

2.2 Post SCF methods

Single-reference correlation methods can be used on top of the HF or DFT references, including Møller-Plesset second-order perturbation theory (MP2), configuration interaction, and coupled cluster theory.

Canonical single-reference coupled cluster theory has been implemented with single and double excitations (CCSD)[18] and with perturbative triples [CCSD(T)]. The associated derivative routines include CCSD and CCSD(T) density matrices, CCSD and CCSD(T) analytic gradients, and equation-of-motion CCSD for the ionization potentials, electron affinities, and excitation energies (EOM-IP/EA/EE-CCSD)[19–21]. The package contains two complementary implementations of each of these methods. The first set are straightforward spin-orbital and spatial-orbital implementations, which are optimized for readability and written in pure Python using syntax of the NumPy `einsum` function (which can use either the default Numpy implementation or a custom `gemm`-based version) for tensor contraction. These im-

plementations are easy for the user to modify. A second spatial-orbital implementation has been intensively optimized to minimize dataflow and uses asynchronous I/O and a threaded `gemm` function for efficient tensor contractions. For a system of 25 occupied orbitals and 1500 virtual orbitals (H_{50} with cc-pVQZ basis), the latter CCSD implementation takes less than 3 hours to finish one iteration using 28 CPU cores.

The configuration interaction code implements two solvers: a solver for configuration interaction with single and double excitations (CISD), and a determinant-based full configuration interaction (FCI) solver[22] for fermion, boson or coupled fermion-boson Hamiltonians. The CISD solver has a similar program layout to the CCSD solver. The FCI solver additionally implements the spin-squared operator, second quantized creation and annihilation operators (from which arbitrary second quantized algebra can be implemented), functions to evaluate the density matrices and transition density matrices (up to fourth order), as well as a function to evaluate the overlap of two FCI wavefunctions in different orbital bases. The FCI solver is intensively optimized for multi-threaded performance. It can perform one matrix-vector operation for 16 electrons and 16 orbitals using 16 CPU cores in 30 seconds.

2.3 Multireference methods

For multireference problems, the PySCF package provides the complete active space self consistent field (CASSCF) method[23,24] and N-electron valence perturbation theory (NEVPT2)[25,26]. When the size of the active space exceeds the capabilities of the conventional FCI solver, one can switch to external variational solvers such as a density matrix renormalization group (DMRG) program[27–29] or a full configuration interaction quantum Monte Carlo (FCIQMC) program[30,31]. Incorporating external solvers into the CASSCF optimizer widens the range of possible applications, while raising new challenges for an efficient CASSCF algorithm. One challenge is the communication between the external solver and the orbital optimization driver; communication must be limited to quantities that are easy to obtain from the external solver. A second challenge is the cost of handling quantities associated with the active space; for example, as the active space becomes large, the memory required to hold integrals involving active labels can easily exceed available memory. Finally, any approximations introduced in the context of the above two challenges should not interfere with the

quality of convergence of the CASSCF optimizer.

To address these challenges, we have implemented a general AO-driven CASSCF optimizer[29] that provides second order convergence and which may easily be combined with a wide variety of external variational solvers, including DMRG, FCIQMC and their state-averaged solvers. Only the 2-particle density matrix and Hamiltonian integrals are communicated between the CASSCF driver and the external CI solver. Further, the AO-driven algorithm has a low memory and I/O footprint. The current implementation supports calculations with 3000 basis functions and 30–50 active orbitals on a single SMP node with 128 GB memory, without any approximations to the AO integrals.

A simple interface is provided to use an external solver in multiconfigurational calculations. Figure 2 shows how to perform a DMRG-CASSCF calculation by replacing the `fcisolver` attribute of the CASSCF method. DMRG-SC-NEVPT2[26], and ic-MPS-PT2 and ic-MPS-LCC[32] methods are also available through the interface to the DMRG program package Block[27,33–35], and the ic-MPS-LCC program of Sharma[32].

2.4 Molecular properties

At the present stage, the program can compute molecular properties such as analytic nuclear gradients, analytic nuclear Hessians, and NMR shielding parameters at the SCF level. The CCSD and CCSD(T) modules include solvers for the Λ-equations. As a result, we also provide one-particle and two-particle density matrices, as well as the analytic nuclear gradients, for the CCSD and CCSD(T) methods[36].

For excited states, time-dependent HF (TDHF) and time-dependent DFT (TDDFT) are implemented on top of the SCF module. The relevant analytic nuclear gradients are also programmed[37]. The CCSD module offers another option to obtain excited states using the EOM-IP/EA/EE-CCSD methods. The third option to obtain excited states is through the multi-root CASCI/CASSCF solvers, optionally followed by the MRPT tool chain. Starting from the multi-root CASCI/CASSCF solutions, the program can compute the density matrices of all the states and the transition density matrices between any two states. One can contract these density matrices with specific AO integrals to obtain different first order molecular properties.

2.5 Relativistic effects

Many different relativistic treatments are available in PySCF. Scalar relativistic effects can be added to all SCF and post-SCF methods through relativistic effective core potentials (ECP)[38] or the all-electron spin-free X2C[39] relativistic correction. For a more advanced treatment, PySCF also provides 4-component relativistic Hartree-Fock and no-pair MP2 methods with Dirac-Coulomb, Dirac-Coulomb-Gaunt, and Dirac-Coulomb-Breit Hamiltonians. Although not programmed as a standalone module, the no-pair CCSD electron correlation energy can also be computed with the straightforward spin-orbital version of the CCSD program. Using the 4-component Hamiltonian, molecular properties including analytic nuclear gradients and NMR shielding parameters are available at the mean-field level[40].

2.6 Orbital localizer and result analysis

Two classes of orbital localization methods are available in the package. The first emphasizes the atomic character of the basis functions. The relevant localization functions can generate intrinsic atomic orbitals (IAO)[41], natural atomic orbitals (NAO)[42], and meta-Löwdin orbitals[13] based on orbital projection and orthogonalization. With these AO-based local orbitals, charge distributions can be properly assigned to atoms in population analysis[41]. In the PySCF population analysis code, meta-Löwdin orbitals are the default choice.

The second class, represented by Boys-Foster, Edmiston-Ruedenberg, and Pipek-Mezey localization, require minimizing (or maximizing) the dipole, the Coulomb self-energy, or the atomic charges, to obtain the optimal localized orbitals. The localization routines can take arbitrary orthogonal input orbitals and call the CIAH algorithm to rapidly converge the solution. For example, using 16 CPU cores, it takes 3 minutes to localize 1620 HF unoccupied orbitals for the C_{60} molecule using Boys localization.

A common task when analysing the results of an electronic structure calculation is to visualize the orbitals. Although PySCF does not have a visualization tool itself, it provides a module to convert the given orbital coefficients to the `molden`[43] format which can be read and visualized by other software, e.g. Jmol[44]. Figure 3 is an example to run Boys localization for the C_{60} HF occupied orbitals and to generate the orbital surfaces of the

localized σ-bond orbital in a single Python script.

2.7 Extended systems with periodic boundary conditions

PBC implementations typically use either plane waves[45–48] or local atomic functions[12,49–53] as the underlying orbital basis. The PBC implementation in PySCF uses the local basis formulation, specifically crystalline orbital Gaussian basis functions ϕ, expanded in terms of a lattice sum over local Gaussians χ

$$\phi_{\boldsymbol{k},\chi}(\boldsymbol{r}) = \sum_{\boldsymbol{T}} e^{i\boldsymbol{k}\cdot\boldsymbol{T}}\chi(\boldsymbol{r}-\boldsymbol{T})$$

where $\boldsymbol{k}$ is a vector in the first Brillouin zone and $\boldsymbol{T}$ is a lattice translational vector. We use a pure Gaussian basis in our PBC implementation for two reasons: to simplify the development of post-mean-field methods for extended systems and to have a seamless interface and direct comparability to finite-sized quantum chemistry calculations. Local bases are favourable for post-mean-field methods because they are generally quite compact, resulting in small virtual spaces[54], and further allow locality to be exploited. Due to the use of local bases, various boundary conditions can be easily applied in the PBC module, from zero-dimensional systems (molecules) to extended one-, two- and three-dimensional periodic systems.

The PBC module supports both all-electron and pseudopotential calculations. Both separable pseudopotentials (e.g. Goedecker-Teter-Hutter (GTH) pseudopotentials[55,56]) and non-separable pseudopotentials (quantum chemistry ECPs and Burkatzi-Filippi-Dolg pseudopotentials[57]) can be used. In the separable pseudopotential implementation, the associated orbitals and densities are guaranteed to be smooth, allowing a grid-based treatment that uses discrete fast Fourier transforms[53,58]. In both the pseudopotential and all-electron PBC calculations, Coulomb-based integrals are handled via density fitting as described in Section 2.10.

The PBC implementation is organized in direct correspondence to the molecular implementation. We implemented the same function interfaces as in the molecular code, with analogous module and function names. Consequently, methods defined in the molecular part of the code can be seamlessly mixed with the PBC functions without modification, especially in Γ-point calculations where the PBC wave functions are real. Thus, starting from

PBC Γ-point mean-field orbitals, one can, for example, carry out CCSD, CASSCF, TDDFT, etc. calculations using the molecular implementations. We also introduce specializations of the PBC methods to support k-point (Brillouin zone) sampling. The k-point methods slightly modify the Γ-point data structures, but inherit from and reuse almost all of the Γ-point functionality. Explicit k-point sampling is supported at the HF and DFT level, and on top of this we have also implemented k-point MP2, CCSD, CCSD(T) and EOM-CCSD methods[58], with optimizations to carefully distribute work and data across cores. On 100 computational cores, mean-field simulations including unit cells with over 100 atoms, or k-point CCSD calculations with over 3000 orbitals, can be executed without difficulty.

2.8 General AO integral evaluator and J/K builds

Integral evaluation forms the foundation of Gaussian-based electronic structure simulation. The general integral evaluator library Libcint supports a wide range of GTO integrals, and PySCF exposes simple APIs to access the Libcint integral functions. As the examples in Figure 4 show, the PySCF integral API allows users to access AO integrals either in a giant array or in individual shells with a single line of Python code. The integrals provided include,

- integrals in the basis of Cartesian, real-spherical and j-adapted spinor GTOs;
- arbitrary integral expressions built from $\boldsymbol{r}$, $\boldsymbol{p}$, and σ polynomials;
- 2-center, 3-center and 4-center 2-electron integrals for the Coulomb operator $1/r_{12}$, range-separated Coulomb operator $\mathrm{erf}(\omega r_{12})/r_{12}$, Gaunt interaction, and Breit interaction.

Using the general AO integral evaluator, the package provides a general AO-driven J/K contraction function. J/K-matrix construction involves a contraction over a high order tensor (e.g. 4-index 2-electron integrals $(ij|kl)$) and a low order tensor (e.g. the 2-index density matrix γ)

$$J_{ij} = \sum_{kl} (ij|kl)\gamma_{kl}$$

$$K_{il} = \sum_{jk} (ij|kl)\gamma_{jk}$$

When both tensors can be held in memory, the NUMPY package offers a convenient tensor contraction function `einsum` to quickly construct J/K matrices. However, it is common for the high order tensor to be too large to fit into the available memory. Using the Einstein summation notation of the NUMPY `einsum` function, our AO-driven J/K contraction implementation offers the capability to contract the high order tensor (e.g. 2-electron integrals or their high order derivatives) with multiple density matrices, with a small memory footprint. The J/K contraction function also supports subsystem contraction, in which the 4 indices of the 2-electron integrals are distributed over different segments of the system which may or may not overlap with each other. This subsystem contraction is particularly useful in two scenarios: in fragment-based methods, where the evaluation of Coulomb or exchange energies involves integral contraction over different fragments, and in parallel algorithms, where one partitions the J/K contraction into small segments and distributes them to different computing nodes.

2.9 General integral transformations

Integral transformations are another fundamental operation found in quantum chemistry programs. A common kind of integral transformation is to transform the 4 indices of the 2-electron integrals by 4 sets of different orbitals. To satisfy this need, we designed a general integral transformation function to handle the arbitrary AO integrals provided by the LIBCINT library and arbitrary kinds of orbitals. To reduce disk usage, we use permutation symmetry over i and j, k and l in $(ij|kl)$ whenever possible for real integrals.

Integral transformations involve high computational and I/O costs. A standard approach to reduce these costs involves precomputation to reduce integral costs and data compression to increase I/O throughput. However, we have not adopted such an optimization strategy in our implementation because it is against the objective of simplicity for the PYSCF package. In our implementation, initialization is not required for the general integral transformation function. Similarly to the AO integral API, the integral transformation can thus be launched with one line of Python code. In the integral data structure, we store the transformed inte-

grals by chunks in the HDF5 format without compression. This choice has two advantages. First, it allows for fast indexing and hyperslab selection for subblocks of the integral array. Second, the integral data can be easily accessed by other program packages without any overhead for parsing the integral storage protocol.

2.10 Density fitting

The density fitting (DF) technique is implemented for both finite-sized systems and crystalline systems with periodic boundary conditions.

In finite-sized systems, one can use DF to approximate the J/K matrix and the MO integrals for the HF, DFT and MP2 methods. To improve the performance of the CIAH algorithm, one can use the DF orbital Hessian in the CIAH orbital optimization for Edmiston-Ruedenberg localization and for the HF, DFT and CASSCF algorithms.

In the PBC module, the 2-electron integrals are represented as the product of two 3-index tensors which are treated as DF objects. Based on the requirements of the system being modelled, we have developed various DF representations. When the calculation involves only smooth bases (typically with pseudopotentials), plane waves are used as the auxiliary fitting functions and the DF 3-index tensor is computed within a grid-based treatment using discrete fast Fourier transforms[58]. When high accuracy in all-electron calculations is required, a mixed density fitting technique is invoked in which the fitting functions are Gaussian functions plus plane waves. Besides the choice of fitting basis, different metrics (e.g. overlap, kinetic, or Coulomb) can be used in the fitting to balance performance against computational accuracy.

The 3-index DF tensor is stored as a giant array in the HDF5 format without compression. With this design, it is straightforward to access the 2-electron integrals with the functions of the PySCF package. Moreover, it allows us to supply 2-electron integrals to calculations by overloading the DF object in cases where direct storage of the 4-index integrals in memory or on disk is infeasible (see discussion in Section 2.11).

2.11 Custom Hamiltonians

Most quantum chemistry approximations are not tied to the details of the ab initio molecular or periodic Hamiltonian. This means that they can also be used with arbitrary model Hamiltonians, which is of interest for semi-empirical quantum chemistry calculations as well as condensed-matter model studies. In PySCF, overwriting the predefined Hamiltonian is straightforward. The Hamiltonian is an attribute of the mean-field calculation object. Once the 1-particle and 2-particle integral attributes of the mean-field object are defined, they are used by the mean-field calculation and all subsequent post-Hartree-Fock correlation treatments. Users can thus carry out correlated calculations with model Hamiltonians in exactly the same way as with standard ab initio Hamiltonians. Figure 5 displays an example of how to input a model Hamiltonian.

2.12 Interfaces to external programs

PySCF can be used either as the driver to execute external programs or as an independent solver to use as part of a computational workflow involving other software. In PySCF, the DMRG programs Block[27] and CheMPS2[28,59] and the FCIQMC program NECI[60] can be used as a replacement for the FCI routine for large active spaces in the CASCI/CASSCF solver. In the QM/MM interface, by supplying the charges and the positions of the MM atoms, one can compute the HF, DFT, MP2, CC, CI and MCSCF energies and their analytic nuclear gradients.

To communicate with other quantum chemistry programs, we provide utility functions to read and write Hamiltonians in the Molpro[61] `FCIDUMP` format, and arbitrary orbitals in the `molden`[43] format. The program also supports to write SCF solution and CI wavefunction in the GAMESS[62] `WFN` format and to read orbitals from Molpro XML output. The real space electron density can be output on cubic grids in the Gaussian[63] `cube` format.

2.13 Numerical tools

Although the Numpy and Scipy libraries provide a wide range of numerical tools for scientific computing, there are some numerical components commonly found in quantum chem-

istry algorithms that are not provided by these libraries. For example, the direct inversion of the iterative space (DIIS) method[64,65] is one of the most commonly used tools in quantum chemistry to speed up optimizations when a second order algorithm is not available. In PySCF we provide a general DIIS handler for an object array of arbitrary size and arbitrary data type. In the current implementation, it supports DIIS optimization both with or without supplying the error vectors. For the latter case, the differences between the arrays of adjacent iterations are minimized. Large scale eigenvalue and linear equation solvers are also common components of many quantum chemistry methods. The Davidson diagonalization algorithm and Arnoldi/Krylov subspace solver are accessible in PySCF through simple APIs.

3 Design and implementation of PySCF

While we have tried to provide rich functionality for quantum chemical simulations with the built-in functions of the PySCF package, it will nonetheless often be the case that a user's needs are not covered by the built-in functionality. A major design goal has been to implement PySCF in a sufficiently flexible way so that users can easily extend its functionality. To provide robust components for complex problems and non-trivial workflows, we have made the following general design choices in PySCF:

1. *Language*: Mostly Python, with a little C. We believe that it is easiest to develop and test new functionality in Python. For this reason, most functions in PySCF are written in pure Python. Only a few computational hot spots have been rewritten and optimized in C.

2. *Style*: Mostly functional, with a little object-oriented programming (OOP). Although OOP is a successful and widely used programming paradigm, we feel that it is hard for users to customize typical OOP programs without learning details of the object hierarchy and interfaces. We have adopted a functional programming style, where most functions are pure, and thus can be invoked alone and independently of each other. This allows users to mix functionality with a minimal knowledge of the PySCF

internals.

We elaborate on these choices below.

3.1 Input language

Almost every quantum chemistry package today uses its own custom input language. This is a burden to the user, who must become familiar with a new domain-specific language for every new package. In contrast, PySCF does not have an input language. Rather, the functionality is simply called from an input script written in the host Python language. This choice has clear benefits:

1. *There is no need to learn a domain-specific language.* Python, as a general programming language, is already widely used for numerical computing, and is taught in modern computer science courses. For novices, the language is easy to learn and help is readily available from the large Python community.

2. *One can use all Python language features in the input script.* This allows the input script to implement complex logic and computational workflows, and to carry out tasks (e.g. data processing and plotting) in the same script as the electronic structure simulation (see Figure 6 for an example).

3. *The computational environment is easily extended beyond that provided by the* PySCF *package.* The PySCF package is a regular Python module which can be mixed and matched with other Python modules to build a personalized computing environment.

4. *Computing can be carried out interactively.* Simulations can be tested, debugged, and executed step by step within the Python interpreter shell.

3.2 Enabling interactive computing

As discussed above, a strength of the PySCF package is that its functionality can be invoked from the interactive Python shell. However, maximizing its usability in this interactive mode entails additional design optimizations. There are three critical considerations to facilitate such interactive computations:

1. The functions and data need to be easy to access;

2. Functions should be insensitive to execution order (when and how many times a function is called should not affect the result);

3. Computations should not cause (significant) halts in the interactive shell.

To address these requirements, we have enforced the following design rules wherever possible in the package:

1. Functions are pure (i.e. state free). This ensures that they are insensitive to execution order;

2. Method objects (classes) only hold results and control parameters;

3. There is no initialization of functions, or at most a short initialization chain;

4. Methods are placed at both the module level and within classes so that the methods and their documentation can be easily accessed by the interactive shell (see Figure 7).

A practical solution to eliminate halting of the interactive shell is to overlap the REPL (read-eval-print-loop) and task execution. Such task parallelism requires the underlying tasks to be independent of each other. Although certain dependence between methods is inevitable, the above design rules greatly reduce function call dependence. Most functions in PySCF can be safely placed in the background using the standard Python `threading` and `multiprocessing` libraries.

3.3 Methods as plugins

Ease-of-use is the primary design objective of the PySCF package. However, function simplicity and versatility are difficult to balance in the same software framework. To balance readability and complexity, we have implemented only the basic algorithmic features in the main methods, and placed advanced features in additional "plugins". For instance, the main mean-field module implements only the basic self-consistent loop. Corrections (such as for relativistic effects) are implemented in an independent plugin module, which

can be activated by reassigning the mean-field 1-electron Hamiltonian method at runtime. Although this design increases the complexity of implementation of the plugin functions, the core methods retain a clear structure and are easy to comprehend. Further, this approach decreases the coupling between different features: for example, independent features can be modified and tested independently and combined in calculations. In the package, this plugin design has been widely used, for example, to enable molecular point group symmetry, relativistic corrections, solvation effects, density fitting approximations, the use of second-order orbital optimization, different variational active space solvers, and many other features (Figure 8).

3.4 Seamless MPI functionality

The Message Passing Interface (MPI) is the most popular parallel protocol in the field of high performance computing. Although MPI provides high efficiency for parallel programming, it is a challenge to develop a simple and efficient MPI program. In compiled languages, the program must explicitly control data communication according to the MPI communication protocol. The most common design is to activate MPI communication from the beginning and to update the status of the MPI communicator throughout the program. When developing new methods, this often leads to extra effort in code development and debugging. To sustain the simplicity of the PySCF package, we have designed a different mechanism to execute parallel code with MPI. We use MPI to start the Python interpreter as a daemon to receive both the functions and data on the remote nodes. When a parallel session is activated, the master process sends to the remote Python daemons both the functions and the data. The function is decoded remotely and then executed. This design allows one to develop code mainly in serial mode and to switch to the MPI mode only when high performance is required. Figure 9 shows an example to perform a periodic calculation with and without a parallel session. Comparing to the serial mode invocation, we see that the user only has to change the density fitting object to acquire parallel functionality.

4 CONCLUSIONS

Python and its large collection of third party libraries are helping to revolutionize how we carry out and implement numerical simulations. It is potentially much more productive to solve computational problems within the Python ecosystem because it frees researchers to work at the highest level of abstraction without worrying about the details of complex software implementation. To bring all the benefits of the Python ecosystem to quantum chemistry and electronic structure simulations, we have started the open-source PySCF project.

PySCF is a simple, lightweight, and efficient computational chemistry program package, which supports ab initio calculations for both molecular and extended systems. The package serves as an extensible electronic structure toolbox, providing a large number of fundamental operations with simple APIs to manipulate methods, integrals, and wave functions. We have invested significant effort to ensure simplicity of use and implementation while preserving competitive functionality and performance. We believe that this package represents a new style of program and library design that will be representative of future software developments in the field.

5 ACKNOWLEDGMENTS

QS would like to thank Junbo Lu and Alexander Sokolov for testing functionality and for useful suggestions for the program package. The development of different components of the PySCF package has been generously supported by several sources. Most of the molecular quantum chemistry software infrastructure was developed with support from the US National Science Foundation, through grants CHE-1650436 and ACI-1657286. The periodic mean-field infrastructure was developed with support from ACI-1657286. The excited-state periodic coupled cluster methods were developed with support from the US Department of Energy, Office of Science, through the grants de-sc0010530 and de-sc0008624. Additional support for the extended-system methods has been provided by the Simons Foundation through the Simons Collaboration on the Many Electron Problem, a Simons Investigator-

Table 1: Features of the PySCF package as of the 1.3 release.

Method	Molecule	Solids	Comments
HF	Yes	Yes	$\sim$ 5000 AOs[b]
MP2	Yes	Yes	$\sim$ 1500 MOs[b]
CCSD	Yes	Yes	$\sim$ 1500 MOs[b]
EOM-CCSD	Yes	Yes	$\sim$ 1500 MOs[b]
CCSD(T)	Yes	Yes[a]	$\sim$ 1500 MOs[b]
MCSCF	Yes	Yes[a]	$\sim$ 3000 AOs;[b] 30–50 active orbitals[c]
MRPT	Yes	Yes[a]	$\sim$ 1500 MOs;[b] 30–50 active orbitals[c]
DFT	Yes	Yes	$\sim$ 5000 AOs[b]
TDDFT	Yes	Yes[a]	$\sim$ 5000 AOs[b]
CISD	Yes	Yes[a]	$\sim$ 1500 MOs[b]
FCI	Yes	Yes[a]	$\sim$ (18e, 18o)[b]
Localizer	Yes	No	IAO, NAO, meta-Löwdin Boys, Edmiston-Ruedenberg, Pipek-Mezey
Relativity	Yes	No	ECP and scalar-relativistic corrections for all methods 2-component, 4-component methods for HF and MP2
Gradients	Yes	No	HF, DFT, CCSD, CCSD(T), TDDFT
Hessian	Yes	No	HF and DFT
Properties	Yes	No	non-relativistic, 4-component relativistic NMR
Symmetry	Yes	No	D_{2h} and subgroup
AO, MO integrals	Yes	Yes	1-electron, 2-electron integrals
Density fitting	Yes	Yes	HF, DFT, MP2

[a] Only available for Γ-point calculations;

[b] An estimation based on a single SMP node with 128 GB memory without density fitting;

[c] Using an external DMRG or FCIQMC program as active space solver.

```
from pyscf import gto, dft
mol = gto.Mole(atom='N 0 0 0; N 0 0 1.1', basis='ccpvtz')
mf = dft.RKS(mol)
mf.xc = '0.2*HF + 0.08*LDA + 0.72*B88, 0.81*LYP + 0.19*VWN'
mf.kernel()
```

Figure 1: Example to define a custom exchange-correlation functional for a DFT calculation.

```
from pyscf import gto, scf, mcscf, dmrgscf
mol = gto.Mole(atom='N 0 0 0; N 0 0 1.1', basis='ccpvtz')
mf = scf.RHF(mol).run()
mc = mcscf.CASSCF(mf, 8, 10)  # 8o, 10e
mc.fcisolver = dmrgscf.DMRGCI(mol)
mc.kernel()
```

Figure 2: Example to enable the DMRG solver in a CASSCF calculation.

```
from pyscf import gto, scf, lo, tools
mol = gto.Mole(atom=open('c60.xyz').read(),
               basis='ccpvtz')
mf = scf.RHF(mol).run()
orb = lo.Boys(mol).kernel(mf.mo_coeff[:,:180])
tools.molden.from_mo(mol, 'c60.molden', orb)

# Invoke Jmol to plot the orbitals
with open('c60.spt', 'w') as f:
  f.write('load c60.molden; isoSurface MO 002;\n')
import os
os.system('jmol c60.spt')
```

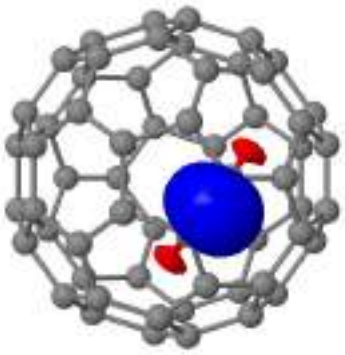


Figure 3: Example to generate localized orbitals and to plot them in Jmol.

```
from pyscf import gto
mol = gto.Mole(atom='N 0 0 0; N 0 0 1.1', basis='ccpvtz')
a = mol.intor('cint1e_nuc_sph')  # nuclear attraction as a giant array
a = mol.intor('cint2e_sph')      # 2e integrals as a giant array
a = mol.intor_by_shell('cint2e_sph', (0,0,0,0))  # (ss|ss) of first N atom
```

Figure 4: Example to access AO integrals.

```
# 10-site Hubbard model at half-filling with U/t = 4
import numpy as np
from pyscf import gto, scf, ao2mo, cc
mol = gto.Mole(verbose=4)
mol.nelectron = n = 10
t, u = 1., 4.

mf = scf.RHF(mol)
h1 = np.zeros((n,n))
for i in range(n-1):
    h1[i,i+1] = h1[i+1,i] = t
mf.get_hcore = lambda *args: h1
mf.get_ovlp = lambda *args: np.eye(n)
mf._eri = np.zeros((n,n,n,n))
for i in range(n):
    mf._eri[i,i,i,i] = u
# 2e Hamiltonian in 4-fold symmetry
mf._eri = ao2mo.restore(4, mf._eri, n)
mf.run()
cc.CCSD(mf).run()
```

Figure 5: Example to use a custom Hamiltonian.

```
import numpy as np
from pyscf import gto, scf
bond = np.arange(0.8, 5.0, .1)
dm_init = None
e_hf = []
for r in reversed(bond):
  mol = gto.Mole(atom=[['N', 0, 0, 0],
                       ['N', 0, 0, r]],
                 basis='ccpvtz')
  mf = scf.RHF(mol).run(dm_init)
  dm_init = mf.make_rdm1()
  e_hf.append(mf.e_tot)

from matplotlib import pyplot
pyplot.plot(bond, e_hf[::-1])
pyplot.show()
```

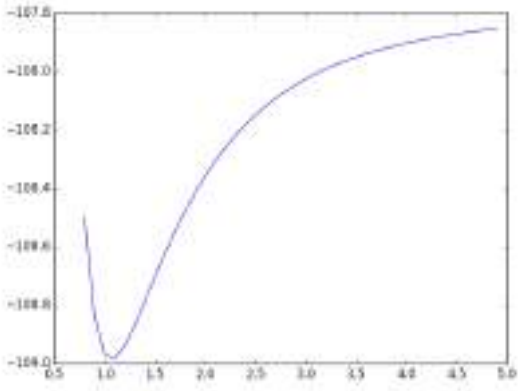


Figure 6: Using Python to combine the calculation and data post-processing in one script.

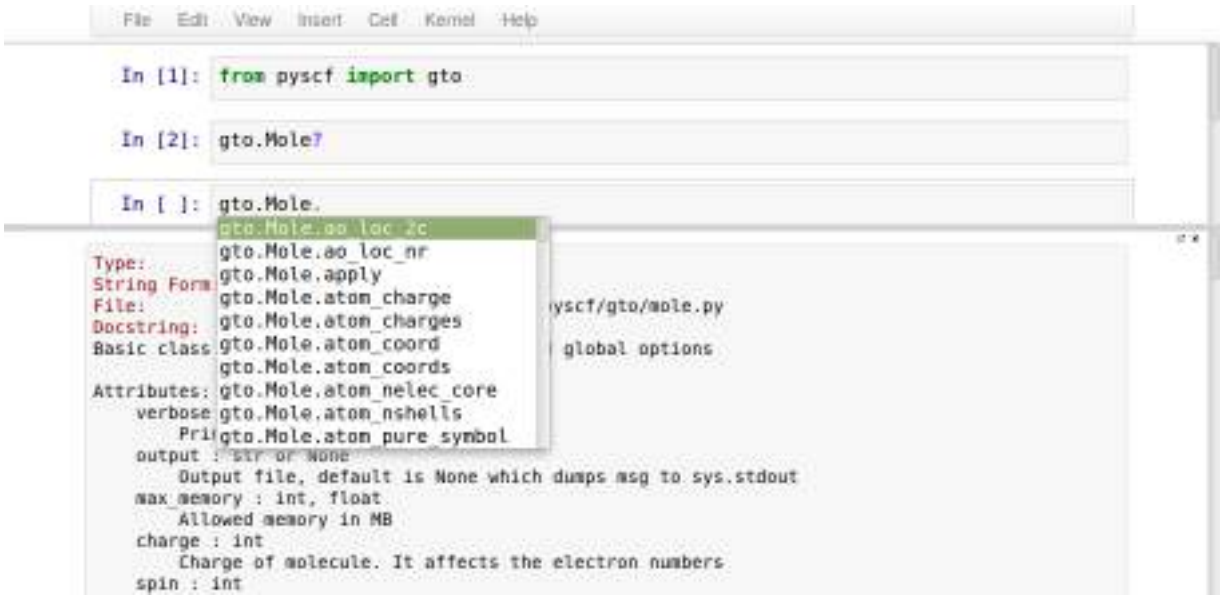


Figure 7: Accessing documentation within the IPython shell. The question mark activates the documentation window in the bottom area. The pop-up menu for code auto-completion is triggered by the <Tab> key.

```
from pyscf import gto, scf
mol = gto.Mole(atom='N 0 0 0; N 0 0 1.1', basis='ccpvtz')
mf = scf.newton(scf.sfx2c(scf.density_fit(scf.RHF(mol)))).run()
```

Figure 8: Example to use plugins in PySCF. The mean-field calculation is decorated by the density fitting approximation, X2C relativistic correction and second order SCF solver.

```python
# Serial mode
# run in cmdline:
# python input.py
from pyscf.pbc import gto, scf
from pyscf.pbc import df
cell = gto.Cell()
cell.atom = 'H 0 0 0; H 0 0 0.7'
cell.basis = 'ccpvdz'
# unit cell lattice vectors
cell.a = '2 0 0; 0 2 0; 0 0 2'
# grid for numerical integration
cell.gs = [10,10,10]
mf = scf.RHF(cell)
mf.with_df = df.DF(cell)
mf.kernel()
```

```python
# MPI mode
# run in cmdline:
# mpirun -np 4 python input.py
from pyscf.pbc import gto, scf
from mpi4pyscf.pbc import df
cell = gto.Cell()
cell.atom = 'H 0 0 0; H 0 0 0.7'
cell.basis = 'ccpvdz'
# unit cell lattice vectors
cell.a = '2 0 0; 0 2 0; 0 0 2'
# grid for numerical integration
cell.gs = [10,10,10]
mf = scf.RHF(cell)
mf.with_df = df.DF(cell)
mf.kernel()
```

Figure 9: Comparison of the input script for serial-mode and MPI-mode calculations. Except for the module to import, the MPI parallel mode takes exactly the same input as the serial mode.